

République Algérienne Démocratique et Populaire
الجمهورية الجزائرية الديمقراطية الشعبية
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
وزارة التعليم العالي و البحث العلمي



ECOLE NATIONALE
SUPÉRIEURE
D'INFORMATIQUE

المدرسة الوطنية العليا للإعلام الآلي
(المعهد الوطني للتكوين في الاعلام الآلي سابقا)
École nationale Supérieure d'Informatique
ex. INI (Institut National de formation en Informatique)

Engineering Dissertation

Submitted for the Degree of Computer Science Engineer

Option: Computer systems and Software engineering

Explainable Artificial Intelligence For Deep Time Series Models

Written by:

BENKHELIFA Bouchra

Supervised by:

Dr BOUZEGHOUB Amel (Samovar Télécom SudParis)

ZAOUIDI Mohamed (Samovar Télécom SudParis)

Dr CHARABI Leila (ESI)

Academic Year 2025-2026

Dedication

Acknowledgement

Abstract

Résumé

Contents

Dedication	1
Acknowledgement	2
Abstract	3
Résumé	4
General Introduction	15
I State of the art	16
1 Time Series	17
Introduction	17
1.1 Definition	17
1.2 Types of Time Series	17
1.3 Components of Time Series	18
1.4 Time Series Decomposition Methods	18
1.5 Objectives and Applications of Time Series Analysis	19
1.6 Classical Forecasting Methods	20
1.6.1 Stationarity: A Prerequisite for Classical Methods	20
1.7 Traditional Time Series Analysis Pipeline	21
1.8 The Temporal Data Manifold	22
1.9 Temporal Dependencies	22
1.10 Mutable and Immutable Observations	23
Synthesis	24
Conclusion	24
2 Deep Learning for Time Series	25
Introduction	25
2.1 Challenges in Time Series Modeling	25
2.2 Architectural Primitives for Deep Time Series Modeling	25
2.3 Classical Deep Learning Architectures for Time Series	26
2.3.1 Multilayer Perceptrons (MLPs)	26
2.3.2 Recurrent Neural Networks (RNNs)	26
2.3.3 Convolutional Neural Networks (CNNs)	27
2.3.4 Graph Neural Networks (GNNs)	28
2.4 Advanced Architectures: Transformers for Time Series	29

2.4.1	Transformer Architecture	29
2.4.2	Tokenization Strategies in Transformer-Based Time Series Models	30
2.5	Advanced Transformer Architectures for Long-Term Time Series Forecasting	31
2.5.1	Informer	31
2.5.2	Autoformer	32
2.5.3	FEDformer	32
2.5.4	PatchTST	32
2.5.5	Crossformer	33
2.5.6	Inverted Transformer (iTransformer)	33
	Conclusion	34
3	Explainable Artificial Intelligence for Time Series Models	37
3.1	Introduction	37
	Introduction	37
3.2	Conceptual Definitions	38
3.2.1	Explainable Artificial Intelligence (XAI)	38
3.2.2	Explicability	38
3.2.3	Explainability	38
3.2.4	Interpretable Models vs Explainable Models	38
3.2.5	Transparent models	38
3.3	Objectives and Key Properties of Explainable AI	39
3.4	Taxonomy of XAI Methods	40
3.4.1	Scale of Explanations	40
3.4.2	Stage of Explanations	41
3.4.3	Model Architecture Dependency	41
3.5	Dimensions of Explainability for Time Series Models	43
3.5.1	Feature Relevance	43
3.5.2	Time Relevance	43
3.5.3	Frequency Relevance	43
3.6	Mechanism-Based Taxonomy of Transformer Explanations	44
3.6.1	Perturbation-based Methods	44
3.6.2	Gradient-based Methods	44
3.6.3	Activation-based Methods	44
3.6.4	Attention-based Methods	44
3.6.5	Hybrid Methods	44
3.7	XAI methods for Time Series	45
3.7.1	LIME	45
3.7.2	SHAP	45
3.7.3	ForecastCF (CounterFactual)	45
3.7.4	Dynamic Mask	45
3.7.5	ShapTime	46
3.7.6	ShapTST	46
3.7.7	Layer-Wise Relevance Propagation (LRP)	46
3.7.8	LRP with Virtual Inspection Layer (VIL)	47
3.7.9	Attention Rollout	47
3.7.10	Attention Flow	48
3.7.11	NeuroSymbolic Time Series Classification	48

3.8	Comparison of XAI Methods for Time Series	48
3.9	Evaluation Methods for XAI Explanations on Time Series	50
3.9.1	Perturbation-Based Methods	50
3.9.2	Sequence-Aware Methods	51
	Synthesis	51
	Conclusion	51
4	Counterfactual Explanations for Time Series	52
	Introduction	52
4.1	Definition and Formulation of Counterfactual Explanations	53
4.2	Motivation and Advantages of Counterfactual Explanations	53
4.3	Counterfactual Explanations for Time Series	53
4.3.1	Counterfactuals for Classification	54
4.3.2	Counterfactuals for Forecasting	55
4.4	Taxonomy of Counterfactual generation Methods	55
4.4.1	Optimization-Based Methods	55
4.4.2	Generative Model-Based Methods	55
4.4.3	Instance-Based Methods	56
4.4.4	Learning-Based Methods	56
4.5	Evaluation Metrics for Counterfactual Explanations	57
4.6	Challenges and Trade-offs in Counterfactual Explanations	58
4.6.1	The Challenge of Global Perturbation and Sparsity	58
4.6.2	Summary of Optimization Challenges	58
4.7	Positioning of Counterfactual Explanations within XAI	59
	Research Gap	60
II	Contribution	62
	Introduction	63
5	Methodology	64
	Introduction	64
5.1	Problem Formulation	65
5.2	Baseline Approaches and Limitations	65
5.2.1	ForecastCF: Gradient-Based Counterfactual Forecasting	65
5.2.2	Reinforcement Learning for Counterfactual Classification	66
5.3	Contributions	66
5.4	Framework Overview	67
5.5	Framework Components	68
5.5.1	TCN Autoencoder	68
5.5.2	Black-Box Forecasting Model	69
5.5.3	Actor-Critic Agent	69
5.5.4	Temporal Masking Mechanism	70
5.5.5	Reward Function	71
5.6	Training and Inference Procedures	71
5.6.1	Training Procedure	71
5.6.2	Inference Procedure	72

5.7	Evaluation Metrics	72
5.7.1	Validity Ratio	72
5.7.2	Stepwise Validity AUC	73
5.7.3	Proximity	73
5.7.4	Compactness	73
5.7.5	Roughness Ratio	73
5.7.6	Temporal Consistency	73
5.7.7	Plausibility	74
6	Realization	75
	Introduction	75
6.1	Development Environment	75
6.2	Datasets	76
6.2.1	ETTh1	76
6.2.2	ETTh2	77
6.2.3	Weather	77
6.2.4	Synthesis	79
6.2.5	Data Preparation	79
6.3	Forecasting Model Training	81
6.3.1	Model Overview	81
6.3.2	Observation Window and Forecasting Horizon Selection	81
6.3.3	Configuration	83
6.3.4	Implementation	83
6.3.5	Results Summary	84
6.3.6	Reproducing the Results	85
6.4	TCN Autoencoder Training	85
6.4.1	Architecture and Implementation	85
6.4.2	Configuration	86
6.4.3	Results and Analysis	87
6.4.4	Reproducing the Results	88
6.5	Anomaly Detector Training	88
6.5.1	Motivation and Role in the Framework	88
6.5.2	Implementation	88
6.5.3	Configuration	89
6.5.4	Baseline Plausibility Scores	89
6.6	Reinforcement Learning Agent and Counterfactual Generation	90
6.6.1	Actor-Critic Module	90
6.6.2	Forecast Bound Computation	91
6.6.3	Reward Function	92
6.6.4	Temporal Mask	92
6.6.5	Full Generation Pipeline	93
6.6.6	Hyperparameter Grid Search	94
6.6.7	Reward Weight Sensitivity	94
6.6.8	Sensitivity to Band Width f_r	96
6.6.9	Sensitivity to Gap Parameter ρ	97
6.6.10	Configuration	98
6.6.11	Results and Analysis	99

6.6.12 Reproducing the Results	99
Conclusion	100
7 Experiments and Results	101
Introduction	101
7.1 Experimental Setup	101
7.1.1 Baselines	101
7.1.2 Evaluation Protocol	102
7.2 Comparison with Baselines	102
7.3 Evaluation Across Forecasting Architectures	103
7.4 Ablation Study	104
7.4.1 Overview	104
7.4.2 Effect of the Learned RL Policy (RLP)	104
7.4.3 Effect of Temporal Masking (w/o Mask)	105
7.4.4 Effect of the Proximity Reward Term (w/o Proximity)	106
7.5 Per-Dataset Visual Evaluation	107
7.5.1 ETTh1	107
7.5.2 ETTh2	108
7.5.3 Weather	109
Conclusion	110
General Conclusion	111
Bibliography	112

List of Figures

1.1	Time Series Components	18
1.2	Comparison of Additive and Multiplicative Time Series Decomposition.	19
1.3	Stationary vs. non-stationary time series.	21
1.4	Time Series Analysis Pipeline.	21
1.5	The temporal data manifold $\mathcal{M} \subset \mathbb{R}^L$. An observed series $\mathbf{x}$ lies on $\mathcal{M}$. A modified series $\mathbf{x}'$ is only meaningful if it remains on $\mathcal{M}$; arbitrary perturbations in $\mathbb{R}^L$ produce implausible sequences outside it.	22
1.6	Mutable and immutable regions of a look-back window of length L . Only the k most recent observations are actionable; modifying earlier timesteps produces a logically incoherent alternative series.	23
2.1	MLP Architecture for Time Series	26
2.2	RNN Architecture for Time Series	27
2.3	CNN Architecture for Time Series	27
2.4	Architectural details of TCN: (a) Dilated causal convolutions with factors $d = 1, 2, 4$ (adapted from Figure 1 in Wan et al. (2019)). (b) Residual block structure including Weight Normalization and ReLU activation (based on Figure 2 in Wan et al. (2019)). . .	28
2.5	GNN Architecture for Time Series	28
2.6	The Transformer architecture Vaswani et al. (2017).	30
2.7	Tokenization strategies for Transformer-based time series models.	31
2.8	Informer model overview H. Zhou et al. (2021)	31
2.9	Autoformer model overview Wu et al. (2021)	32
2.10	FEDformer model overview T. Zhou et al. (2022)	32
2.11	PatchTST architecture: patch-based tokenization with channel-independent Transformer encoders Nie et al. (2023).	33
2.12	Crossformer architecture: DSW embedding and Two-Stage Attention (TSA) across time and variate dimensions Yunhao Zhang and Yan (2023).	33
2.13	iTransformer architecture overview showing the inverted embedding process (adapted from Y. Liu et al. (2023)).	34
2.14	Performance comparison of iTransformer against baseline models on various benchmarks.	34
3.1	Knowledge graph relating the purposes of explainability methods Rojat et al. (2021). . . .	39
3.2	Multi-Criteria Taxonomy of XAI Methods.	40
3.3	Transformers Explanations Mechanisms	44
3.4	ShapTST's Architecture	46
3.5	LRP with Virtual Inspection Layer Vielhaben et al. (2024).	47
3.6	Time series Evaluation Startegies	50

4.1	Taxonomy of Explainable AI Methods with Emphasis on Counterfactual Explanations . .	52
4.2	Example of a counterfactual explanation for time series classification. The original signal (blue) is modified to generate a counterfactual instance (Red) that changes the predicted class.	54
4.3	Counterfactual forecasting under bounded target constraints	55
5.1	Overview of the RL-MCF framework for counterfactual forecasting. The Actor-Critic agent operates in the latent space of a pre-trained TCN autoencoder, guided by a reward signal that balances validity and proximity. The temporal masking mechanism enforces sparse and localized perturbations on the most recent timesteps of the input sequence.	68
6.1	STL decomposition of the ETTh1 target variable (OT, period = 24 h). Trend ($\approx 86\%$), seasonal ($\approx 7\%$), residual ($\approx 6\%$). Green vertical line: seq_len = 96; orange vertical line: pred_len = 48.	77
6.2	STL decomposition of the ETTh2 target variable (OT, period = 24 h). Trend ($\approx 65\%$), seasonal ($\approx 20\%$), residual ($\approx 10\%$). The stronger seasonal share and larger residual make ETTh2 harder than ETTh1.	78
6.3	STL decomposition of the Weather target variable (WetBulbCelsius, period = 24 h = 144 steps at 10-minute resolution). Trend ($\approx 76\%$), seasonal ($\approx 2.3\%$), residual ($\approx 13\%$). The near-absent seasonality and large residual distinguish this dataset from the ETT series. .	79
6.4	Autocorrelation function (ACF) of the target variable for each dataset after linear detrending. ETTh1 and ETTh2 (top, middle): dominant peak at lag ≈ 24 (24-hour daily cycle, 1 step = 1 hour). Weather (bottom): dominant peak at lag ≈ 144 (24-hour cycle, 1 step = 10 minutes). Green vertical line: seq_len = 96; orange vertical line: pred_len = 48. . .	82
6.5	iTransformer forecasting results on the test set for ETTh1, ETTh2, and Weather. The predicted series (dashed orange) closely tracks the ground truth (solid blue) across all three datasets, with error increasing from ETTh1 to Weather, consistent with their residual variance profiles (Table 6.3).	85
6.6	TCN autoencoder reconstruction on the test set for ETTh1, ETTh2, and Weather. The reconstructed series (dashed red) closely tracks the original (solid blue) across all three datasets, recovering both low-frequency trends and higher-frequency oscillations. MSE values are reported per dataset.	87
6.7	Plausibility score distributions for the three anomaly detectors (Isolation Forest, Local Outlier Factor, One-Class SVM) on ETTh1. Train (green) and Test (blue) distributions are concentrated near zero, confirming that realistic sequences are correctly identified as inliers. The Anomalous distribution (red) is clearly shifted towards higher scores, validating that the ensemble has learned a meaningful boundary of the training distribution. These reference distributions anchor the interpretation of plausibility scores reported in Chapter 7.	90
6.8	Reward weight sensitivity on ETTh1. Left: effect of w_{val} ($w_{\text{prox}} = 1.5$ fixed). Right: effect of w_{prox} ($w_{\text{val}} = 3.5$ fixed). The optimal configuration ($w_{\text{val}} = 3.5$, $w_{\text{prox}} = 1.5$) achieves the best trade-off between validity and proximity.	96
6.9	Validity Ratio vs. f_r (ETTh1, iTransformer). VR increases monotonically; star marks $f_r = 0.5$ (VR=0.976).	97
6.10	Plausibility vs. ρ . Higher ρ degrades plausibility; star marks $\rho = 0.2$ (score=0.169).	98
6.11	Validity-Proximity trade-off vs. ρ . $\rho = 0.2$ (star) lies at the knee of the curve.	98

6.12	Counterfactual example on ETTh1 (iTransformer, $\rho = 0.2$, $f_r = 0.5$, $k = 12$). Blue: original input and forecast. Orange dashed: counterfactual input and modified forecast. Green shaded: target band $[\alpha, \beta]$. The modification is confined to the last $k = 12$ timesteps; the modified forecast successfully enters the target band. Sample validity ratio: 0.96. . . .	99
7.1	Qualitative example on a single ETTh1/iTransformer test sample ($\rho=0.2$, $f_r=0.5$). Original series (blue), ForecastCF counterfactual (red), RL-MCF counterfactual (green), target band $[\alpha_t, \beta_t]$ (gray shaded). For this sample: RL-MCF achieves VR = 96% vs. ForecastCF VR = 46%. Dataset-level averages are reported in Table 7.1.	103
7.2	Qualitative comparison of counterfactual series generated by RL-MCF, RLP, and the w/o-mask ablation on a representative ETTh1 test sample (iTransformer backbone).	105
7.3	Radar chart comparing the full model and the no-proximity ablation across five evaluation dimensions on ETTh1. Proximity and Plausibility are displayed as $1 - \text{score}$ so that a larger area consistently indicates better performance. The full model (blue) dominates on all axes, most notably on Validity and Proximity.	105
7.4	Qualitative examples on ETTh1. Per-sample validity: 0.94, 0.98, 1.00, 0.94.	107
7.5	ETTh1: RL-MCF across 5 forecasting architectures.	107
7.6	Qualitative examples on ETTh2. Per-sample validity: 0.96, 0.88, 0.69, 0.17.	108
7.7	ETTh2: RL-MCF across 5 forecasting architectures.	108
7.8	Qualitative examples on Weather. Per-sample validity: 1.00, 1.00, 1.00, 1.00.	109
7.9	Weather: RL-MCF across 5 forecasting architectures.	109

List of Tables

2.1	Comparison of Deep Learning Architectures for Time Series	29
2.2	Comparative analysis of Transformer architectures for time series forecasting.	36
3.1	Comparative overview of XAI methods for time series.	49
6.1	Libraries and frameworks used throughout the project.	75
6.2	Summary of datasets used in the experiments.	76
6.3	Synthesis of STL decomposition results and modeling implications.	79
6.4	Dataset split sizes after windowing ($L = 96$, $H = 48$).	80
6.5	Summary of the five forecasting architectures trained in this work.	81
6.6	Observation window and forecasting horizon for each dataset.	83
6.7	Shared training configuration for all forecasting models.	83
6.8	iTransformer forecasting performance on the test set.	84
6.9	TCN autoencoder reconstruction performance on the test set.	87
6.10	Anomaly detector configuration.	89
6.11	Grid search results on ETTh1 (RL counterfactual generation).	95
6.12	Impact of w_{val} ($w_{\text{prox}} = 1.5$ fixed, ETTh1).	96
6.13	Impact of w_{prox} ($w_{\text{val}} = 3.5$ fixed, ETTh1).	97
6.14	RL-MCF configuration for ETTh1 with iTransformer.	98
6.15	RL-MCF results on ETTh1 (iTransformer backbone).	99
7.1	Quantitative comparison across datasets, methods and forecasting architectures. Bold = best per column per dataset. $\downarrow$ lower is better (Prox., Plaus.); $\uparrow$ higher is better (Valid., AUC, Comp., T-Cons.).	102
7.2	Performance of RL-MCF across datasets and forecasting architectures. Bold indicates best per dataset.	104
7.3	Ablation study: quantitative comparison on ETTh1 (iTransformer backbone). Best values in bold.	104

List of Algorithms

1	RL-MCF: Training Procedure	72
2	RL-MCF: Inference Procedure	72

General Introduction

Part I

State of the art

Chapter 1

Time Series

Introduction

Time series data are central to a wide range of scientific and industrial domains, from energy management and financial forecasting to healthcare monitoring and climate analysis. Understanding, modelling, and forecasting such data requires a careful account of the structural properties that make temporal data fundamentally different from tabular or image data.

This chapter lays the conceptual groundwork for the rest of the thesis. It opens with the classical foundations of time series analysis: definitions, components, decomposition methods, and the statistical forecasting models that have long dominated the field. Their limitations are then examined, revealing why modern deep learning architectures became necessary. Building on this, the chapter introduces deeper structural constraints of real-world series that classical methods fail to capture, and that any model operating on temporal data must respect.

1.1 Definition

A time series is a sequence of observations collected over time, typically at regular intervals such as hourly or daily. It captures the temporal evolution of one or more variables, and its analysis aims to model, understand, and forecast that evolution Alsudani and J. Liu (2017).

Unlike tabular data, where observations are assumed to be independent, time series observations are intrinsically ordered and statistically dependent on one another. This temporal structure is not a nuisance to be corrected — it is the defining characteristic of the data, and any model must account for it.

1.2 Types of Time Series

Time series can be classified according to their dimensionality Chudo and Terdik (2025):

- **Univariate time series:** a single variable is measured over time. The analysis focuses on its internal temporal structure — trend, seasonality, and autocorrelation.
- **Multivariate time series:** multiple variables are tracked simultaneously. Beyond the temporal structure of each individual series, the analysis must also account for cross-variable dependencies, which may be nonlinear, time-varying, or lagged.

1.3 Components of Time Series

Time series data are commonly decomposed into constituent components that reveal the underlying structure of the signal (see Figure ??):

- **Trend:** the long-term movement of the series, reflecting a persistent increase or decrease over time (yellow line).
- **Seasonality:** repeating patterns at fixed, known intervals, such as daily or yearly cycles (green line).
- **Residual (Noise):** the random variation that remains after the trend and seasonal components have been accounted for (red line).
- **Cyclic variations:** fluctuations occurring over non-fixed, longer periods, driven by economic or environmental cycles rather than calendar regularity.

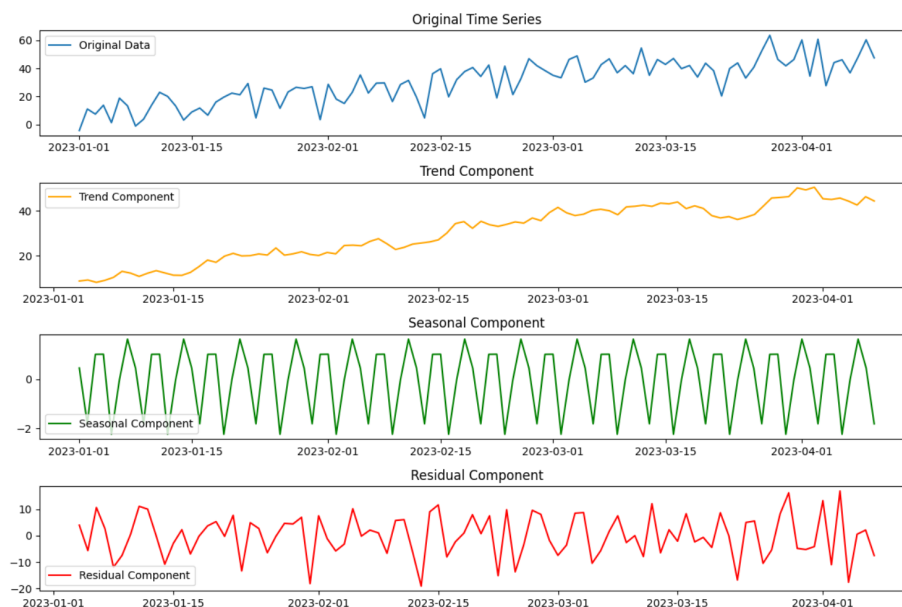


Figure 1.1: Time Series Components

Understanding these components is the first step toward selecting appropriate modeling strategies, as different forecasting methods are designed to capture different subsets of these patterns in Figure 1.1

1.4 Time Series Decomposition Methods

Time series decomposition separates a series into its trend, seasonal, and residual components. Two classical approaches are widely used: additive and multiplicative decomposition. The choice between them depends on whether seasonal fluctuations remain approximately constant in magnitude or scale proportionally with the level of the series.

In **additive decomposition**, components combine as:

$$y_t = T_t + S_t + R_t$$

In **multiplicative decomposition**, they combine as:

$$y_t = T_t \times S_t \times R_t$$

where T_t , S_t , and R_t denote the trend, seasonal, and residual components at time t , respectively. Additive models are appropriate when seasonal effects remain roughly constant regardless of the level of the series. Multiplicative models are preferred when the amplitude of seasonal fluctuations grows with the trend, as is common in economic or energy data Kreuzer, Zdravkovic, and Papapetrou (2025).

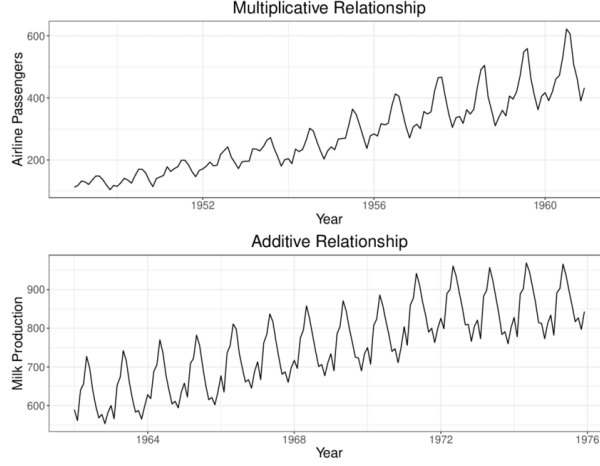


Figure 1.2: Comparison of Additive and Multiplicative Time Series Decomposition.

As shown in Figure 1.2, the additive model maintains a constant seasonal amplitude, while the multiplicative model scales it with the trend. These decomposition frameworks are foundational to time series analysis, and their logic is embedded as inductive biases in several deep learning architectures reviewed in Chapter 2.

1.5 Objectives and Applications of Time Series Analysis

The analysis of time series serves three principal objectives in practice:

1. **Classification:** assigning an input sequence to a discrete category based on learned temporal patterns. Given a sequence $\mathbf{x} \in \mathbb{R}^L$, a classifier $g : \mathbb{R}^L \rightarrow \{1, \dots, C\}$ maps it to a label.
Example: classifying heart rate patterns as normal or arrhythmic, recognizing daily activity patterns from wearable sensors.
2. **Anomaly Detection:** identifying observations or subsequences that deviate significantly from the expected statistical structure of the series. A sequence is flagged as anomalous when it falls outside the distributional boundaries learned from normal behavior.
Example: detecting a sudden drop in server performance, spotting unusual spikes in bank transactions.
3. **Forecasting:** predicting future values from past observations. A forecasting model $f : \mathbb{R}^L \rightarrow \mathbb{R}^H$ maps a look-back window of length L to a continuous trajectory of H future values:

$$\hat{\mathbf{y}} = f(\mathbf{x}), \quad \hat{\mathbf{y}} \in \mathbb{R}^H$$

Example: estimating next week's electricity demand from historical usage, forecasting next quarter's sales.

Among these objectives, forecasting is the most demanding. Unlike classification, which reduces a sequence to a single discrete label, forecasting requires the model to produce a coherent future trajectory — a continuous, multi-step output that must remain physically consistent with the observed past. This consistency is only possible because real-world series are not arbitrary sequences: they are constrained by the underlying physical process that generated them, a property examined in depth later in this chapter.

1.6 Classical Forecasting Methods

Before the rise of deep learning, time series forecasting relied on a family of statistical models built around explicit assumptions about the structure of the data. These methods remain relevant as interpretable baselines and as conceptual references for understanding what deep learning models are designed to overcome. The principal families are:

- **Moving Average (MA):** smooths observations by averaging a fixed window of recent values, suppressing short-term noise. Effective only for series without strong trend or seasonality.
- **ARIMA(p, d, q):** combines autoregressive terms (dependence on past values), differencing (to enforce stationarity), and moving-average error terms. The dominant classical model for univariate forecasting.
- **SARIMA(p, d, q)(P, D, Q) _{s} :** extends ARIMA with seasonal autoregressive, integration, and moving-average terms at period s , handling recurring seasonal cycles.
- **Exponential Smoothing (SES, Holt, Holt-Winters):** produces forecasts as exponentially weighted averages of past observations. Holt-Winters additionally models trend and seasonality explicitly.

1.6.1 Stationarity: A Prerequisite for Classical Methods

A fundamental assumption underlying all classical forecasting methods is **stationarity**: a time series is weakly stationary if its mean, variance, and autocovariance are time-invariant:

$$\mathbb{E}(x_t) = \mu, \quad \text{Var}(x_t) = \sigma^2, \quad \text{Cov}(x_t, x_{t-\tau}) = c_\tau$$

If statistical properties drift over time, past behaviour provides an unreliable basis for prediction. Classical models such as ARIMA and exponential smoothing are only valid when applied to a stationary series — or one that has been transformed into one.

Real-world series are rarely stationary. Standard remedies include:

- **Differencing:** $Y'_t = Y_t - Y_{t-1}$ removes linear trends by computing consecutive differences.
- **Detrending:** subtracting a fitted polynomial function of time from the series.
- **Variance-stabilising transformations:** logarithmic or square-root scaling to stabilise heteroscedastic variance.

Once stationarised, the series is modelled using the appropriate classical method, and a back-transformation is applied to restore the original scale. While these approaches are interpretable and computationally inexpensive, they operate under two restrictive assumptions: linearity and stationarity. These assumptions make them ill-suited to the complex, high-dimensional, and non-stationary datasets encountered in modern forecasting tasks, as discussed in the following section.

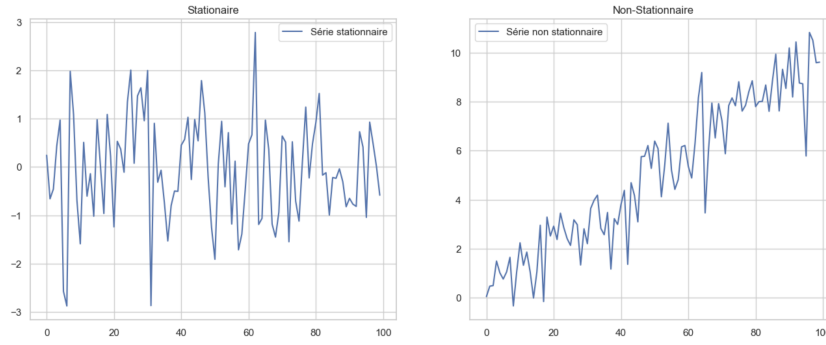


Figure 1.3: Stationary vs. non-stationary time series.

1.7 Traditional Time Series Analysis Pipeline

The classical analysis pipeline organises the steps described above into a structured sequence, illustrated in Figure 1.4:

1. **Exploration / Inspection:** visual inspection of the series using time plots and summary statistics such as mean, variance, and autocorrelation. This step identifies the presence of trend, seasonality, and potential anomalies, and informs all subsequent modelling choices.
2. **Decomposition:** the series is separated into its structural components — trend, seasonal, and residual — using either additive or multiplicative decomposition, depending on the characteristics identified in the inspection step.
3. **Stationarity Check and Transformation:** formal tests such as the Augmented Dickey-Fuller test are applied to verify stationarity. If the series is non-stationary, appropriate transformations are applied before modelling.
4. **Modelling and Forecasting:** a suitable forecasting model is selected and fitted to the transformed series. The fitted model is then used to generate predictions for future time points, and the output is back-transformed to the original scale.

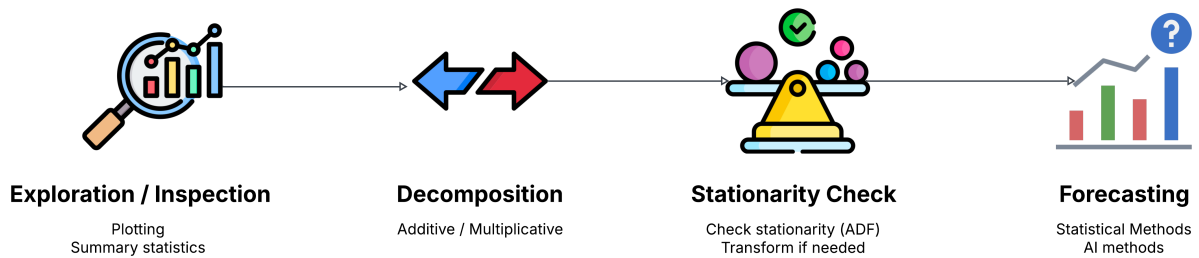


Figure 1.4: Time Series Analysis Pipeline.

This pipeline is interpretable, computationally inexpensive, and well understood theoretically. However, it treats time series as simple mathematical sequences governed by linear and stationary dynamics, rather than as realisations of complex physical systems. As a consequence, it fails to capture the non-linear, long-range, and multivariate dependencies that characterise real-world data. These limitations motivate the shift to deep learning, examined in Chapter 2. Before that, however, it is necessary to understand the deeper structural properties of time series data that any model classical or deep must respect.

1.8 The Temporal Data Manifold

Time series generated by real-world physical systems do not occupy the full observation space $\mathbb{R}^L$ arbitrarily. They reside on a structured subspace defined by the latent dynamics, transition laws, and statistical dependencies of the underlying generative process. This subspace is referred to as the **temporal data manifold** $\mathcal{M}$ Rojat et al. (2021).

Formally, let $\mathbf{x} \in \mathbb{R}^L$ denote an observed sequence of length L . Rather than being an arbitrary point in $\mathbb{R}^L$, a valid time series is a realisation of a stochastic process constrained to $\mathcal{M}$:

$$\mathbf{x} \in \mathcal{M} \subset \mathbb{R}^L, \quad \dim(\mathcal{M}) \ll L \quad (1.1)$$

This low-dimensional structure imposes two critical constraints on any meaningful manipulation of a series:

- **Temporal coherence.** Each observation x_t is intrinsically coupled with its predecessors and successors through the autocorrelation structure of the process. A modified sequence that ignores this coupling may be mathematically feasible yet physically nonsensical — it leaves $\mathcal{M}$.
- **Structural plausibility.** A valid series must respect the distributional boundaries of $\mathcal{M}$: its amplitude, variance, and local dynamics must remain consistent with the patterns observed in the training distribution.

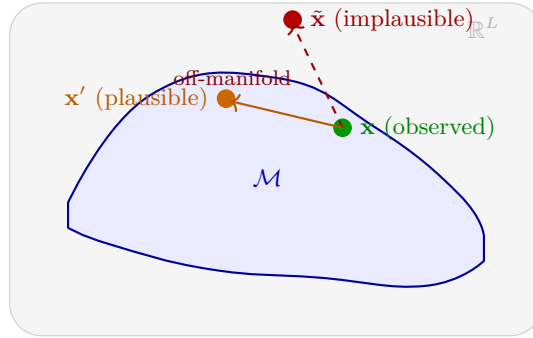


Figure 1.5: The temporal data manifold $\mathcal{M} \subset \mathbb{R}^L$. An observed series $\mathbf{x}$ lies on $\mathcal{M}$. A modified series $\mathbf{x}'$ is only meaningful if it remains on $\mathcal{M}$; arbitrary perturbations in $\mathbb{R}^L$ produce implausible sequences outside it.

These constraints have a direct practical consequence: any method that generates a new series from an observed one must ensure that the result remains on $\mathcal{M}$. Methods that operate directly in the raw input space $\mathbb{R}^L$ provide no such guarantee, frequently producing sequences that are temporally incoherent and that domain experts cannot interpret or act upon. This confinement to $\mathcal{M}$ has a concrete manifestation in the data: observations are not independent. The nature of these statistical links and the scales at which they operate is what the following section examines.

1.9 Temporal Dependencies

A defining characteristic of time series is that observations are not statistically independent: x_t depends on its past. This dependence is quantified by the autocorrelation function:

$$\rho(\tau) = \frac{\text{Cov}(x_t, x_{t-\tau})}{\text{Var}(x_t)}, \quad \tau = 1, 2, \dots \quad (1.2)$$

Real-world series exhibit dependencies at multiple temporal scales simultaneously Y. Liu et al. (2023):

- **Short-term dependencies** ($\tau \leq$ a few steps): local continuity and smooth transitions between consecutive observations. Any realistic series must preserve these.
- **Long-term dependencies** ($\tau \gg 1$): periodic patterns such as daily and weekly cycles in energy data, sustained trends, and regime changes spanning dozens or hundreds of timesteps.
- **Inter-variate dependencies** (multivariate series): cross-channel correlation structures — for instance, the relationship between load and temperature in the ETTh datasets — that must be modelled jointly across all variates.

These multi-scale dependencies reveal an asymmetry that is unique to time series and absent from tabular or image domains: within a look-back window, not all observations carry the same practical status with respect to what can be known or changed.

1.10 Mutable and Immutable Observations

A critical distinction in time series one that has no direct equivalent in tabular or image domains concerns the **actionability** of individual observations within a look-back window $\mathbf{x} = (x_1, \dots, x_L)$.

- **Historically fixed observations** $x_1, \dots, x_{L-k}$ record events that have already occurred. They are *immutable*: no present action can alter them. A series that differs from $\mathbf{x}$ at these positions describes a past that never happened — it is logically incoherent regardless of its statistical plausibility.
- **Recent observations** $x_{L-k+1}, \dots, x_L$ correspond to the immediate past, where an operator or system may still intervene. These are *mutable*: modifying them describes a scenario that is both physically realisable and temporally consistent with the causal structure of the process.

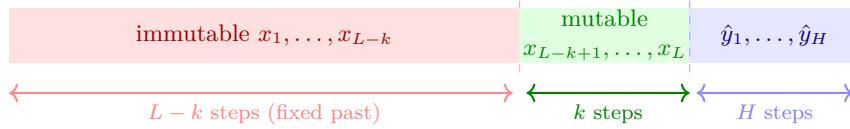


Figure 1.6: Mutable and immutable regions of a look-back window of length L . Only the k most recent observations are actionable; modifying earlier timesteps produces a logically incoherent alternative series.

This distinction has a direct implication for how a meaningful alternative series should be constructed: modifications must be **sparse** (few timesteps altered) and **localised** (concentrated in the recent, mutable region). A modified series that satisfies these two properties is both physically interpretable and practically useful — it prescribes precisely what to change and when, without altering a past that can no longer be modified.

Synthesis

This chapter established the foundational properties of time series data, progressing from classical analytical frameworks to the deeper structural constraints that govern real-world temporal data.

Classical methods — ARIMA, exponential smoothing, and their variants — provide a mathematically transparent and computationally inexpensive framework for modelling time series. However, they rest on two restrictive assumptions: linearity and stationarity. These assumptions render them inadequate for the complex, high-dimensional, and non-stationary datasets encountered in modern forecasting tasks, where nonlinear interactions and long-range dependencies are the norm rather than the exception.

Beyond these modelling limitations, real-world series are subject to structural constraints that classical methods fail to capture: the confinement of valid sequences to a low-dimensional data manifold, the multi-scale nature of temporal dependencies, and the asymmetry between historically fixed and recently actionable observations. Together, these constraints define what it means for a series — or a modification of one — to be physically meaningful, and they will guide the design choices developed throughout this thesis.

Conclusion

Time series analysis has evolved from rigid statistical frameworks toward a richer and more principled understanding of temporal dynamics. The classical pipeline — decomposition, stationarity transformation, and statistical modelling — remains a useful reference, but its assumptions are too restrictive for the complexity of modern data.

The structural properties introduced in the second part of this chapter — manifold confinement, multi-scale temporal dependencies, and the mutable/immutable distinction — go beyond what classical methods can capture or enforce. They define the deeper constraints that any model operating on temporal data must respect, and that will guide the design choices presented throughout this thesis.

The following chapter examines how deep learning architectures address the modelling limitations of classical methods, by learning hierarchical temporal representations directly from data.

Chapter 2

Deep Learning for Time Series

Introduction

Following the evolution of time series analysis from classical statistical foundations, recent years have witnessed a major paradigm shift toward deep learning-based approaches. Traditional models such as ARIMA, Exponential Smoothing, and VAR rely on assumptions of linearity and stationarity, which limit their ability to capture the complex, nonlinear, and long-range dependencies characteristic of real-world time series. Deep learning models overcome these limitations by learning hierarchical and expressive temporal representations directly from data, making them particularly suitable for high-dimensional, noisy, and non-stationary sequences.

This chapter summarizes key advances in deep learning for time series, tracing the evolution from classical deep learning families to advanced Transformer-based architectures, and highlighting their shared limitation of interpretability.

2.1 Challenges in Time Series Modeling

Time series data present two fundamental challenges Y. Wang et al. (2024). First, **temporal dependencies**: capturing multi-scale dynamics across short-term patterns, long-term trends, and irregular fluctuations is difficult, especially for long sequences where conventional models suffer from vanishing gradients or limited receptive fields. Second, **variate correlations**: in multivariate settings, interactions among variables may be nonlinear, time-varying, or misaligned, requiring expressive mechanisms beyond simple linear assumptions.

2.2 Architectural Primitives for Deep Time Series Modeling

The stationarization and decomposition methods introduced in Chapter 1 as preprocessing steps of the classical forecasting pipeline are no longer treated as external operations in modern deep learning. Instead, they are directly embedded into the network architecture as learnable building blocks. Stationarization mechanisms such as RevIN and DAIN adaptively normalize inputs to reduce distribution shifts across time. Decomposition modules separate trend, seasonality, and residual components within the model itself — a principle central to Autoformer and N-BEATS. Similarly, Fourier analysis, which was classically used to inspect periodicity, is now integrated as a computational primitive in models such as FEDformer and TimesNet for long-horizon forecasting Y. Wang et al. (2024). This transition from preprocessing to architecture reflects a broader shift: rather than preparing data for a model, the model learns to handle

temporal structure internally.

2.3 Classical Deep Learning Architectures for Time Series

Deep learning has introduced a wide range of architectures capable of modeling the nonlinear, multi-scale, and high-dimensional characteristics of time series data. According to the survey in Y. Wang et al. (2024), existing approaches can be grouped into five major architectural families, each offering different modeling capabilities and inductive biases.

2.3.1 Multilayer Perceptrons (MLPs)

MLP-based models treat forecasting as direct mappings from historical inputs to future outputs. Although simple, they are highly efficient and surprisingly competitive for many benchmarks. Models such as N-BEATS and N-HiTS use deep fully connected stacks and basis expansions to capture multi-scale temporal patterns, while lightweight architectures like DLinear demonstrate strong performance even with linear layers. Figure 2.1 illustrates how an MLP processes time-series inputs to generate predictions.

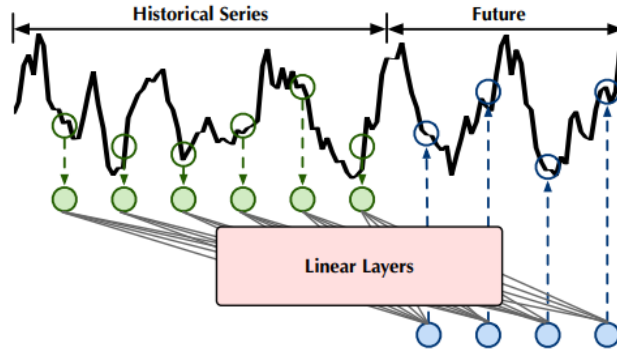


Figure 2.1: MLP Architecture for Time Series

However, MLPs lack inherent mechanisms to model long-range dependencies, requiring fixed-length input windows and treating time steps as independent features rather than sequential observations.

2.3.2 Recurrent Neural Networks (RNNs)

RNNs, including LSTM and GRU variants, naturally model sequential dependencies by maintaining hidden states over time. They were early dominant architectures in time series forecasting, imputation, and generative modeling. Extensions such as DeepAR, recurrent-skip connections (LSTNet), and state-space models (e.g., S4, Mamba) improve long-range modeling and temporal stability. Figure 2.2 provides an illustration of how an RNN consumes a time series sequentially to update hidden representations and produce predictions.

However, RNNs suffer from vanishing gradient problems in very long sequences, inherently sequential computation that prevents parallelization during training, and difficulty capturing long-range dependencies.

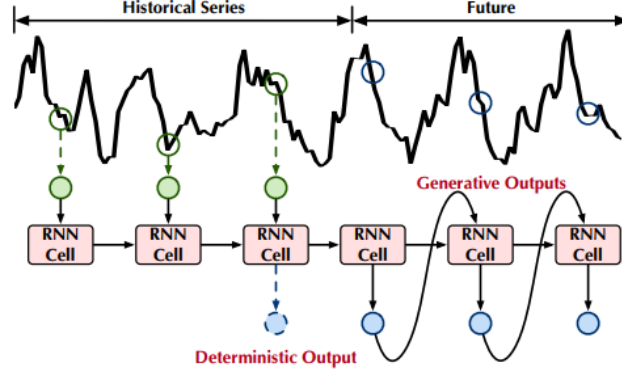


Figure 2.2: RNN Architecture for Time Series

2.3.3 Convolutional Neural Networks (CNNs)

CNN-based models capture local temporal patterns efficiently through 1D convolutions and dilated filters. They scale well to long sequences and large datasets. Architectures such as TCN, WaveNet, SCINet, and TimesNet enhance temporal modeling using hierarchical, dilated, or multi-period convolutional designs. Figure 2.3 illustrates the principle of applying temporal convolutions to a time series.

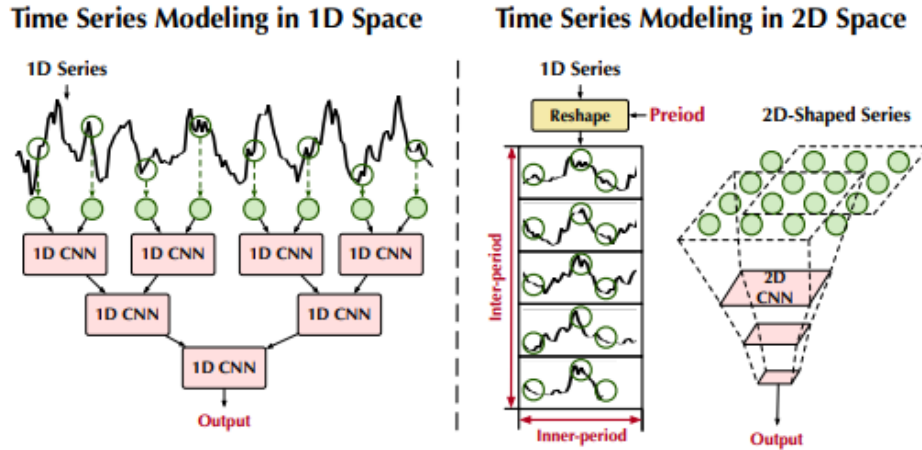


Figure 2.3: CNN Architecture for Time Series

Among CNN-based architectures, **Temporal Convolutional Networks (TCN)** Wan et al. (2019) have emerged as a particularly robust variant, overcoming the limitations of RNNs through three core mechanisms: **causal convolutions**, which ensure that the output at time t depends strictly on inputs from 0 to t , respecting the arrow of time; **dilated convolutions**, which grow the receptive field exponentially with network depth without increasing parameters; and **residual connections**, which stabilize deep network training and prevent information degradation. Unlike RNNs, TCNs process long sequences in parallel during training, combining a stable gradient flow with a large effective receptive field. The architectural details are illustrated in Figure 2.4.

However, CNNs in general — including TCNs — have limited receptive fields that require deep stacking or large dilation rates to capture very long-range dependencies, and their fixed kernel sizes may not adapt well to time series with varying temporal scales or irregular sampling patterns.

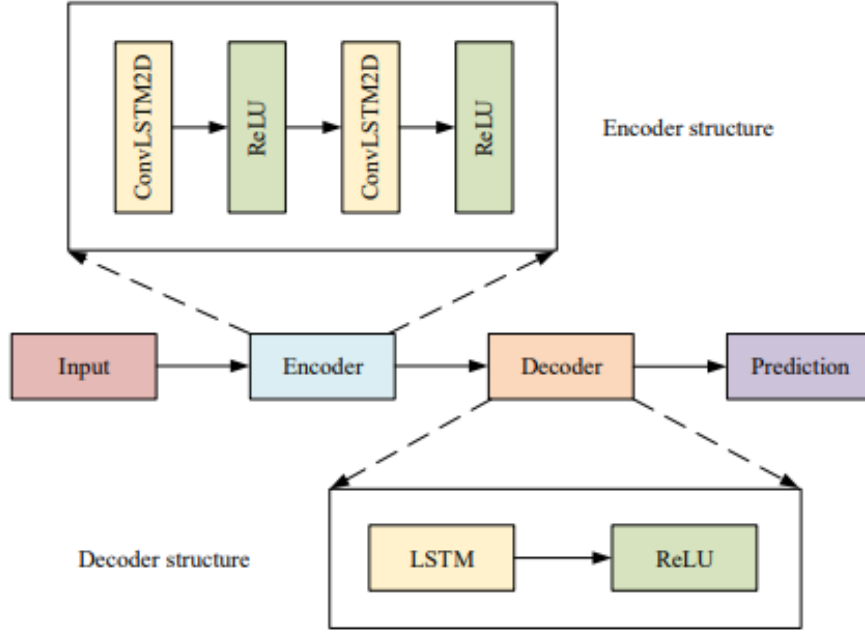


Figure 2.4: Architectural details of TCN: (a) Dilated causal convolutions with factors $d = 1, 2, 4$ (adapted from Figure 1 in Wan et al. (2019)). (b) Residual block structure including Weight Normalization and ReLU activation (based on Figure 2 in Wan et al. (2019)).

2.3.4 Graph Neural Networks (GNNs)

GNNs are designed for multivariate time series where variables exhibit structured or latent relationships. They model interactions among variables through graph edges while learning temporal evolution at each node. Examples include DCRNN, STGCN, Graph WaveNet, AGCRN, and StemGNN, which combine graph convolutions with recurrent or spectral components. Figure 2.5 shows how a time series with multiple correlated variables can be structured as a graph for spatiotemporal modeling.

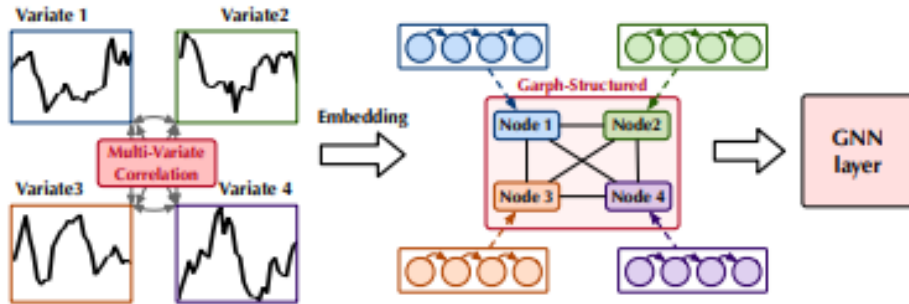


Figure 2.5: GNN Architecture for Time Series

However, GNNs require predefined or learned graph structures that may not generalize across different datasets or domains, and they primarily focus on spatial relationships among variables rather than capturing complex temporal patterns within individual series.

Architectural Trade-offs

While MLPs, RNNs, CNNs, and GNNs offer specific advantages for time series modeling, they exhibit significant limitations when applied to complex, multivariate, and long-range temporal data. These

Architecture	Advantages	Limitations	Examples
MLP	Simple, fast, effective for short and linear series	Cannot capture temporal order, relies on fixed input window	N-BEATS, N-HiTS, DLinear
RNN / LSTM / GRU	Sequential modeling, captures local and long-term dependencies	Vanishing gradients, sequential $\rightarrow$ slow training, hard to parallelize	DeepAR, LSTNet
CNN / TCN	Captures local patterns, parallelizable, efficient for long series	Limited receptive field, requires deep stacking for long-term dependencies	WaveNet, SCINet, TimesNet, TCN
GNN	Models inter-variable relationships, suitable for multivariate series	Requires a graph structure, less effective for intra-series temporal patterns	DCRNN, STGCN, AGCRN

Table 2.1: Comparison of Deep Learning Architectures for Time Series

observations highlight that simpler architectures alone are insufficient for capturing the intricate patterns present in real-world time series. **Transformers, with their attention mechanisms and parallel processing capabilities, provide the flexibility and expressive power necessary for modern time series tasks.**

2.4 Advanced Architectures: Transformers for Time Series

2.4.1 Transformer Architecture

The Transformer architecture, introduced by Vaswani et al. (2017), is a sequence modeling framework that relies entirely on attention mechanisms, replacing recurrent and convolutional networks. Its ability to process sequences in parallel and capture long-range dependencies makes it particularly suitable for time series forecasting.

The main components of a Transformer layer are: **Self-Attention**, which computes relevance scores between all positions in the sequence; **Multi-Head Attention**, which runs multiple attention heads in parallel to capture different types of relationships; **Positional Encoding**, which injects order information since Transformers have no inherent sequential bias; **Feed-Forward Networks (FFN)**, applied position-wise for nonlinear transformation; and **Residual Connections** with **Layer Normalization** to stabilize training and facilitate gradient flow.

The core operation is the scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.1)$$

where Q , K , V are the query, key, and value matrices, and d_k is the key dimension used for numerical stability. Multi-head attention extends this by running h parallel attention heads over different projection subspaces:

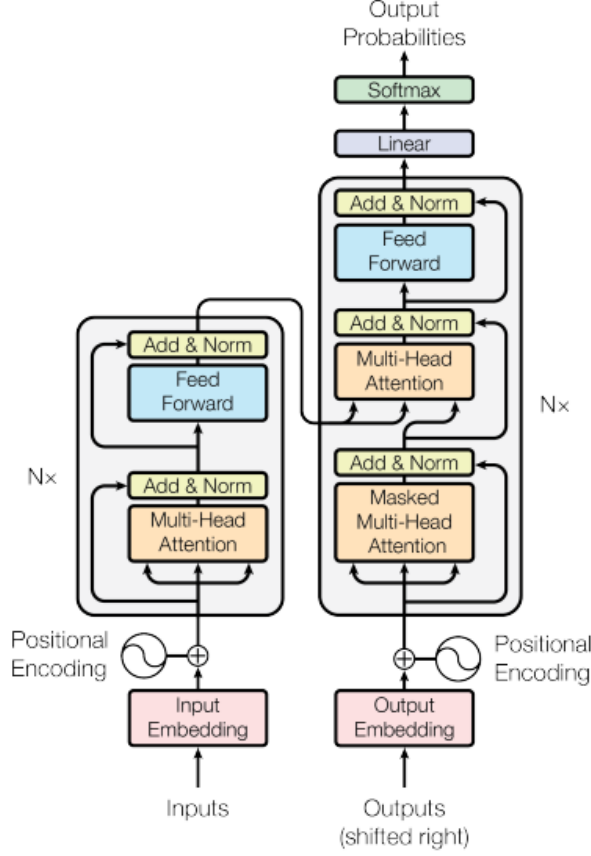


Figure 2.6: The Transformer architecture Vaswani et al. (2017).

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O, \quad \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (2.2)$$

This mechanism allows the model to simultaneously attend to different aspects of the sequence, making it powerful for capturing both local and long-range temporal dependencies.

2.4.2 Tokenization Strategies in Transformer-Based Time Series Models

A key design choice in Transformer-based time series models is how the input sequence is divided into tokens fed to the attention mechanism. As illustrated in Figure 2.7, three main strategies exist:

- **Point-wise tokens:** each time step is a separate token. Used by the vanilla Transformer and Informer — captures fine-grained temporal correlations but scales poorly with sequence length.
- **Patch-wise tokens:** contiguous segments of each variate form a token. Used by PatchTST and Crossformer — reduces the number of tokens while preserving local temporal structure.
- **Series-wise tokens:** the entire sequence of a single variate forms one token. Used by iTransformer — attention operates across variates rather than time steps, making it particularly effective for multivariate forecasting.

This tokenization choice directly determines what relationships the model can learn: point-wise captures temporal order, patch-wise balances local and global patterns, and series-wise prioritises inter-variate dependencies.

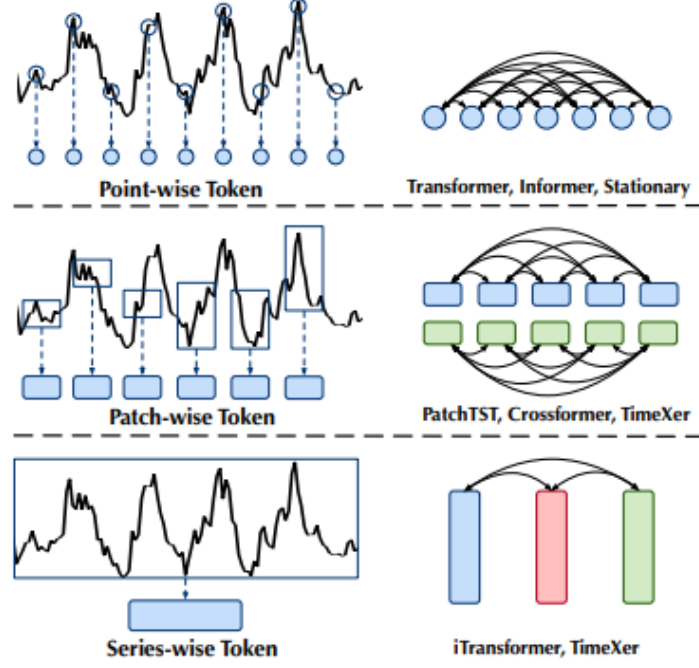


Figure 2.7: Tokenization strategies for Transformer-based time series models.

2.5 Advanced Transformer Architectures for Long-Term Time Series Forecasting

Building on the standard Transformer framework, several architectures have been proposed to address the scalability and accuracy limitations of the vanilla Transformer for long-term time series forecasting Wen et al. (2022).

2.5.1 Informer

H. Zhou et al. (2021) tackles the quadratic complexity of standard self-attention by introducing a ProbSparse mechanism that retains only the most informative queries, reducing complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log N)$. A distilling operation further compresses encoder representations across layers, and a generative decoder predicts the full horizon in a single forward pass, avoiding error accumulation typical of auto-regressive decoding.

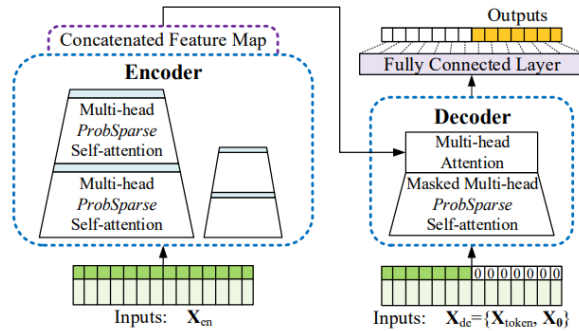


Figure 2.8: Informer model overview H. Zhou et al. (2021)

2.5.2 Autoformer

Wu et al. (2021) replaces standard attention with an auto-correlation mechanism that identifies repeating periodic patterns by measuring similarity between a series and its lagged versions. A series decomposition block explicitly separates the input into trend and seasonal components at each encoder and decoder layer, allowing the model to handle long-range dependencies while preserving temporal structure.

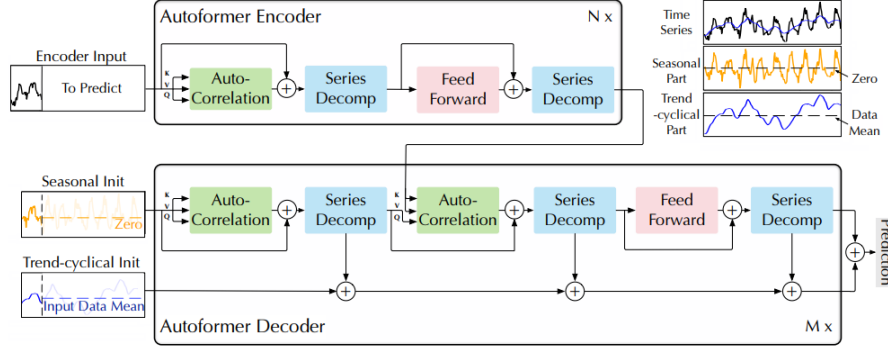


Figure 2.9: Autoformer model overview Wu et al. (2021)

2.5.3 FEDformer

T. Zhou et al. (2022) operates in the frequency domain rather than the time domain. Frequency Enhanced Blocks replace self-attention with Fourier or Wavelet transforms over a randomly selected subset of frequency modes, achieving linear complexity $\mathcal{O}(L)$. A Mixture-of-Experts decomposition block adaptively extracts multi-scale trend components, complementing the frequency-domain seasonal modeling.

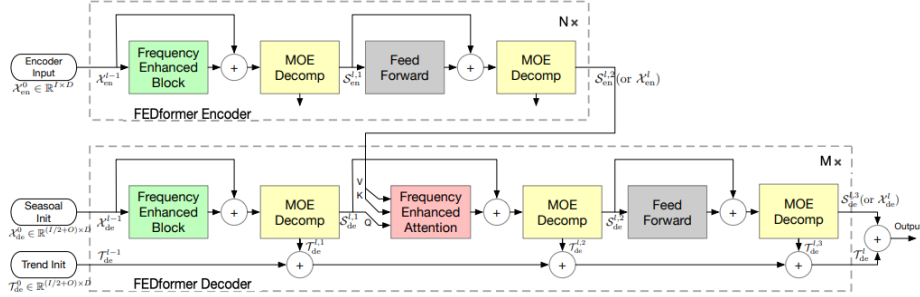


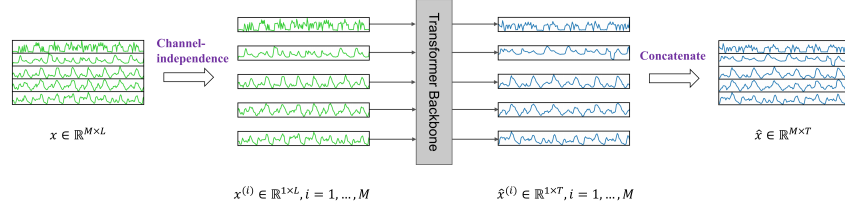
Figure 2.10: FEDformer model overview T. Zhou et al. (2022)

2.5.4 PatchTST

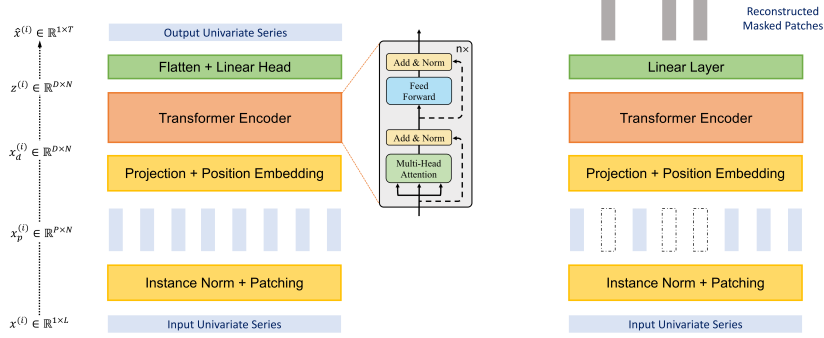
Nie et al. (2023) introduces a patch-based tokenization strategy that fundamentally rethinks how time series data is fed into the Transformer. Instead of treating each individual time step as a token, PatchTST divides each univariate time series into non-overlapping or overlapping *patches* (contiguous segments) that serve as tokens. This design reduces the number of tokens by a factor equal to the patch length, dramatically lowering memory and computational requirements.

Two key design principles distinguish PatchTST. First, **channel independence**: each variate is processed independently by the same Transformer, allowing the model to capture intra-series temporal patterns without interference from other variables. Second, **patch-level semantic richness**: each token

represents a local temporal segment rather than a single point, providing the attention mechanism with richer contextual information per token.



(a) PatchTST Model Overview



(b) Transformer Backbone (Supervised)

(c) Transformer Backbone (Self-supervised)

Figure 2.11: PatchTST architecture: patch-based tokenization with channel-independent Transformer encoders Nie et al. (2023).

2.5.5 Crossformer

Yunhao Zhang and Yan (2023) introduces a dual-attention mechanism that explicitly captures dependencies across both the *time dimension* and the *variate dimension* simultaneously. The **Cross-Time Stage** segments the input sequence into patches and applies self-attention within each variate to capture local and global temporal patterns. The **Cross-Dimension Stage** then applies attention across all variates at each patch position, modeling inter-variable correlations that evolve over time.

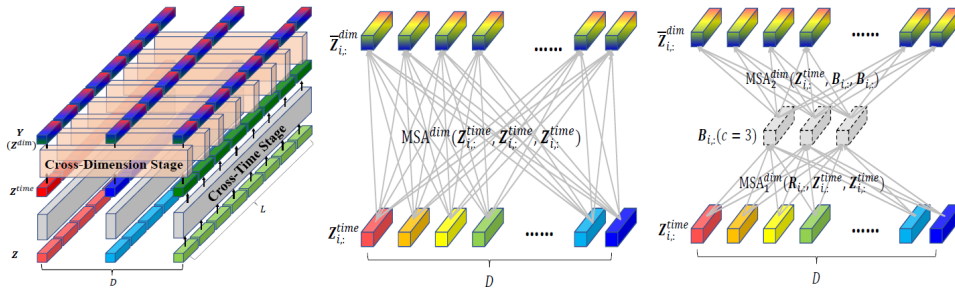


Figure 2.12: Crossformer architecture: DSW embedding and Two-Stage Attention (TSA) across time and variate dimensions Yunhao Zhang and Yan (2023).

2.5.6 Inverted Transformer (iTransformer)

Y. Liu et al. (2023) proposed the *iTransformer*, an alternative formulation of Transformer-based models for time series forecasting. Unlike conventional Transformers that treat each timestamp as a token, iTransformer adopts an inverted representation where each variate is considered as a token. This design

allows the self-attention mechanism to capture inter-variable correlations, while feed-forward networks model temporal dynamics within each variate. This simple yet effective reformulation achieves state-of-the-art results on multiple benchmarks.

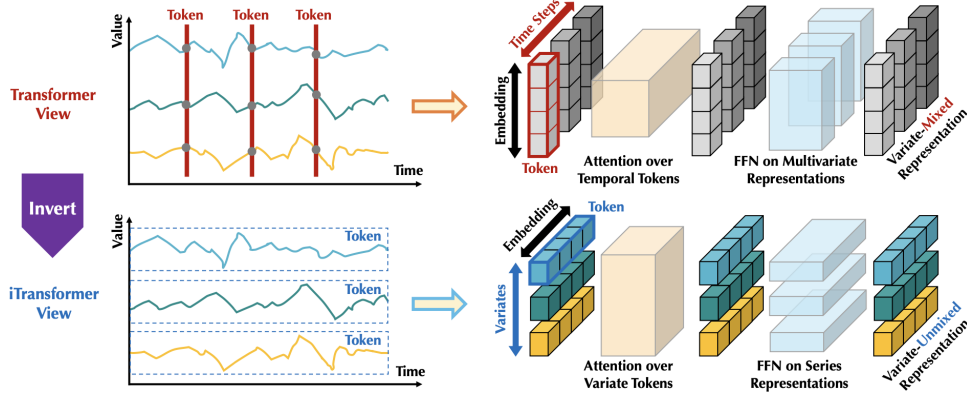


Figure 2.13: iTransformer architecture overview showing the inverted embedding process (adapted from Y. Liu et al. (2023)).

The inverted architecture manages to handle long-term dependencies much better than point-wise Transformers, which often suffer from performance degradation as the lookback window increases, positioning the iTransformer among the strongest forecasting architectures currently available for multivariate time series.

Models	iTransformer (Ours)	RLinear (2023)	PatchTST (2023)	Crossformer (2023)	TiDE (2023)	TimesNet (2023)	DLinear (2023)	SCINet (2022a)	FEDformer (2022)	Stationary (2022b)	Autoformer (2021)
Metric	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE	MSE MAE
Merchant	0.072 0.147	0.152 0.247	0.084 0.171	0.117 0.181	0.187 0.289	0.093 0.184	0.110 0.206	0.316 0.385	0.296 0.401	0.172 0.277	0.494 0.531
Wealth	0.345 0.289	0.585 0.461	0.394 0.326	0.429 0.288	0.595 0.481	0.360 0.318	0.501 0.412	0.660 0.514	0.625 0.543	0.499 0.415	0.772 0.612
Finance	0.184 0.216	0.395 0.336	0.231 0.248	5.333 0.618	0.987 0.442	0.516 0.308	0.765 0.372	2.817 0.734	1.621 0.569	1.368 0.643	1.872 0.681
Terminal	0.065 0.150	0.180 0.286	0.077 0.179	0.071 0.162	0.216 0.311	0.080 0.179	0.106 0.210	0.280 0.360	0.295 0.403	0.180 0.296	0.518 0.547
Payment	0.072 0.144	0.143 0.245	0.084 0.174	0.207 0.179	0.208 0.278	0.105 0.182	0.116 0.200	0.288 0.322	0.300 0.373	0.166 0.271	0.417 0.460
Customer	0.094 0.150	0.214 0.261	0.118 0.180	0.309 0.194	0.308 0.307	0.142 0.191	0.184 0.219	0.461 0.385	0.350 0.391	0.242 0.301	0.669 0.593

Figure 2.14: Performance comparison of iTransformer against baseline models on various benchmarks.

Conclusion

The architectures reviewed in this chapter reflect a clear progression in the treatment of time series data: from simple fixed-window mappings in MLPs, through sequential state propagation in RNNs and local pattern extraction in CNNs, to parallel attention-based modeling in Transformers. Each family addresses specific limitations of its predecessors, yet introduces new trade-offs, as summarized in Table 2.2.

Among all architectures surveyed, Transformer-based models stand out for their unique ability to simultaneously capture long-range dependencies and model complex multivariate interactions in parallel. The variants discussed — Informer, Autoformer, FEDformer, PatchTST, Crossformer, and iTransformer — each propose targeted innovations addressing a different bottleneck: computational efficiency, seasonal decomposition, frequency-domain modeling, patch-based tokenization, joint temporal and cross-variate attention, and inter-variate correlation, respectively.

However, this expressive power comes at a fundamental cost: the internal mechanisms of these models remain largely opaque to end users. Attention maps, while sometimes informative, do not directly

translate into actionable explanations — they indicate which variates or time steps the model attended to, but not *why*, nor what changes to the input would lead to a different forecast. This opacity is not merely a technical inconvenience; in high-stakes domains such as healthcare, finance, or energy, the inability to justify a prediction can undermine trust and limit deployment. This fundamental limitation directly motivates the study of dedicated Explainable AI methods, which form the focus of the next chapters.

Criteria	Transformer	Informer	Autoformer	FEDformer	PatchTST	Crossformer	iTransformer
Computational Complexity	$\mathcal{O}(N^2)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(L)$ (linear)	$\mathcal{O}(N/P)^2$ (P = patch size)	$\mathcal{O}(N \cdot M)$	$\mathcal{O}(M^2)$ (M = variates)
Long-range Modeling	Full self-attention (all-to-all)	ProbSparse attention (dominant queries)	Auto-correlation (periodic patterns)	Frequency-domain representation	Patch-level temporal attention	Dual attention: time + variate	Variate-wise attention (inter-series)
Seasonality Handling	Positional encoding only	Explicit timestamp encoding	Series decomposition (trend + seasonal)	Frequency analysis (Fourier/Wavelet)	Implicit via patch context	Implicit via patch context	Not explicitly modeled
Data Adaptability	Requires careful positional encoding	Needs timestamp tuning per dataset	Sensitive to decomposition parameters	Random frequency mode selection	Sensitive to patch size choice	Sensitive to patch and variate alignment	Best suited for multivariate series
Strengths	Theoretical expressivity	Efficient long sequences	Explicit seasonal modeling	Linear complexity, interpretable	Efficient, strong univariate forecasting	Joint temporal + cross-variate modeling	Strong multivariate forecasting
Limitations	Quadratic cost, no inductive bias for TS	Query selection sensitivity	Decomposition quality dependent	Random selection sensitivity	Channel-independent, ignores inter-variate	Higher complexity, harder to tune	Less effective for univariate series
When to Use	General baseline, short sequences	Very long univariate sequences	Series with strong seasonality	Global trend + periodic patterns	Univariate or channel-independent forecasting	Multivariate with temporal and cross-variate dependencies	High-dimensional multivariate forecasting

Table 2.2: Comparative analysis of Transformer architectures for time series forecasting.

Chapter 3

Explainable Artificial Intelligence for Time Series Models

3.1 Introduction

As established in the previous chapter, deep learning models such as TCNs and iTransformers have set new standards for forecasting accuracy. However, their multi-layered non-linear architectures function as "black boxes," providing predictions without transparent reasoning. In industrial or financial contexts, a forecast without an explanation is often unusable, as stakeholders require trust and accountability.

This chapter explores the field of Explainable Artificial Intelligence (XAI), which aims to demystify these models. We will classify existing XAI techniques and analyze their limitations when applied to the temporal and multivariate nature of time series, ultimately introducing counterfactual explanations as a superior alternative for actionable insights.

3.2 Conceptual Definitions

3.2.1 Explainable Artificial Intelligence (XAI)

Explainable Artificial Intelligence (XAI) is a subfield of AI that aims not only to achieve high predictive performance but also to ensure that the reasoning behind model decisions is understandable by human users. The need for XAI arises from the increasing complexity of modern AI systems, particularly deep learning models, which often operate as "black boxes". By providing explanations for predictions, XAI enhances transparency, fosters user trust, facilitates model debugging, and supports accountability in critical applications such as healthcare, finance, and autonomous systems Arsenault et al. (2024).

3.2.2 Explicability

Explicability refers to the property of an AI system that allows its structure, parameters, or components to be examined and inspected. It focuses on making the internal mechanisms of the model accessible and understandable to experts, without necessarily providing context or reasoning for individual decisions. Explicability is considered a prerequisite for explainability, as a model must first be inspectable before its decisions can be meaningfully interpreted Bruckert, Finzel, and Schmid (2020).

3.2.3 Explainability

Explainability goes beyond mere inspection of model components. It involves providing contextual information about how a model arrives at its outputs, why certain predictions are made, and which features contribute most to a given decision. Explainability is critical for non-expert users, regulators, and stakeholders who need to understand and trust AI decisions, especially in high-stakes domains Bruckert, Finzel, and Schmid (2020).

3.2.4 Interpretable Models vs Explainable Models

Interpretable models and explainable models are related but distinct concepts:

- **Interpretable models** are inherently understandable without requiring additional tools or post-hoc methods. Their internal logic is transparent, and their behavior can be directly analyzed by humans.
- **Explainable models** are more complex and may act as black boxes. Their behavior can only be understood with the help of dedicated explanation techniques. Explainable models allow practitioners to extract meaningful insights and justify decisions even from highly complex architectures.

3.2.5 Transparent models

Transparent models are machine learning models whose internal mechanisms are inherently interpretable without requiring post-hoc explanation methods. Transparency can be characterized along three levels Arrieta et al. (2020) :

- **Simulatability**: refers to the ability of a human to understand and mentally simulate the entire model as a whole. Simple models such as sparse linear models or single-layer perceptrons are typically simulatable, whereas complex or large-scale models are not.

- **Decomposability**: denotes the interpretability of individual components of a model, including inputs, parameters, and computations. A decomposable model allows each part to be understood independently, but this does not necessarily imply that the model is simulatable as a whole.
- **Algorithmic Transparency** describes the extent to which the learning and decision-making process of a model can be understood. Linear models are often algorithmically transparent, while deep neural networks lack this property due to complex optimization dynamics and opaque internal representations.

3.3 Objectives and Key Properties of Explainable AI

The effectiveness of an XAI system is measured not only by its ability to provide explanations, but also by the quality and trustworthiness of those explanations. According to Rojat et al. (2021), several key properties are critical for designing and evaluating XAI systems, as illustrated in Figure 3.1.

- **Explainability** refers to the capacity of a system to provide meaningful insights into its decision-making process, making predictions understandable for both experts and non-experts. Closely related,
- **Interpretability** is a passive property reflecting the degree to which a model’s behavior can be directly comprehended by a human observer without additional tools.
- **Trustworthiness** represents the confidence a user has that the model behaves as intended, enhanced by clear and reliable explanations.
- **Stability** and **Robustness** ensure that explanations remain consistent under small input perturbations or intentional disturbances.
- **Reproducibility** guarantees consistent outputs across multiple runs, while **Confidence** quantifies the certainty of predictions, helping users assess decision risk.



Figure 3.1: Knowledge graph relating the purposes of explainability methods Rojat et al. (2021).

3.4 Taxonomy of XAI Methods

XAI methods can be systematically categorized along multiple complementary dimensions, including the scale of explanation (local versus global), the stage at which explanations are generated (ante-hoc versus post-hoc), and the degree of model dependency (model-agnostic versus model-specific). This multi-criteria taxonomy, illustrated in Figure 3.2, provides a structured perspective for comparing explanation strategies and understanding their respective trade-offs.

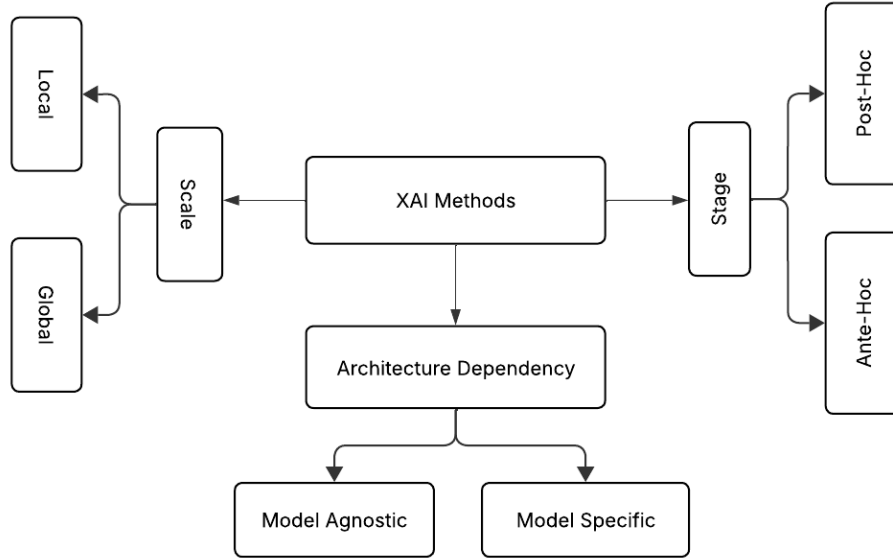


Figure 3.2: Multi-Criteria Taxonomy of XAI Methods.

3.4.1 Scale of Explanations

XAI methods provides either **local** or **global** explanations, depending on the scope of their outputs Rojat et al. (2021) :

Local explanations

Local explanations focus on individual predictions and are valid for a specific sample. They aim to answer questions such as “Why was this particular example classified as a X ?” or “Why did the model make this decision for this input?”. Local methods evaluate the contribution of each feature to a single prediction, making them particularly useful for debugging, trust assessment, or justifying specific decisions. Examples of techniques producing local explanations include LIME, SHAP applied per instance.

Global explanations

Global explanations characterize the overall behavior of a model across the entire dataset or a subset of inputs. They address questions such as “How does the model generally arrive at its conclusions ?” or “Which features are most influential across all predictions ?”. Some local methods can be extended to generate global insights, for example by aggregating SHAP values across the dataset or using clustering techniques to summarize patterns. Global explanations allow a broader understanding of complex models and facilitate model validation, regulatory compliance, and high-level interpretability.

3.4.2 Stage of Explanations

Another important dimension in the taxonomy of explainable AI methods concerns the stage at which interpretability is achieved, distinguishing between techniques that are integrated into the model design and those that are applied after a model has been trained Schwalbe and Finzel (2023) and Institute (2025) :

Ante-hoc Methods

Ante-hoc methods, also referred to as intrinsic explainability, incorporate transparency directly into the model’s architecture or learning process. These methods aim to make the decision logic understandable by design, rather than through external analysis after training. By constraining the model structure or embedding interpretable components into its architecture. An example of an architectural mechanism that can contribute to intrinsic explainability is the **attention mechanism** in neural networks. Attention layers assign weights to input features based on their relevance in producing a prediction.

Post-hoc Methods

In contrast, post-hoc explainability refers to methods that are applied after a complex model has been trained. These techniques aim to produce explanations for the behavior of black-box models without altering their internal structure or parameters. Common examples include feature attribution methods (such as LIME and SHAP). Post-hoc explainers are widely used in practice due to their flexibility and broad applicability, but they do not change the model itself; instead, they derive interpretable insights by analyzing the trained model’s outputs or internal activations .

3.4.3 Model Architecture Dependency

Model Agnostic

Model-agnostic post-hoc techniques are designed to be applied to any machine learning model with the goal of extracting interpretable information from its predictions. These methods do not require access to the internal structure of the model and often rely on three main strategies: simplification, feature relevance estimation, and visualization Arrieta et al. (2020) :

- **Simplification:** Create interpretable proxies, such as rule-based or linear models (e.g., LIME, G-REX), to approximate complex models and make their predictions understandable.
- **Feature Relevance:** Quantify the contribution of input features to predictions using techniques like SHAP, local gradients, or influence functions, providing local and global insights.
- **Visualization:** Represent model behavior or feature importance graphically through ICE plots, saliency maps, or cluster-based visualizations, often combined with feature relevance for clearer interpretation.

These approaches collectively enable interpretability for any model while remaining independent of its architecture, supporting better understanding and trust in machine learning systems.

Model Specific

Model-specific explainability methods are tailored to a particular type of model and exploit its internal structure. They are divided into two main categories:

- **Shallow Machine Learning** Shallow ML models include a variety of supervised learning approaches. Some models, such as KNN and single decision trees, are inherently interpretable. More sophisticated models, such as tree ensembles and Support Vector Machines (SVMs), require post-hoc explainability techniques to make their predictions understandable. Techniques commonly used :

- **Explanation by Simplification:** Rule extraction from support vectors (e.g., sequential covering algorithms, fuzzy rules, Hyper-rectangle Rule Extraction) and clustering prototype vectors to approximate the decision boundary.
- **Local Explanations:** Highlight individual predictions and their relation to nearby support vectors or margins.
- **Visualization Techniques:** Visualizing kernel matrices, heatmaps, or weight vectors to understand feature contributions and decision margins.
- **Bayesian Approaches:** Interpreting SVMs as Maximum A Posteriori (MAP) solutions with Gaussian Process priors, providing probabilistic predictions and hyperparameter transparency.

- **Deep Learning Models** Deep learning models, including MLPs, CNNs, and RNNs, are highly expressive but often considered black boxes. Model-specific explainability techniques leverage the architecture and internal representations:

- **MLPs:** Use model simplification (e.g., DeepRED, distillation methods) and feature relevance (e.g., Deep Taylor decomposition, DeepLIFT, Integrated Gradients) to interpret hidden layers.
- **CNNs:** Employ visual explanation techniques such as saliency maps and activation visualizations to interpret hierarchical features.
- **RNNs:** Use feature relevance (LRP, finite-horizon n-grams) and architectural modifications (attention mechanisms, interpretable latent vectors) to understand sequential dependencies.
- **Hybrid Methods:** Combine black-box DNNs with transparent models (Decision Trees, kNN, rule-based systems) or integrate domain knowledge to improve interpretability while preserving performance.

3.5 Dimensions of Explainability for Time Series Models

Explainability in time series aims to identify which parts of the input data are most influential in a model's prediction. Depending on the dimension being analyzed, explainability methods can be categorized into:

3.5.1 Feature Relevance

Feature relevance aims to determine **which input variables or features** are most influential in the model's decision. In multivariate time series, different features may have different impacts at different time steps. Feature relevance explanations typically provide:

- Importance scores for each input feature,
- Feature contributions at specific time steps,
- Local or global feature importance.

Common methods include Integrated Gradients, DeepLIFT, SHAP-based approaches, and feature-level attention mechanisms. These explanations are crucial in domains such as healthcare, finance, and energy systems, where understanding variable influence is essential.

3.5.2 Time Relevance

Time relevance aims to explain **which time steps in the input time series are the most influential** for a model's prediction. Instead of focusing on variables or frequencies, this type of explainability highlights the temporal positions that contribute most to the output. Time relevance methods typically provide:

- Importance scores for each timestamp,
- Identification of critical past events or periods,
- Local or global temporal contributions.

Representative methods include TimeSHAP, which estimates the importance of individual time steps, as well as attention-based and gradient-based approaches that highlight influential timestamps. These methods help identify whether recent or past observations are most relevant for prediction tasks such as forecasting and anomaly detection.

3.5.3 Frequency Relevance

Frequency relevance focuses on understanding **which frequency components** of a time series influence the model's predictions. Many time series contain hidden periodic patterns that are better represented in the frequency domain rather than the time domain. Frequency relevance explanations analyze:

- Dominant frequency bands (e.g., low-frequency trends or high-frequency fluctuations),
- Seasonal and periodic patterns,
- Noise vs. informative signal components.

This type of explainability often relies on spectral analysis techniques such as Fourier Transform or Wavelet Transform, and is commonly used in frequency-aware models like Autoformer and FEDformer. Frequency relevance is particularly useful for understanding seasonality and long-term cyclic behaviors.

3.6 Mechanism-Based Taxonomy of Transformer Explanations

Following the taxonomy proposed in Fantozzi and Naldi (2024), explanation techniques for Transformer models can be systematically classified into four mechanism-oriented families, Figure 3.3 summarizes the five explanation mechanisms discussed and illustrates representative XAI techniques associated with each mechanism:

3.6.1 Perturbation-based Methods

Perturbation-based methods assess feature importance by measuring changes in model predictions when input components are systematically altered, masked, or removed, providing model-agnostic insights into predictive behavior.

3.6.2 Gradient-based Methods

Gradient-based approaches derive explanations by computing gradients or relevance scores with respect to model inputs or intermediate representations, thereby identifying features that most strongly influence the model’s output.

3.6.3 Activation-based Methods

Activation-based methods interpret Transformer models by analyzing the internal activation patterns produced within intermediate layers, aiming to identify which hidden representations or neurons contribute most significantly to the model’s predictions.

3.6.4 Attention-based Methods

Attention-based methods interpret model behavior by analyzing the attention weights computed within Transformer layers, under the assumption that these weights reflect the relative importance of input tokens in the decision process.

3.6.5 Hybrid Methods

Hybrid methods integrate multiple explanatory mechanisms—such as attention, gradients, and perturbations—to combine their complementary strengths and deliver more stable and reliable interpretations.

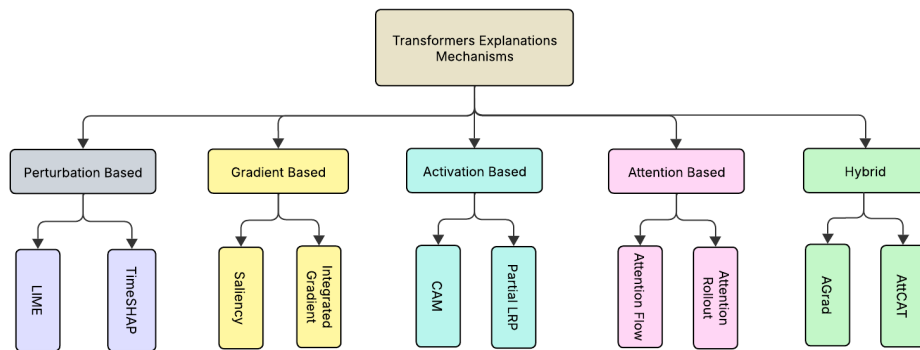


Figure 3.3: Transformers Explanations Mechanisms

3.7 XAI methods for Time Series

3.7.1 LIME

Local Interpretable Model-agnostic Explanations (LIME) Ribeiro, Singh, and Guestrin (2016) is a post-hoc, model-agnostic method that explains individual predictions of any black-box model by fitting a simple interpretable surrogate — such as a linear model — in the local neighborhood of the input instance. Perturbed samples are generated around the instance, weighted by proximity, and used to train the surrogate, whose coefficients reveal each feature’s contribution to the prediction. In time series analysis, LIME can be adapted to perturb temporal segments or individual time steps, identifying the intervals most influential for a given prediction. However, temporal correlations between observations may be disrupted during perturbation, leading to unrealistic samples and potentially misleading explanations.

3.7.2 SHAP

SHapley Additive exPlanations (SHAP) Lundberg and Lee (2017) is a unified framework grounded in cooperative game theory that assigns each feature a theoretically consistent contribution to the model’s prediction. SHAP values are additive — their sum equals the difference between the model output and the expected baseline — providing both local (per-instance) and global (dataset-level) explanations. In time series forecasting, SHAP identifies which time steps, features, or lags are most influential for a prediction, making it particularly helpful for models with complex dependencies like Transformers.

3.7.3 ForecastCF (CounterFactual)

ForecastCF, introduced by Z. Wang et al. (2023), is the first method to adapt counterfactual explanations specifically to time series forecasting. Unlike feature attribution methods, ForecastCF generates actionable modifications of the input sequence that lead a forecasting model to produce a desired future trajectory. The approach applies gradient-based perturbations to the original time series, iteratively adjusting it until the model’s predictions satisfy user-defined upper and lower bound constraints over the forecasting horizon.

Evaluated on six real-world datasets (CIF2016, NN5, Tourism, M4-Finance, SP500, MIMIC) across four architectures (GRU, Seq2seq, WaveNet, N-BEATS), ForecastCF consistently achieves validity rates above 75% while producing more compact and interpretable modifications than nearest-neighbor and naive shifting baselines. However, the method operates directly in the input space without any plausibility constraint, which can lead to unrealistic or temporally incoherent counterfactuals — a key limitation that motivates our proposed approach.

Limitations

3.7.4 Dynamic Mask

Dynamic Mask Crabbé and Van Der Schaar (2021) is a post-hoc, model-agnostic technique that provides local interpretability for time series models by identifying the contribution of individual features and time steps to a given prediction. The method optimizes a mask matrix M applied to the input sequence X , producing a modified input X_{mask} whose prediction remains as close as possible to the original output. The resulting mask highlights the most influential temporal segments and features without requiring any modification to the underlying model. While effective for multivariate and high-dimensional sequences, Dynamic Mask is computationally intensive for long series and relies on the quality of the perturbation strategy, which may introduce artifacts that distort the explanation.

3.7.5 ShapTime

ShapTime Yuyi Zhang et al. (2024) is a model-agnostic XAI approach specifically designed for time series forecasting, extending Shapley values from cooperative game theory to the temporal dimension. Rather than attributing importance to individual features, it divides the input sequence into contiguous groups of time points called *super-times* and computes the marginal contribution of each segment to the model’s prediction. This temporal granularity makes ShapTime particularly interpretable for forecasting tasks, where understanding which historical intervals drive future predictions is more meaningful than feature-level attribution. However, the method becomes computationally expensive as the number of segments grows, and its effectiveness remains tied to the quality of the underlying forecasting model.

3.7.6 ShapTST

ShapTST Cheng et al. (2025) is an ante-hoc explanation method that integrates interpretability directly into the Transformer architecture through a dedicated *explainer head* — a Shapley-based module that computes feature and temporal importance simultaneously with predictions, without any post-hoc overhead. This tight coupling allows the model to jointly optimize predictive accuracy and explanation fidelity, providing both local and global attributions in a single forward pass. Benchmark evaluations show that ShapTST outperforms or matches ShapTime in computational efficiency, accuracy, and explanation faithfulness. Unlike post-hoc methods, however, ShapTST is model-specific and requires retraining the Transformer with the explainer head, limiting its applicability to black-box or already-trained models.

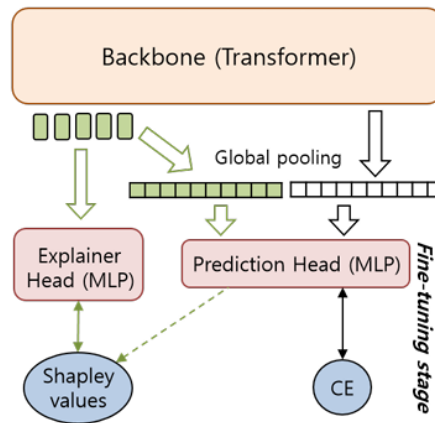


Figure 3.4: ShapTST’s Architecture

3.7.7 Layer-Wise Relevance Propagation (LRP)

Layer-Wise Relevance Propagation (LRP) is a post-hoc technique that decomposes a model’s output into relevance scores assigned to each input feature by propagating the prediction backward through the network layers. A conservation constraint ensures that relevance is neither created nor destroyed across layers — neurons contributing more strongly to a given output receive a proportionally larger share of its relevance. The resulting relevance map highlights which input features support or inhibit the prediction.

However, when applied directly to raw time series, LRP often produces diffuse and semantically unclear explanations, as individual time steps lack intrinsic meaning and spectral patterns remain hidden.

3.7.8 LRP with Virtual Inspection Layer (VIL)

To address this limitation, the Virtual Inspection Layer (VIL) Vielhaben et al. (2024) introduces an invertible linear transform — either a Discrete Fourier Transform (DFT) or a Short-Time Fourier Transform (STFT) — as a conceptual layer placed before the model input. LRP relevance is then propagated through both the model and this transform, expressing relevance in the frequency or time-frequency domain rather than the raw time domain.

DFT-VIL reveals which global frequency components support or inhibit the prediction, while STFT-VIL produces a time-frequency relevance map that additionally captures *when* those frequencies matter. Both variants preserve LRP’s conservation principle and expose spectral decision strategies invisible to standard LRP.

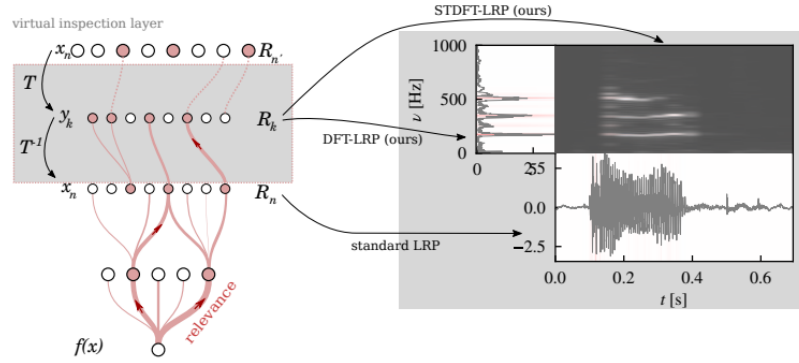


Figure 3.5: LRP with Virtual Inspection Layer Vielhaben et al. (2024).

3.7.9 Attention Rollout

Standard attention weights in Transformers are limited for interpretability, as they provide only local, layer-wise information and lose token identity in deeper layers. Consequently, they cannot reliably reflect the true contribution of input tokens to model predictions Abnar and Zuidema (2020).

To address this, Attention Rollout is proposed as a **post-hoc global** method that estimates token contributions by propagating attention across layers while incorporating residual connections. The augmented attention matrix combines attention weights with identity connections to preserve original token information.

The cumulative attention is computed recursively across layers, capturing all possible paths from input tokens to higher-level representations. This provides a global view of information flow, where each value reflects the proportion of information contributed by an input token to the final representation.

Attention Rollout is particularly useful for time series, as it captures long-range temporal dependencies and identifies influential time steps. However, it relies on a linear propagation assumption, may lose fine-grained details due to head averaging, and does not provide causal explanations.

3.7.10 Attention Flow

Attention Flow extends Attention Rollout by addressing its limitation in handling multiple overlapping paths. It models attention as a flow network, where tokens are nodes and attention weights (including residual connections) define edge capacities Abnar and Zuidema (2020).

The method computes the **maximum flow** from output tokens to input tokens, ensuring that all possible paths are considered without underestimating their contributions. This results in a more robust and distributed estimation of token importance compared to Rollout.

The output represents the amount of information that can be propagated between tokens, allowing identification of the most influential inputs in the final prediction.

While Attention Flow provides improved interpretability and better alignment with attribution methods, it is computationally more expensive and, like Rollout, does not guarantee causal explanations.

3.7.11 NeuroSymbolic Time Series Classification

3.8 Comparison of XAI Methods for Time Series

Explainable Artificial Intelligence (XAI) methods provide different ways to interpret model predictions in time series analysis. These methods vary in terms of required inputs, produced outputs, and the type and scale of explanations they provide.

Table 3.1 presents a comparative overview of commonly used XAI methods. For each method, it summarizes the main inputs, the form of the outputs, and the nature of the explanations (e.g., temporal, feature-level, or structural).

This comparison is purely descriptive and does not aim to evaluate or rank the methods. It is intended to provide a general overview of existing approaches.

Method	Input	Output	XAI Scale	XAI Stage	Architecture Dependency
Attention Weights Visualization	Model attention matrices	Attention heatmaps	Local	Post-hoc	Model-specific
Attention Rollout	Multi-layer attention + residuals	Global token importance	Global	Post-hoc	Model-specific
Attention Flow	Attention graph representation	Information flow paths	Local + Global	Post-hoc	Model-specific
Integrated Gradients	Input + baseline	Attribution map	Local	Ante-hoc	Model-agnostic
LRP	Model weights + activations	Relevance maps	Local + Global	Post-hoc	Model-specific
SHAP	Input + model outputs	Feature contributions	Local + Global	Post-hoc	Model-agnostic
ForecastCF	Time series forecasting input	Counterfactual explanations	Local	Post-hoc	Model-agnostic
Dynamic Mask	Time series input (masking perturbation)	Feature importance (temporal attribution)	Local	Post-hoc	Model-agnostic
ShapTime	Time series input + model predictions	Temporal feature importance	Local + Global	Post-hoc	Model-agnostic
ShapTST	Transformer-based time series input	Attention-aware SHAP explanations	Local + Global	Post-hoc	Model-specific

Table 3.1: Comparative overview of XAI methods for time series.

3.9 Evaluation Methods for XAI Explanations on Time Series

This section summarizes the verification methodology for evaluating explainable AI (XAI) methods on time series data, as proposed by Schlegel et al. in Schlegel et al. (2019) and shown in the figure 3.6. The core idea is to perturb the input time series based on relevance scores from an XAI method and measure the impact on model performance. If the explanation correctly identifies important features, perturbing them should degrade performance more than perturbing irrelevant or random features.

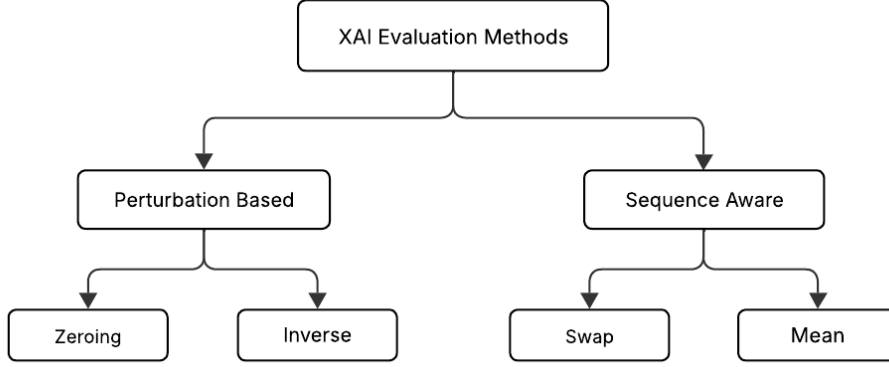


Figure 3.6: Time series Evaluation Strategies

Two main categories of evaluation methods are introduced: A **random baseline** is used as a control to ensure effects are due to relevance, not noise.

3.9.1 Perturbation-Based Methods

Given a time series t and relevance vector r , points with relevance above a threshold e are modified.

Zeroing:

Set relevant points to zero. Tests dependence on magnitude.

$$t_i^{\text{zero}} = \begin{cases} 0 & \text{if } r_i > e \\ t_i & \text{otherwise} \end{cases}$$

If the explanation is correct, setting relevant points to zero should cause a noticeable drop in model performance (e.g., accuracy). This drop should be larger than when zeroing random points (random baseline), indicating that the model indeed relies on the magnitude of those specific time points.

Inverse:

Replace with inverse value. Tests sensitivity to direction.

$$t_i^{\text{inverse}} = \begin{cases} \max(t) - t_i & \text{if } r_i > e \\ t_i & \text{otherwise} \end{cases}$$

If the explanation correctly identifies directional features, inverting relevant points should degrade performance more than random inversion. This suggests the model is sensitive to the polarity or shape direction (e.g., peaks vs. troughs) of the signal.

3.9.2 Sequence-Aware Methods

Modify contiguous subsequences starting from high-relevance points.

Swap:

Reverse the subsequence. Tests importance of temporal order.

$$t_{\text{sub}}^{\text{swap}} = (t_{i+n_s}, \dots, t_{i+1}, t_i)$$

If the explanation captures meaningful temporal patterns, reversing relevant subsequences should lead to a significant performance decrease, more so than reversing randomly selected subsequences. This indicates the model relies on the original temporal ordering (e.g., trends, slopes, or patterns).

Mean:

Flatten subsequence to its mean. Tests reliance on local variability.

$$t_{\text{sub}}^{\text{mean}} = (\mu, \dots, \mu), \quad \mu = \frac{1}{n_s} \sum_{k=i}^{i+n_s} t_k$$

If the explanation identifies regions where local variability matters, flattening those subsequences should harm performance more than flattening random subsequences.

Synthesis

While the XAI methods reviewed—ranging from gradient-based saliency to model-agnostic perturbations like SHAP—provide valuable insights into feature importance, they face significant hurdles in the temporal domain. The comparative analysis reveals a fundamental trade-off: methods are either computationally prohibitive for long-lookback models like the iTransformer, or they fail to respect the underlying physical and temporal correlations of multivariate data. Consequently, most existing approaches produce ‘static’ importance maps that lack the dynamic context required for complex forecasting.

Conclusion

Despite their effectiveness in improving model transparency, most existing XAI methods remain strictly *descriptive*. They identify which historical timestamps or variables influenced a forecast, but they do not provide *actionable* guidance. In critical decision-making scenarios, knowing that a variable was ‘important’ is insufficient; stakeholders need to know how to intervene to prevent an unfavorable outcome.

This limitation marks the boundary of traditional feature attribution. Counterfactual explanations emerge as a necessary paradigm shift, moving from passive description to active intervention by identifying the minimal, plausible changes needed to alter model predictions.

Accordingly, the following chapter is dedicated to the theory and generation of counterfactual explanations, exploring how their integration with reinforcement learning can overcome the current limitations of time series interpretability.

Chapter 4

Counterfactual Explanations for Time Series

Introduction

In recent years, explainable artificial intelligence (XAI) methods have been widely used to improve the interpretability of machine learning models. These approaches provide valuable insights into model behavior by identifying the features or temporal patterns that influence predictions. However, most of these methods remain descriptive, focusing on explaining why a prediction is made rather than providing guidance on how to change it.

In many real-world applications, particularly in time series analysis, decision-making requires actionable insights rather than descriptive explanations. In this context, counterfactual explanations offer a complementary perspective by identifying the minimal changes needed in the input to achieve a desired outcome.

This chapter focuses on counterfactual explanations for time series. It first introduces their definition and underlying principles, then reviews existing methods for classification and forecasting tasks. Finally, it discusses their limitations and challenges, highlighting the need for more robust approaches.

To better position counterfactual explanations within the broader landscape of XAI methods, Figure 4.1 from Speith (2022) presents a taxonomy highlighting different categories of explainability approaches.

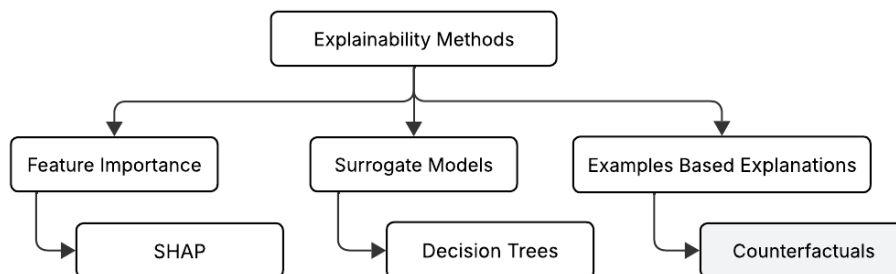


Figure 4.1: Taxonomy of Explainable AI Methods with Emphasis on Counterfactual Explanations

4.1 Definition and Formulation of Counterfactual Explanations

According to Wachter, Mittelstadt, and Russell (2017), a counterfactual explanation is defined as the minimal change to an input instance that results in a change in the model’s prediction to a desired target outcome. These explanations are model-agnostic and provide actionable insights by focusing on how to alter the input rather than explaining the internal mechanisms of the model.

4.2 Motivation and Advantages of Counterfactual Explanations

Counterfactual explanations have gained increasing attention in explainable artificial intelligence due to their intuitive and actionable nature. Instead of explaining a prediction by assigning importance scores to input features, counterfactual explanations focus on identifying how an input should be minimally modified to obtain a different prediction.

This type of explanation directly answers a contrastive question: *what should be different for the model to produce another outcome?* Such reasoning closely matches how humans naturally explain decisions, often by comparing the observed situation with alternative scenarios.

Given an input X and a predictive model $f(\cdot)$, a counterfactual explanation consists of a modified input X_{cf} such that the model output changes, while keeping the modification as small as possible. The goal is not only to change the prediction, but also to ensure that the resulting instance remains realistic and interpretable.

Compared to attribution-based methods such as feature importance scores, counterfactual explanations provide more actionable insights. They explicitly indicate which changes in the input would lead to a different prediction, rather than simply highlighting influential features.

However, counterfactual explanations describe the behavior of the model rather than true causal relationships. They should therefore be interpreted as local, model-dependent explanations.

Overall, counterfactual explanations offer a complementary perspective in XAI by focusing on minimal and meaningful transformations of the input that lead to different outcomes.

4.3 Counterfactual Explanations for Time Series

Counterfactual explanations aim to provide actionable insights by identifying how an input should be modified in order to achieve a desired outcome. Unlike traditional XAI methods that focus on explaining model predictions, counterfactual approaches explicitly define an objective and search for minimal changes to the input that satisfy this objective.

In the context of time series, defining such objectives is more complex due to the sequential nature of the data and the dependencies between time steps. The type of objective depends on the learning task, and different formulations arise for classification and forecasting problems.

In the following, we distinguish between counterfactual objectives for classification and forecasting, as they involve fundamentally different constraints and challenges :

4.3.1 Counterfactuals for Classification

Counterfactual explanations for classification tasks are relatively well studied compared to other domains. In this setting, a counterfactual explanation aims to determine how an input time series should be modified so that the predicted class changes from an original class c to a target class c' . Formally, given a time series T_q classified as c , the goal is to generate a counterfactual instance T' such that the classifier predicts c' instead.

Several approaches have been proposed in the literature. Among them, instance-based methods have shown promising results for time series classification. For example, Delaney, Greene, and Keane (2021) introduce a model-agnostic technique that generates counterfactuals by adapting existing instances from the dataset. Their approach relies on retrieving a *nearest unlike neighbor* (NUN), which belongs to a different class, and modifying the most discriminative subsequences of the original time series to produce a plausible counterfactual.

Unlike traditional feature-based methods, these approaches consider the sequential nature of time series data by focusing on contiguous subsequences rather than independent features. This is important because time series data exhibit strong temporal dependencies, and meaningful explanations must preserve these structures.

Moreover, several key properties of good counterfactual explanations have been identified in this context, including proximity (minimal change from the original instance), sparsity (modification of limited regions), plausibility (remaining within the data distribution), and diversity (providing multiple alternative explanations).

Figure 4.2 illustrates a typical example of a counterfactual explanation for time series classification. The original signal is modified to produce a counterfactual instance that leads to a different predicted class.

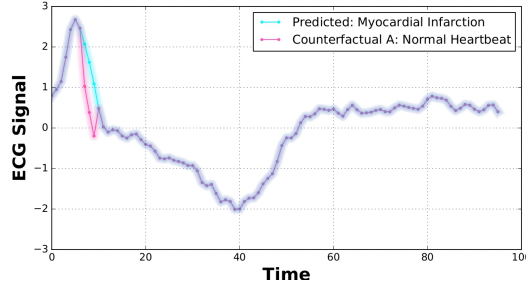


Figure 4.2: Example of a counterfactual explanation for time series classification. The original signal (blue) is modified to generate a counterfactual instance (Red) that changes the predicted class.

Overall, counterfactual explanations for classification provide intuitive and actionable insights by showing how a time series instance can transition from one class to another. However, extending these approaches to forecasting tasks introduces additional challenges due to the continuous nature of predictions and temporal consistency constraints.

4.3.2 Counterfactuals for Forecasting

In time series forecasting, counterfactual explanations consist in generating alternative input sequences that lead to a desired change in the predicted trajectory. This requires not only achieving the target forecasting objective but also ensuring that the generated sequences remain temporally coherent, plausible, and close to the original input. Consequently, counterfactual generation in time series is inherently more complex than in tabular settings due to sequential dependencies and structural constraints.

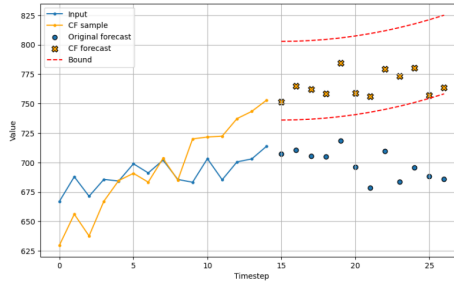


Figure 4.3: Counterfactual forecasting under bounded target constraints

Figure 4.3 illustrates the concept of counterfactual explanations in time series forecasting. The original input sequence is shown along with its corresponding forecasted values. A CF instance is generated by minimally modifying the input sequence, resulting in a new forecast that satisfies a predefined objective.

In this example, the goal is to increase the predicted values within a target range, represented by the dashed bounds. The counterfactual input leads to a shift in the predicted trajectory, while remaining close to the original sequence.

This illustrates the key idea of counterfactual explanations: identifying minimal and plausible changes to the input that produce a desired outcome in the model’s prediction Z. Wang et al. (2023).

4.4 Taxonomy of Counterfactual generation Methods

Counterfactual explanation methods can be broadly categorized into several families based on how counterfactual instances are generated. Following the taxonomy proposed by Sahil, Dickerson, and Hines (2010), common categories include optimization-based, generative model-based, and instance-based approaches. In addition, recent works have introduced learning-based methods, such as reinforcement learning, as a promising direction for counterfactual generation.

4.4.1 Optimization-Based Methods

Optimization-based approaches formulate counterfactual generation as a constrained optimization problem, where the objective is to find minimal changes to the input that achieve a desired prediction. These methods are flexible and model-agnostic, but often require solving an optimization problem for each instance, making them computationally expensive and sometimes producing unrealistic samples.

4.4.2 Generative Model-Based Methods

Generative approaches rely on models such as VAEs or GANs to learn the data distribution and generate counterfactuals within a latent space. These methods improve plausibility and realism by constraining generated samples to the data manifold. However, their effectiveness depends on the quality of the generative model and can be challenging for complex time series data.

4.4.3 Instance-Based Methods

Instance-based methods generate counterfactuals by retrieving or adapting existing data points, often using nearest-neighbor techniques. These approaches are computationally efficient and naturally ensure plausibility, but they are limited by dataset diversity and may fail to produce minimal or precise changes.

4.4.4 Learning-Based Methods

More recently, learning-based approaches have emerged, where models learn to directly generate counterfactuals. In particular, reinforcement learning (RL) formulates counterfactual generation as a sequential decision-making problem, enabling efficient exploration of the input space. These methods are well-suited for complex settings such as time series forecasting, but require careful design of reward functions to balance validity, proximity, and plausibility Li et al. (2026).

Taxonomy of Reinforcement Learning

Reinforcement Learning (RL) is a distinct paradigm of Machine Learning where an agent learns to make sequences of decisions by interacting with an environment. The goal of the agent is to maximize a cumulative scalar signal called the *reward*. Unlike supervised learning, RL does not require labeled input/output pairs; instead, it relies on an iterative trial-and-error process to discover which actions yield the highest long-term benefit.

The taxonomy of RL algorithms can be broadly categorized based on the agent’s strategy and its internal representation of the environment Arulkumaran et al. (2017):

- **Model-Based vs. Model-Free RL:**

- *Model-Based*: The agent attempts to learn a transition model of the environment to predict future states and rewards. This allows for planning but often suffers from model bias in complex time series.
- *Model-Free*: The agent learns directly from experience without explicitly modeling the environment’s dynamics. This approach is more robust for high-dimensional data where the underlying rules are unknown.

- **Value-Based vs. Policy-Based Methods:**

- *Value-Based (e.g., Q-Learning)*: The agent learns a value function (Q-function) to estimate the expected return for each action. It then chooses the action with the highest estimated value.
- *Policy-Based (e.g., Policy Gradient)*: The agent directly optimizes the policy $\pi(a|s)$, which is the probability distribution over actions given a state. This is particularly effective for continuous or high-dimensional action spaces.

- **Actor-Critic Methods**: This hybrid approach combines the strengths of both value-based and policy-based methods. The **Actor** proposes an action by following the current policy, while the **Critic** evaluates that action by computing a value function. This synergy reduces the variance typically found in pure policy gradient methods and speeds up convergence.

The integration of Deep Learning into these RL frameworks (Deep RL) allows agents to process complex temporal inputs, such as those found in multivariate time series, by learning latent representations of the state space.

4.5 Evaluation Metrics for Counterfactual Explanations

Evaluating counterfactual explanations (CFs) is a critical yet challenging task in explainable artificial intelligence. While numerous methods have been proposed, the literature highlights significant limitations in their evaluation, including the lack of standardized metrics and limited user-centered validation Keane et al. (2021)

Counterfactual explanations are typically assessed using computational proxy metrics that aim to capture desirable properties such as proximity, sparsity, and plausibility. However, these metrics are often not grounded in human perception, raising questions about their effectiveness in measuring explanation quality. Several core properties have emerged as essential criteria for evaluating counterfactual explanations:

Validity Validity measures whether the generated counterfactual successfully achieves the desired outcome. In the context of time series forecasting, this corresponds to verifying that the counterfactual input leads to a forecast satisfying a predefined objective (e.g., reducing the predicted values by a given percentage).

Proximity Proximity assesses how close the counterfactual instance is to the original input. Most methods rely on distance metrics such as L1 or L2 norms. While proximity is widely used, it does not necessarily guarantee plausibility, as small perturbations may still produce unrealistic samples.

Plausibility Plausibility refers to whether the generated counterfactual is realistic and consistent with the data distribution. Existing methods often approximate plausibility using distributional constraints or generative models. However, the literature highlights that plausibility remains poorly defined and is rarely validated through user studies.

Sparsity Sparsity measures the number of changes between the original instance and the counterfactual. Sparse explanations are generally preferred, as they are easier to interpret. Nevertheless, there is no consensus on the optimal level of sparsity, and empirical evidence suggests that overly minimal changes may not always lead to better explanations .

Coverage Coverage evaluates how often a method is able to generate valid and acceptable counterfactuals across a dataset. This metric is important for assessing the robustness of a method, as some approaches may fail or produce unrealistic counterfactuals in a significant number of cases .

4.6 Challenges and Trade-offs in Counterfactual Explanations

Del Ser et al. (2022) formulate the generation of counterfactual explanations as a multi-objective optimization problem. Indeed, several conflicting objectives must be satisfied simultaneously to produce meaningful and interpretable counterfactual instances.

In particular, three key objectives are traditionally considered:

- **Adversarial effectiveness:** The counterfactual must successfully change the model’s prediction toward the desired target.
- **Plausibility:** The generated counterfactual should remain realistic and lie within the data distribution, respecting physical laws and temporal correlations.
- **Change intensity (Proximity):** The modification applied to the original instance should be minimal to ensure the explanation remains relevant.

4.6.1 The Challenge of Global Perturbation and Sparsity

A significant challenge in time series counterfactuals is the issue of *Global Perturbation*. Many optimization methods tend to apply small, subtle changes across the entire temporal horizon to achieve a prediction flip. However, from a user’s perspective, modifying every single timestamp makes the explanation unreadable and lacks **actionability**. An operator cannot realistically adjust a process at every millisecond of a 24-hour window.

To address this, the concept of **structured temporal masking** is introduced. Instead of perturbing the whole sequence, masking allows the model to localize changes within specific, contiguous segments. By restricting modifications to these "saliency windows," the resulting explanation becomes more focused and provides a clear, actionable instruction: "If you change the values in this specific time interval, the outcome will change."

4.6.2 Summary of Optimization Challenges

The primary difficulty in balancing these trade-offs within multivariate time series forecasting lies in the high-dimensional and complex search space. Traditional optimization methods often struggle with:

1. **Non-differentiability:** Many plausibility constraints (such as staying within a specific data manifold) are non-differentiable, making gradient-based search difficult.
2. **Local Optima:** There is a high risk of falling into local optima that produce "adversarial examples"—samples that flip the prediction through noise—rather than "interpretable counterfactuals" that reflect real-world logic.
3. **Computational Intensity:** Querying high-capacity black-box models like the *iTransformer* repeatedly during optimization is computationally expensive.

These challenges—specifically the need to balance plausibility with localized, actionable changes—motivate the need for a more robust and intelligent search strategy. This necessity leads to our proposed approach using Reinforcement Learning, which will be the focus of the contribution part of this work.

4.7 Positioning of Counterfactual Explanations within XAI

A fundamental distinction must be drawn between counterfactual explanations and the broader family of XAI methods reviewed in Chapter 3. While attribution-based methods such as SHAP, LIME, and LRP provide valuable insights into model behavior, they remain inherently **descriptive** — they answer the question *"why did the model produce this prediction?"* but offer no guidance on *"what should be changed to obtain a different outcome?"*

This distinction is not merely semantic. In high-stakes real-world applications — such as energy grid management, financial forecasting, or clinical decision support — a passive explanation is often insufficient. An operator facing a predicted power overload cannot act on the information that *"load and temperature were the most important features."* What they need is a concrete, actionable intervention: *"if the industrial load is reduced between timesteps 60 and 80, the predicted overload disappears."*

Counterfactual explanations are the only XAI family that provides this level of actionability. This actionability gap is precisely what motivates the counterfactual approach adopted in this work. Rather than producing a static importance map, our framework generates a **modified input sequence** that directly encodes the intervention required to steer the model’s forecast toward a desired trajectory.

Furthermore, counterfactual explanations satisfy the core properties expected of any XAI method. They are **local**, focusing on individual predictions rather than global model behavior. They are **model-agnostic**, applicable to any black-box forecasting architecture. And critically for time series, they are **temporally grounded** — the explanation is expressed in the same sequential language as the data itself, making it directly interpretable by domain experts.

In this sense, counterfactual explanations do not replace attribution-based methods — they complete them. Where SHAP tells the analyst *what mattered*, a counterfactual tells them *what to do about it*.

Research Gap: Toward a Unified XAI Framework for Time Series

The review conducted in Chapters 1 through 4 reveals a structured hierarchy of limitations that collectively motivate the contribution proposed in this thesis.

At the level of XAI methods broadly, the field has reached maturity in producing descriptive explanations. Methods such as SHAP, LRP, and attention-based approaches can reliably identify which features or timesteps influenced a given prediction. However, as established in Section ??, this descriptive capability does not translate into actionable guidance. In decision-critical domains, the inability to prescribe concrete interventions represents a fundamental limitation of the current XAI paradigm.

At the level of counterfactual methods specifically, the transition from classification to forecasting remains largely unresolved. The vast majority of existing counterfactual approaches — including instance-based, optimization-based, and generative methods — were designed for discrete classification objectives. Their direct application to time series forecasting, where outputs are continuous trajectories over an extended horizon, introduces fundamental incompatibilities in objective formulation, constraint design, and evaluation.

At the level of forecasting counterfactuals, the sole dedicated method — ForecastCF Z. Wang et al. (2023) — demonstrates the feasibility of the approach but exposes three critical shortcomings. First, by operating directly in the raw input space, it provides no guarantee that generated counterfactuals remain within the data distribution, frequently producing temporally incoherent or physically implausible sequences. Second, its perturbation strategy is global, modifying the entire input sequence rather than localizing changes to actionable temporal windows — a property that is essential for real-world interpretability. Third, it requires solving a gradient-based optimization problem for each individual instance, making it computationally prohibitive at scale.

At the level of temporal plausibility, no existing counterfactual method for time series explicitly enforces the structural constraints of the data manifold — the temporal coherence and distributional realism that distinguish a valid time series modification from an adversarial perturbation. This gap is particularly acute for multivariate forecasting models such as the iTransformer, where inter-variable correlations must be preserved alongside temporal dependencies.

These four nested limitations — the actionability gap in XAI, the classification-to-forecasting gap in counterfactuals, the plausibility and sparsity gap in existing forecasting counterfactuals, and the absence of manifold-aware generation — collectively define the research gap that this thesis addresses. The contribution presented in Part ?? proposes a unified framework that simultaneously resolves each of

these limitations through the integration of a TCN-based autoencoder, a structured temporal masking mechanism, and a reinforcement learning agent operating in the learned latent space.

Part II

Contribution

Introduction

The state of the art established in Part ?? reveals a fundamental tension at the intersection of deep time series forecasting and explainable AI. On one hand, Transformer-based architectures such as the iTransformer have achieved remarkable forecasting accuracy on real-world benchmarks. On the other hand, their internal mechanisms remain opaque, and the existing XAI methods designed to explain them fall short in two critical respects: they either provide feature attributions that are not actionable, or they generate counterfactual explanations that are restricted to classification settings and cannot be directly transposed to forecasting.

The research gap identified at the end of Part ?? defines four nested limitations that no existing method simultaneously addresses: the actionability gap in XAI, the classification-to-forecasting gap in counterfactuals, the plausibility and sparsity gap in existing forecasting counterfactuals, and the absence of manifold-aware generation. This part presents our contribution, **RL-MCF**, a unified framework designed to resolve each of these limitations through the integration of three complementary components: a TCN-based autoencoder that provides a structured latent space, a temporal masking mechanism that enforces sparse and localized perturbations, and a reinforcement learning agent that learns a global counterfactual policy operating entirely within this latent space.

The contribution is organized into three chapters. Chapter ?? presents the methodology: the formal problem statement, the analysis of baseline approaches and their limitations, and the complete description of the RL-MCF framework including its components, reward function, training procedure, and evaluation metrics. Chapter ?? details the full realization of the framework, covering the experimental environment, the datasets, and the independent training and validation of each pipeline component. Chapter 7 presents the experimental results, including a comparative evaluation against baselines, an ablation study validating each design choice, and a sensitivity analysis of the key hyperparameters.

Chapter 5

Methodology

Introduction

This chapter presents our approach for generating counterfactual explanations in time series forecasting. Unlike classification tasks, forecasting involves continuous outputs and temporal dependencies, which makes the direct application of existing counterfactual methods challenging.

To address this limitation, we adapt a counterfactual generation framework originally designed for classification and redefine its objective to handle forecasting tasks. In particular, we focus on generating counterfactual time series that lead to desired changes in future predictions while remaining close to the original input and preserving temporal coherence.

The following sections describe the baseline method, the proposed adaptation, and the formulation of the counterfactual objective.

5.1 Problem Formulation

Counterfactual explanations have been extensively studied in classification settings, where the objective is to identify minimal changes to an input instance that alter the predicted class. Formally, given an input $\mathbf{x}$ and a classifier $f(\cdot)$, a counterfactual instance $\mathbf{x}^{cf}$ is sought such that $f(\mathbf{x}^{cf}) \neq f(\mathbf{x})$, while keeping $\mathbf{x}^{cf}$ as close as possible to $\mathbf{x}$. However, extending this formulation to time series forecasting introduces fundamental challenges, as the model output is no longer a discrete label but a continuous trajectory over a future horizon.

Let $\mathbf{x} \in \mathbb{R}^L$ denote a univariate time series of length L , referred to as the look-back window, and let $f : \mathbb{R}^L \rightarrow \mathbb{R}^H$ be a pre-trained black-box forecasting model producing a predicted trajectory:

$$\hat{\mathbf{y}} = f(\mathbf{x}) \in \mathbb{R}^H \quad (5.1)$$

where H denotes the forecasting horizon. A counterfactual explanation is defined as a perturbed input $\mathbf{x}^{cf} \in \mathbb{R}^L$ such that the resulting forecast $f(\mathbf{x}^{cf})$ satisfies a user-defined target constraint, while remaining as close as possible to the original input and preserving temporal realism. The counterfactual generation problem is thus formulated as finding $\mathbf{x}^{cf}$ that simultaneously satisfies three competing criteria:

- **Validity:** the counterfactual forecast must satisfy a predefined target constraint on the predicted trajectory.
- **Proximity:** the perturbation $\|\mathbf{x}^{cf} - \mathbf{x}\|$ must be minimized to ensure the explanation remains actionable and interpretable.
- **Temporal Plausibility:** the generated series must remain realistic and temporally coherent, respecting the underlying data distribution.

These three criteria are inherently conflicting — achieving high validity may require large perturbations that reduce proximity, while enforcing plausibility constrains the search space. This tension defines the core challenge of counterfactual generation for time series forecasting.

5.2 Baseline Approaches and Limitations

Two existing lines of work directly motivate our contribution: gradient-based counterfactual forecasting, and reinforcement learning-based counterfactual generation for time series classification. While each addresses part of the problem, neither is sufficient for our setting, as we detail below.

5.2.1 ForecastCF: Gradient-Based Counterfactual Forecasting

ForecastCF Z. Wang et al. (2023) is, to our knowledge, the first and only dedicated method for counterfactual explanations in time series forecasting. It defines a range-based validity objective requiring the forecasted values to lie within user-specified bounds $[\alpha, \beta]$ at each horizon step:

$$\alpha_t \leq f(\mathbf{x}^{cf})_t \leq \beta_t, \quad \forall t \in \{1, \dots, H\} \quad (5.2)$$

In ForecastCF, the target bounds are centered on the median of the original input sequence $\text{median}(\mathbf{x})$, and counterfactuals are generated via instance-specific gradient-based optimization directly in the raw input space, iteratively adjusting $\mathbf{x}^{cf}$ until the predicted trajectory satisfies the range constraint. Evaluated across multiple real-world datasets and forecasting architectures, ForecastCF demonstrates the feasibility of the counterfactual forecasting paradigm and establishes the range-based objective as a natural and interpretable formulation.

However, despite its pioneering contribution, ForecastCF exhibits three critical limitations that restrict its practical applicability:

- **Distributional Unrealism.** By operating directly in the raw input space without any distributional constraint, ForecastCF provides no guarantee that generated counterfactuals remain within the data manifold. The resulting sequences are frequently temporally incoherent or physically implausible, limiting their interpretability for domain experts.
- **Global Perturbation.** The gradient-based optimization modifies the entire input sequence uniformly, rather than localizing changes to specific actionable temporal windows. From a practical standpoint, an explanation that alters every timestep of a look-back window is neither interpretable nor actionable — a decision-maker cannot realistically intervene at every point in time.
- **Limited Scalability.** ForecastCF requires solving an independent optimization problem for each test instance, with no knowledge transfer across samples. This makes the method computationally prohibitive at scale, with reported runtimes exceeding one hour on standard benchmarks.

5.2.2 Reinforcement Learning for Counterfactual Classification

A second relevant line of work is that of Li et al. Li et al. (2026), who propose an actor-critic reinforcement learning framework for counterfactual generation in time series *classification*. In this approach, a Temporal Convolutional Network autoencoder (TCN-AE) is first trained to encode the input time series $\mathbf{x}$ into a compact latent representation $\mathbf{z} = \phi(\mathbf{x})$. A reinforcement learning agent then learns a policy to modify this latent representation in order to achieve a desired prediction outcome. The modified latent vector $\mathbf{z}^{cf}$ is decoded to produce a counterfactual instance $\mathbf{x}^{cf} = \psi(\mathbf{z}^{cf})$.

This latent-space formulation offers two key advantages over gradient-based input-space methods. First, by constraining perturbations to the learned latent space, plausibility is implicitly enforced — the decoder acts as a projection onto the data manifold, ensuring that generated counterfactuals respect the statistical structure of the training distribution. Second, once trained, the policy generates counterfactuals in a single forward pass, enabling efficient and scalable inference across any number of test instances.

However, this framework is inherently designed for classification tasks, where the objective is to switch from one discrete label to another. Such a formulation is not directly applicable to forecasting tasks for two fundamental reasons. First, the model output is no longer a discrete class but a continuous trajectory over a future horizon, making the notion of label flipping meaningless. Second, the reward function is designed around classification objectives and cannot be directly adapted to enforce range-based constraints on predicted trajectories. This limitation motivates the need to extend and adapt this framework to the forecasting setting.

5.3 Contributions

Building on the analysis of the two baseline approaches, this work addresses the problem of counterfactual explanations for time series forecasting by combining their respective strengths while resolving their individual limitations. Our main contributions are as follows:

- **Problem Formalization.** We identify and formalize the limitations of existing counterfactual methods when applied to forecasting, where outputs are continuous trajectories rather than discrete labels, and where both plausibility and actionability must be jointly enforced.
- **Novel Objective Formulation.** We redefine the range-based counterfactual objective of ForecastCF in the reinforcement learning setting by anchoring the target bounds directly on the original

forecast $\hat{\mathbf{y}}$ rather than on the median of the input, ensuring by construction that the counterfactual constraint is always non-trivial:

$$\beta_t = \hat{y}_t - \rho \cdot \sigma_{\mathbf{x}}, \quad \alpha_t = \beta_t - f_r \cdot \sigma_{\mathbf{x}} \quad (5.3)$$

where $\sigma_{\mathbf{x}} = \text{std}(\mathbf{x})$, $\rho > 0$ enforces a strict gap between $\hat{\mathbf{y}}$ and the upper bound, and $f_r > 0$ controls the band width. When a higher forecast is desired, the bounds are reflected symmetrically above $\hat{\mathbf{y}}$:

$$\alpha_t = \hat{y}_t + \rho \cdot \sigma_{\mathbf{x}}, \quad \beta_t = \alpha_t + f_r \cdot \sigma_{\mathbf{x}} \quad (5.4)$$

By construction, $\hat{y}_t \notin [\alpha_t, \beta_t]$ for all t , guaranteeing a non-trivial counterfactual objective in all cases.

- **Latent-Space RL Framework.** We extend the actor-critic framework of Li et al. (2026) from classification to forecasting by redesigning the reward function around the range-based validity objective and proximity constraints, enabling efficient policy learning that generalizes across instances in a single forward pass.
- **Temporal Masking Mechanism.** We introduce a structured temporal masking mechanism that restricts perturbations to the most recent and relevant timesteps of the look-back window, producing sparse, localized, and actionable counterfactual explanations.
- **Model-Agnostic Empirical Validation.** We provide an extensive empirical evaluation across multiple real-world datasets spanning different domains and multiple forecasting architectures, demonstrating that our framework is model-agnostic and consistently outperforms existing baselines in terms of validity, compactness and plausibility.

5.4 Framework Overview

The proposed framework, referred to as **RL-MCF** (Reinforcement Learning Masked Counterfactual Framework), is designed to generate counterfactual explanations for time series forecasting in a scalable and principled manner. Unlike gradient-based approaches that solve an independent optimization problem for each instance, RL-MCF learns a *global policy* that maps any input to a counterfactual perturbation in a single forward pass.

The overall pipeline, illustrated in Figure 5.1, consists of three main components: two frozen pre-trained module|s and one trainable agent.

The pipeline operates as follows. Given an input time series $\mathbf{x}$, a temporal mask is first applied to identify the modifiable region of the sequence. The masked input is then encoded into a compact latent representation $\mathbf{z} = \phi(\mathbf{x})$ by the pre-trained TCN encoder. The actor-critic agent receives this latent code concatenated with the original forecast $\hat{\mathbf{y}} = f(\mathbf{x})$ as its state, and outputs a perturbation action $\mathbf{a}$ in the latent space. The perturbed latent vector $\mathbf{z}^{cf} = \text{clip}(\mathbf{z} + \eta \cdot \mathbf{a}, -1, 1)$ is decoded back to the input space to produce the counterfactual candidate $\tilde{\mathbf{x}} = \psi(\mathbf{z}^{cf})$, which is then combined with the original input via the temporal mask to yield the final counterfactual $\mathbf{x}^{cf}$. The forecasting model f evaluates $\mathbf{x}^{cf}$ to produce $\hat{\mathbf{y}}^{cf}$, which is used alongside $\mathbf{x}^{cf}$ and $\mathbf{x}$ to compute the reward signal that drives the policy update.

The three main components of the framework are detailed in the following section:

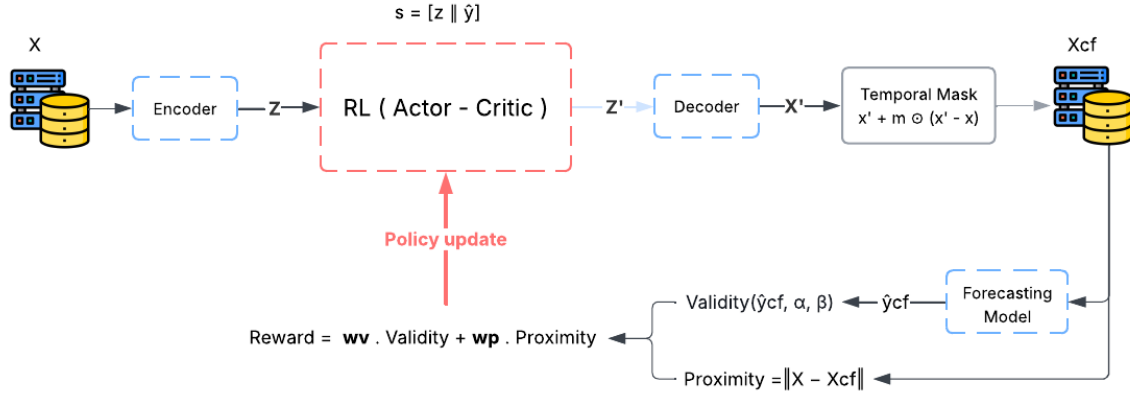


Figure 5.1: Overview of the RL-MCF framework for counterfactual forecasting. The Actor-Critic agent operates in the latent space of a pre-trained TCN autoencoder, guided by a reward signal that balances validity and proximity. The temporal masking mechanism enforces sparse and localized perturbations on the most recent timesteps of the input sequence.

- **TCN Autoencoder** (ϕ, ψ) : a pre-trained Temporal Convolutional Network autoencoder that provides a structured latent space in which perturbations are implicitly constrained to the data manifold, ensuring plausibility of generated counterfactuals.
- **Black-Box Forecasting Model** f : a pre-trained forecasting model used as a fixed evaluator. It receives both the original input $\mathbf{x}$ and the counterfactual candidate $\mathbf{x}^{cf}$, and provides the forecast signals $\hat{\mathbf{y}}$ and $\hat{\mathbf{y}}^{cf}$ used to compute the validity component of the reward. The framework is model-agnostic: any differentiable or black-box forecaster can be substituted without modifying the RL agent.
- **Actor-Critic Agent** π_θ : a trainable reinforcement learning agent that learns a global policy mapping latent states to perturbation actions. Once trained, the policy generates counterfactuals in a single forward pass, enabling efficient inference at scale.

The temporal masking mechanism, which constrains perturbations to the most recent timesteps of the look-back window, is described in detail in Section 5.5.4. The reward function, the training procedure, and the inference algorithm are presented in Sections 5.5.5, 5.6, and 5.6.2, respectively.

5.5 Framework Components

This section details each component of the RL-MCF framework: the TCN autoencoder, the black-box forecasting model, the actor-critic agent, the temporal masking mechanism, and the reward function.

5.5.1 TCN Autoencoder

To ensure that generated counterfactual instances remain realistic and preserve temporal dependencies, we employ a Temporal Convolutional Network (TCN)-based autoencoder to learn a structured latent representation of the input time series. The autoencoder consists of two components:

- An **encoder** ϕ based on Temporal Convolutional Networks that maps the input time series $\mathbf{x} \in \mathbb{R}^L$

into a compact latent representation $\mathbf{z} \in \mathbb{R}^d$:

$$\mathbf{z} = \phi(\mathbf{x}) \quad (5.5)$$

- A **decoder** ψ that reconstructs the time series from the latent representation:

$$\hat{\mathbf{x}} = \psi(\mathbf{z}) \quad (5.6)$$

The TCN encoder leverages dilated causal convolutions to capture both short-term and long-range temporal dependencies efficiently and in parallel, overcoming the sequential bottleneck of recurrent architectures. The autoencoder is trained independently to minimize the mean squared reconstruction loss:

$$\mathcal{L}_{rec} = \frac{1}{L} \|\mathbf{x} - \psi(\phi(\mathbf{x}))\|_2^2 \quad (5.7)$$

Once trained, the encoder and decoder are frozen. The latent space learned by the autoencoder plays a central role in our framework: by constraining all perturbations to this space, the decoder acts as a learned projection onto the data manifold, implicitly enforcing the plausibility of generated counterfactuals. Any modification produced in the latent space is guaranteed to be decoded into a temporally coherent sequence that respects the statistical structure of the training distribution.

5.5.2 Black-Box Forecasting Model

The forecasting model $f : \mathbb{R}^L \rightarrow \mathbb{R}^H$ is a pre-trained black-box predictor that maps any input sequence to a future trajectory. In our main experiments, we adopt the iTransformer Y. Liu et al. (2023) as the primary forecasting backbone, owing to its strong performance on multivariate time series benchmarks. However, the framework is fully model-agnostic: any forecasting architecture can be substituted without modifying the RL agent or the autoencoder.

Within the pipeline, the forecasting model serves as a fixed evaluator. It receives both the original input $\mathbf{x}$ and the counterfactual candidate $\mathbf{x}^{cf}$, producing the forecasts $\hat{\mathbf{y}} = f(\mathbf{x})$ and $\hat{\mathbf{y}}^{cf} = f(\mathbf{x}^{cf})$. These predictions are used to compute the validity component of the reward signal that guides the RL agent. Crucially, no gradient is propagated through the forecasting model during training — the agent interacts with it purely as a black box, querying it for output evaluations.

5.5.3 Actor-Critic Agent

The generation of counterfactual explanations is formulated as a sequential decision-making problem, where a reinforcement learning agent learns to perturb the latent representation of the input time series to satisfy the counterfactual objective. We adopt an actor-critic architecture, which combines the stability of value-based methods with the flexibility of policy gradient approaches.

State. The agent receives as state the concatenation of the latent code and the original forecast:

$$\mathbf{s} = [\mathbf{z} \parallel \hat{\mathbf{y}}] \in \mathbb{R}^{d+H} \quad (5.8)$$

This state representation provides the agent with both a compact encoding of the input history and the forecasting context it needs to assess how far the current prediction is from the target band.

Action. The **actor** network π_θ outputs a Gaussian distribution over perturbation actions $\mathbf{a} \in \mathbb{R}^d$, from which a sample is drawn during training and the mean is used at inference:

$$\mathbf{a} \sim \pi_\theta(\mathbf{s}), \quad \mathbf{z}^{cf} = \text{clip}(\mathbf{z} + \eta \cdot \mathbf{a}, -1, 1) \quad (5.9)$$

where η is a step-size hyperparameter controlling the magnitude of latent perturbations. The clipping operation ensures that the perturbed latent code remains within the range observed during autoencoder training, preventing out-of-distribution decoding artifacts.

Critic. The **critic** network V_ϕ estimates the expected return from state $\mathbf{s}$:

$$V_\phi(\mathbf{s}) \approx \mathbb{E}[R \mid \mathbf{s}] \quad (5.10)$$

It is used to compute the advantage for policy gradient updates, reducing variance during training and stabilizing convergence. The perturbed latent code is decoded to obtain the counterfactual candidate $\tilde{\mathbf{x}} = \psi(\mathbf{z}^{cf})$, which is then passed through the temporal masking mechanism to produce the final counterfactual $\mathbf{x}^{cf}$.

5.5.4 Temporal Masking Mechanism

A key limitation of existing counterfactual generation methods for time series is the tendency to produce global modifications across the entire input sequence. Such changes are often unrealistic and difficult to interpret, as they imply altering historical observations at every timestep. To address this issue, we introduce a structured temporal masking mechanism that restricts counterfactual modifications to the most recent and relevant region of the look-back window.

Decoding $\mathbf{z}^{cf}$ produces a full-length sequence modification $\tilde{\mathbf{x}}$. Rather than using this directly as the counterfactual, we apply a temporal mask $\mathbf{m} \in [0, 1]^L$ that blends the decoded sequence with the original input:

$$\mathbf{x}^{cf} = \mathbf{x} + \mathbf{m} \odot (\tilde{\mathbf{x}} - \mathbf{x}) \quad (5.11)$$

The mask is defined as a ramp followed by a full-activation window, concentrating modifications on the k most recent timesteps:

$$m_t = \begin{cases} 0 & \text{if } t < L - k - r \\ \frac{t - (L - k - r)}{r} & \text{if } L - k - r \leq t < L - k \\ 1 & \text{if } t \geq L - k \end{cases} \quad (5.12)$$

where k is the number of fully active timesteps and r is the length of the smooth ramp transition. This design ensures that only the k most recent observations are modified, while earlier timesteps are preserved exactly as in the original input. The smooth ramp avoids abrupt transitions that could introduce artificial discontinuities in the generated sequence.

This masking mechanism directly addresses the global perturbation limitation of ForecastCF by producing counterfactuals that are:

- **Sparse:** only a small fraction of timesteps are modified, directly improving the Compactness metric.
- **Localized:** modifications are concentrated on the most recent and forecasting-relevant region of the sequence.

- **Temporally coherent:** the preserved early timesteps maintain the original temporal structure, improving Temporal Consistency.
- **Actionable:** the modified region directly indicates where and when an intervention is needed to steer the forecast toward the target band.

5.5.5 Reward Function

The reward signal guides the actor-critic agent toward generating counterfactuals that are simultaneously valid and close to the original input. It is defined as a weighted combination of two complementary terms:

$$R = w_v \cdot r_{\text{valid}} + w_p \cdot r_{\text{prox}} \quad (5.13)$$

where w_v and w_p are hyperparameters controlling the trade-off between validity and proximity. Note that plausibility is not included as an explicit reward term, as it is implicitly enforced through the latent-space perturbation mechanism: any counterfactual decoded from a valid latent code is guaranteed to lie within the data manifold learned by the autoencoder.

Validity Term. The validity term r_{valid} measures how close the counterfactual forecast $\hat{\mathbf{y}}^{cf} = f(\mathbf{x}^{cf})$ is to the target band $[\alpha, \beta]$. Rather than using a hard binary indicator, we adopt a soft exponential formulation that provides a smooth gradient signal throughout training:

$$r_{\text{valid}} = \frac{1}{H} \sum_{t=1}^H \exp \left(-2 \cdot \frac{d_t}{\beta_t - \alpha_t} \right) \quad (5.14)$$

where $d_t = \max(0, \alpha_t - \hat{y}_t^{cf}) + \max(0, \hat{y}_t^{cf} - \beta_t)$ is the distance of the counterfactual forecast from the feasible interval at timestep t . When $\hat{y}_t^{cf} \in [\alpha_t, \beta_t]$, we have $d_t = 0$ and $r_{\text{valid}} = 1$. As the forecast moves away from the target band, the reward decays exponentially, providing a continuous incentive for the agent to steer predictions toward the admissible region.

Proximity Term. The proximity term r_{prox} penalizes the distance between the original input and the counterfactual, encouraging minimal and actionable perturbations:

$$r_{\text{prox}} = \exp \left(-\frac{1}{L} \sum_{t=1}^L |x_t^{cf} - x_t| \right) \quad (5.15)$$

The exponential formulation maps the ℓ_1 distance to a reward in $[0, 1]$, with $r_{\text{prox}} = 1$ when $\mathbf{x}^{cf} = \mathbf{x}$ and decaying smoothly as the perturbation grows. This formulation is consistent with the proximity loss used in ForecastCF Li et al. (2026), enabling direct comparison.

5.6 Training and Inference Procedures

5.6.1 Training Procedure

The training process consists of two stages: an offline pre-training phase and a reinforcement learning phase. During pre-training, the forecasting model f and the TCN autoencoder (ϕ, ψ) are trained independently on the target dataset and then frozen. The RL agent is then trained on top of these fixed components, as described in Algorithm 1.

Algorithm 1: RL-MCF: Training Procedure

Input: Training set $\mathcal{D} = \{\mathbf{x}_i\}_{i=1}^N$; Frozen forecaster f ; Frozen autoencoder (ϕ, ψ) ; Actor π_θ ; Critic V_ϕ ; Hyperparameters w_v, w_p, η, k, r

Output: Trained actor policy π_θ

for $t = 1$ **to** T **do**

Sample $\mathbf{x} \sim \mathcal{D}$;
 $\mathbf{z} \leftarrow \phi(\mathbf{x})$;
 $\hat{\mathbf{y}} \leftarrow f(\mathbf{x})$;
 $\mathbf{s} \leftarrow [\mathbf{z} \parallel \hat{\mathbf{y}}]$;
 $\mathbf{a} \sim \pi_\theta(\mathbf{s})$;
 $\mathbf{z}^{cf} \leftarrow \text{clip}(\mathbf{z} + \eta \cdot \mathbf{a}, -1, 1)$;
 $\tilde{\mathbf{x}} \leftarrow \psi(\mathbf{z}^{cf})$;
 $\mathbf{x}^{cf} \leftarrow \mathbf{x} + \mathbf{m} \odot (\tilde{\mathbf{x}} - \mathbf{x})$;
 $\hat{\mathbf{y}}^{cf} \leftarrow f(\mathbf{x}^{cf})$;
 $r \leftarrow w_v \cdot r_{\text{valid}}(\hat{\mathbf{y}}^{cf}) + w_p \cdot r_{\text{prox}}(\mathbf{x}^{cf}, \mathbf{x})$;
 Update V_ϕ : minimize $\mathcal{L}_{\text{critic}} = (r - V_\phi(\mathbf{s}))^2$;
 Update π_θ : maximize $\mathcal{L}_{\text{actor}} = (r - V_\phi(\mathbf{s})) \log \pi_\theta(\mathbf{a} \mid \mathbf{s})$;

return π_θ

5.6.2 Inference Procedure

Once training is complete, the actor policy π_θ generates counterfactuals in a single forward pass for any new input, as described in Algorithm 2.

Algorithm 2: RL-MCF: Inference Procedure

Input: Input time series $\mathbf{x}$; Frozen forecaster f ; Frozen autoencoder (ϕ, ψ) ; Trained actor π_θ ; Temporal mask $\mathbf{m}$

Output: Counterfactual $\mathbf{x}^{cf}$; Counterfactual forecast $\hat{\mathbf{y}}^{cf}$

$\mathbf{z} \leftarrow \phi(\mathbf{x})$;
 $\hat{\mathbf{y}} \leftarrow f(\mathbf{x})$;
 $\mathbf{s} \leftarrow [\mathbf{z} \parallel \hat{\mathbf{y}}]$;
 $\mathbf{a} \leftarrow \mathbb{E}[\pi_\theta(\mathbf{s})]$;
 $\mathbf{z}^{cf} \leftarrow \text{clip}(\mathbf{z} + \eta \cdot \mathbf{a}, -1, 1)$;
 $\tilde{\mathbf{x}} \leftarrow \psi(\mathbf{z}^{cf})$;
 $\mathbf{x}^{cf} \leftarrow \mathbf{x} + \mathbf{m} \odot (\tilde{\mathbf{x}} - \mathbf{x})$;
 $\hat{\mathbf{y}}^{cf} \leftarrow f(\mathbf{x}^{cf})$;
return $(\mathbf{x}^{cf}, \hat{\mathbf{y}}^{cf})$

5.7 Evaluation Metrics

We evaluate counterfactual quality along six complementary axes: validity, proximity, compactness, temporal quality, plausibility, and stepwise validity.

5.7.1 Validity Ratio

The validity ratio measures the proportion of predicted counterfactual values that fall within the target bounds across all timesteps and samples:

$$\text{Validity} = \frac{1}{K \cdot H} \sum_{k=1}^K \sum_{t=1}^H \mathbf{1} \left[\alpha_{k,t} \leq f(\mathbf{x}_k^{cf})_t \leq \beta_{k,t} \right] \quad (5.16)$$

where K is the number of test instances and H the forecasting horizon. A value of 1 indicates that all predicted steps satisfy the range constraint for all instances.

5.7.2 Stepwise Validity AUC

The stepwise AUC captures how many steps per instance satisfy the constraint. For each instance k , let c_k denote the number of valid timesteps. We define the cumulative validity curve:

$$\phi(\tau) = \frac{1}{K} \sum_{k=1}^K \mathbf{1}[c_k \geq \tau], \quad \tau \in \{0, \dots, H\} \quad (5.17)$$

and compute its normalized area under the curve:

$$\text{AUC} = \int_0^1 \phi(\tau) d\tau \quad (5.18)$$

A higher AUC indicates that counterfactuals are valid across a larger fraction of the horizon per instance.

5.7.3 Proximity

Proximity measures how much the counterfactual deviates from the original input:

$$\text{Proximity} = \frac{1}{K} \sum_{k=1}^K \left\| \mathbf{x}_k^{cf} - \mathbf{x}_k \right\|_2 \quad (5.19)$$

Lower values indicate more actionable and minimal explanations.

5.7.4 Compactness

Compactness measures the proportion of timesteps that remain effectively unchanged, where the perturbation is below a tolerance threshold ϵ :

$$\text{Compactness} = \frac{1}{K \cdot L} \sum_{k=1}^K \sum_{t=1}^L \mathbf{1}[|x_{k,t}^{cf} - x_{k,t}| \leq \epsilon] \quad (5.20)$$

where $\epsilon = 10^{-3}$ by default. Higher compactness reflects more localized perturbations, directly improved by the temporal masking mechanism.

5.7.5 Roughness Ratio

To assess temporal smoothness, we compute the mean absolute first-order difference of both the original and counterfactual sequences:

$$\text{Roughness}(\mathbf{x}) = \frac{1}{L-1} \sum_{t=1}^{L-1} |x_{t+1} - x_t| \quad (5.21)$$

We report the roughness ratio $\rho_r = \text{Roughness}(\mathbf{x}^{cf}) / \text{Roughness}(\mathbf{x})$, averaged over all instances. A ratio close to 1 indicates that the counterfactual preserves the temporal dynamics of the original series.

5.7.6 Temporal Consistency

Temporal consistency measures whether the counterfactual preserves the directional dynamics of the original series. For each instance, we compute the Pearson correlation between the first-order differences

of both sequences:

$$\text{TC}_k = \frac{1}{2} \left(1 + \text{Pearson}(\Delta \mathbf{x}_k, \Delta \mathbf{x}_k^{cf}) \right) \quad (5.22)$$

where $\Delta \mathbf{x}_t = x_{t+1} - x_t$. Higher values indicate that the counterfactual follows the same temporal trends as the original input.

5.7.7 Plausibility

Plausibility evaluates whether the generated counterfactuals resemble realistic time series drawn from the training distribution. We train three anomaly detectors on the training set: Isolation Forest (IF), Local Outlier Factor (LOF), and One-Class SVM (OC-SVM). Each detector produces a normalized anomaly score in $[0, 1]$, where lower values indicate that the counterfactual is closer to the training distribution. We report the ensemble score:

$$\text{Plausibility} = \frac{1}{3} (s_{\text{IF}} + s_{\text{LOF}} + s_{\text{OC-SVM}}) \quad (5.23)$$

Since plausibility is implicitly encouraged by constraining perturbations to the latent space of the autoencoder, we expect our method to produce lower anomaly scores than gradient-based approaches operating directly in the input space.

Conclusion

This chapter presented the RL-MCF framework for counterfactual generation in time series forecasting. Starting from the limitations of gradient-based and classification-oriented baselines, we introduced a principled reinforcement learning approach that combines latent-space policy learning with a temporal masking mechanism to produce valid, compact, and plausible counterfactual explanations efficiently. The following chapter describes the experimental setup and presents the empirical validation of the proposed framework across multiple datasets and forecasting architectures.

Chapter 6

Realization

Introduction

This chapter describes the complete implementation of the **RL-MCF** framework, from the experimental environment to the end-to-end counterfactual generation pipeline. It is organized as follows.

Section 6.1 presents the development environment and the software stack used throughout the project. Section 6.2 describes the three benchmark datasets and motivates the configuration choices through a systematic temporal structure analysis. Sections 6.3–6.5 detail the independent training of each pipeline component: the forecasting models, the TCN autoencoder, and the anomaly detector ensemble. Section 6.6 then covers the reinforcement learning agent, including its architecture, reward function, and temporal masking mechanism. Finally, Section ?? presents the end-to-end pipeline that connects all components, along with a sensitivity analysis of the key hyperparameters and a step-by-step inference walkthrough.

Each component is validated independently before being frozen and integrated into the full pipeline. The experimental results and comparative evaluation against baselines are reported in Chapter 7.

6.1 Development Environment

The **RL-MCF** framework was implemented in Python 3.10 and developed using Visual Studio Code. All deep learning components were built on top of PyTorch, while data manipulation, statistical analysis, and anomaly detection relied on the standard scientific Python ecosystem. Table 6.1 summarises the full software stack.

Table 6.1: Libraries and frameworks used throughout the project.

Library	Role
PyTorch $\geq 2.0.0$	Deep learning framework
NumPy $\geq 1.24.0$	Numerical computation
Pandas $\geq 1.5.0$	Data loading and preprocessing
Scikit-learn $\geq 1.2.0$	Anomaly detectors, scaling
SciPy $\geq 1.9.0$	Signal processing utilities
Matplotlib $\geq 3.6.0$	Visualisation
TensorBoard $\geq 2.11.0$	Training monitoring
Einops $\geq 0.6.0$	Tensor manipulation
tqdm $\geq 4.64.0$	Progress tracking
PyYAML ≥ 6.0	Configuration management

Computationally intensive experiments were executed on Google Colaboratory using an NVIDIA Tesla T4 GPU (15 GB VRAM), which provided sufficient memory for training all five forecasting architectures as well as the TCN autoencoder and the reinforcement learning agent. Version control was managed via Git, and the full codebase is publicly available at:

<https://github.com/bouchrabenkhelifa/counterfactual-forecasting-rl>

6.2 Datasets

The experimental evaluation is conducted on three publicly available benchmark datasets spanning two distinct domains: energy systems and meteorology. They were selected to reflect a diversity of temporal structures, sampling frequencies, and signal complexities, ensuring that the RL-MCF framework is evaluated under genuinely varied conditions. Table 6.2 summarises their key characteristics.

Table 6.2: Summary of datasets used in the experiments.

Dataset	Frequency	Length	Target variable	Domain
ETTh1	Hourly	17,420	OT (oil temperature)	Energy
ETTh2	Hourly	17,420	OT (oil temperature)	Energy
Weather	10-minute	52,696	WetBulbCelsius	Meteorology

To inform the configuration choices made in Section 6.3.2, each dataset is subjected to an STL decomposition that separates the signal into its trend, seasonal, and residual components. The results are presented below, one dataset at a time.

6.2.1 ETTh1

ETTh1 records oil temperature from an electrical transformer station in China at hourly resolution over approximately two years. It is the primary dataset used throughout this work and serves as the reference configuration for all ablation and sensitivity analyses.

Figure 6.1 shows the STL decomposition with period = 24 hours. The series is strongly trend-dominated: the trend component alone accounts for $\approx 86\%$ of the total variance, reflecting a slow and persistent downward drift over the observation window. The seasonal component ($\approx 7\%$) captures a stable daily cycle driven by the 24-hour operational rhythm of the transformer station, while the residual ($\approx 6\%$) is the smallest among the three datasets — confirming that the signal is well-explained by its structural components with minimal unexplained noise.

From a modeling perspective, the low residual variance implies that a well-trained forecaster should achieve low prediction error, a necessary condition for reliable counterfactual validity signals during RL training. A look-back window of $L = 96$ (four complete daily cycles) and a forecasting horizon of $H = 48$ (two cycles ahead) are directly motivated by this structure, as derived in Section 6.3.2.



Figure 6.1: STL decomposition of the ETTh1 target variable (OT, period = 24h). Trend ($\approx 86\%$), seasonal ($\approx 7\%$), residual ($\approx 6\%$). Green vertical line: `seq_len = 96`; orange vertical line: `pred_len = 48`.

6.2.2 ETTh2

ETTh2 shares the same schema and recording period as ETTh1 but originates from a different transformer station, exhibiting meaningfully different temporal dynamics.

Figure 6.2 shows the STL decomposition under the same configuration (period = 24 hours). The most striking difference from ETTh1 is the seasonal component, which accounts for $\approx 20\%$ of the total variance — nearly three times its contribution in ETTh1 — indicating that daily cycles play a far more prominent role in driving the signal. The trend component ($\approx 65\%$) remains dominant but less overwhelming, and the residual ($\approx 10\%$) is noticeably larger, reflecting greater short-term variability and structural complexity.

These differences have direct implications for counterfactual generation. A stronger seasonal component means the forecasting model must track more pronounced oscillations, making it harder to steer predictions into a target band through localised input perturbations. Together with the larger residual, these factors explain why ETTh2 is systematically harder than ETTh1 in Chapter 7: validity scores are lower and larger perturbations are required to satisfy the range constraint.

6.2.3 Weather

The Weather dataset records 21 meteorological variables at 10-minute intervals across a full year, with wet-bulb temperature (`WetBulbCelsius`) as the forecasting target. Its higher sampling frequency, larger variate count, and distinct signal structure make it a more challenging and complementary testbed.

Figure 6.3 shows the STL decomposition with period = 24 h (= 144 steps at 10-minute resolution). The Weather dataset exhibits a fundamentally different profile from both ETT datasets. While the trend component remains dominant ($\approx 76\%$), reflecting the gradual seasonal evolution of atmospheric temperature across the year, the periodic component is almost negligible ($\approx 2.3\%$): daily cycles contribute

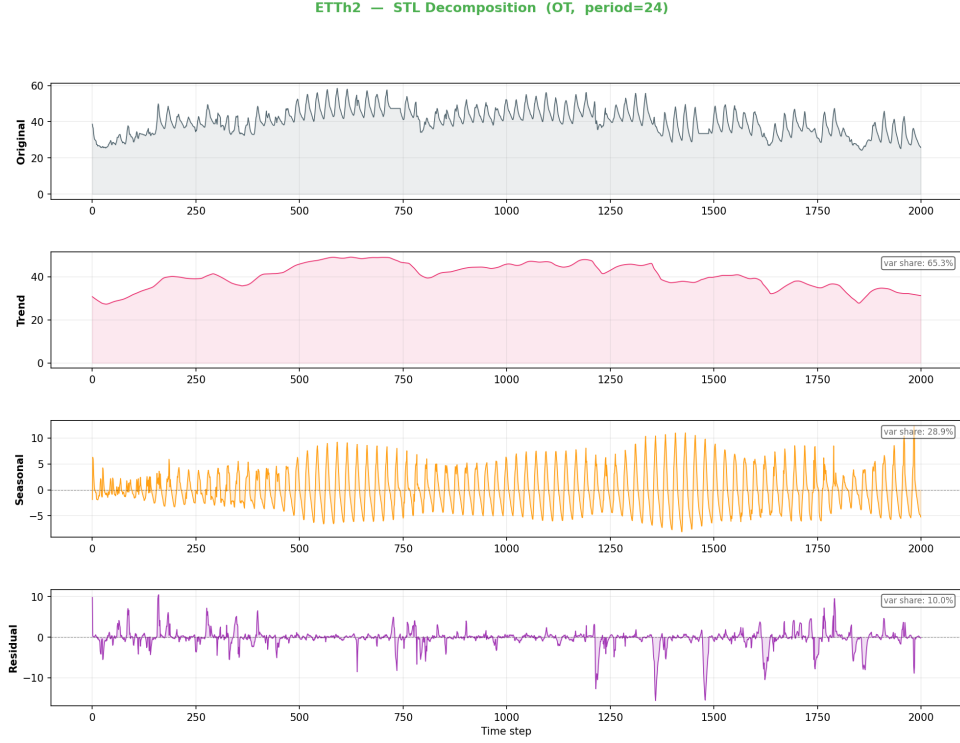


Figure 6.2: STL decomposition of the ETTh2 target variable (OT, period = 24h). Trend ($\approx 65\%$), seasonal ($\approx 20\%$), residual ($\approx 10\%$). The stronger seasonal share and larger residual make ETTh2 harder than ETTh1.

minimally to the overall dynamics, a direct consequence of the physical nature of wet-bulb temperature, which is governed by synoptic-scale weather patterns rather than fixed daily rhythms. The residual ($\approx 13\%$) is the largest among the three datasets, indicating that a substantial fraction of the signal is driven by irregular atmospheric events that neither trend nor seasonality can capture.

This structural profile — trend-dominated, seasonality-poor, and noise-rich — makes the Weather dataset a genuinely distinct testbed and justifies dedicated hyperparameter configurations, particularly regarding the band width f_r and the number of active mask steps k (see Section ??).

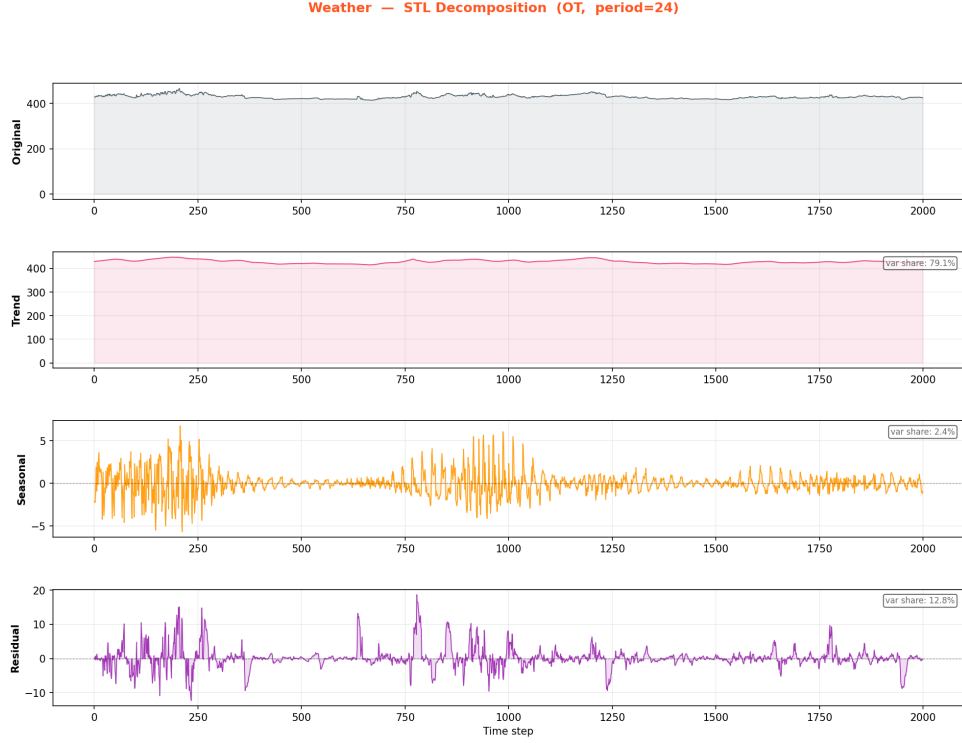


Figure 6.3: STL decomposition of the Weather target variable (WetBulbCelsius, period = 24 h = 144 steps at 10-minute resolution). Trend ($\approx 76\%$), seasonal ($\approx 2.3\%$), residual ($\approx 13\%$). The near-absent seasonality and large residual distinguish this dataset from the ETT series.

6.2.4 Synthesis

The three datasets collectively span a broad spectrum of temporal complexity. Table 6.3 consolidates the key structural characteristics extracted from the STL decompositions and the configuration choices they motivate. This analysis confirms that the datasets are not interchangeable and that a single fixed configuration would be suboptimal across all settings. Dataset-specific configurations are detailed in Section ?? and all configuration files are available in the repository under `assets/configs/`.

Table 6.3: Synthesis of STL decomposition results and modeling implications.

Dataset	Trend (%)	Seasonal (%)	Residual (%)	Dominant structure	Difficulty	Key implication
ETTh1	≈ 86	≈ 7	≈ 6	Trend	Low	$L=96, k=12$, standard config
ETTh2	≈ 65	≈ 20	≈ 10	Trend + Seasonal	Medium	Larger perturbations needed
Weather	≈ 76	≈ 2.3	≈ 13	Trend + Residual	High	Higher f_r , adjusted k

6.2.5 Data Preparation

Raw data is stored as CSV files containing a date column and one column per feature variable. In univariate mode (`features='S'`), only the target column is retained:

```

1 if self.features == 'S':
2     df_data = df_raw[[self.target]]
3     # "OT" for ETT, "T (degC)" for Weather

```

Listing 6.1: Univariate target selection.

Chronological split. Splits are strictly chronological to preserve temporal causality — no shuffling is applied at any stage. The partition boundaries for ETTh1/ETTh2 are fixed as follows:

```

1 # 12 months train / 4 months val / 4 months test
2 border1s = [0,
3             12*30*24 - seq_len,
4             12*30*24 + 4*30*24 - seq_len]
5 border2s = [12*30*24,
6             12*30*24 + 4*30*24,
7             12*30*24 + 8*30*24]

```

Listing 6.2: Chronological split boundaries (ETTh1/ETTh2).

Table 6.4: Dataset split sizes after windowing ($L = 96$, $H = 48$).

Dataset	Train	Validation	Test
ETTh1 / ETTh2	8 640 samples (70%)	2 880 samples (10%)	2 880 samples (20%)
Weather	36 696 samples (70%)	5 242 samples (10%)	10 444 samples (20%)

Normalisation. A `StandardScaler` is fitted exclusively on the training split and then applied to the full dataset, preventing any data leakage:

```

1 scaler = StandardScaler()
2 scaler.fit(train_data.values) # mu, sigma computed on train only
3 data = scaler.transform(df_data.values) # applied to train + val + test

```

Listing 6.3: Leak-free normalisation pipeline.

Each feature is zero-centred ($\mu = 0$) and unit-scaled ($\sigma = 1$). The fitted scaler is saved alongside the autoencoder checkpoint to allow de-normalisation at inference time.

Sliding window. The normalised series is segmented into overlapping windows with a stride of 1:

```

1 seq_x = data[i : i + seq_len] # input: (96, 1)
2 seq_y = data[i + seq_len - label_len :
3             i + seq_len + pred_len] # target: (96, 1)

```

Listing 6.4: Sliding window segmentation.

Temporal encoding. Two encoding schemes are used depending on the model family:

```

1 # timeenc=1: frequency encoding for iTransformer and PatchTST
2 data_stamp = time_features(dates, freq='h')
3 # -> [hour_sin, hour_cos, day_sin, day_cos, ...]
4
5 # timeenc=0: raw encoding for GRU, TimesNet, DLinear
6 data_stamp = [month, day, weekday, hour]

```

Listing 6.5: Temporal encoding schemes.

DataLoader. Training batches are shuffled; validation and test sets are evaluated in order:

```

1 DataLoader(dataset,
2             batch_size=32,
3             shuffle=(flag == 'train'),
4             drop_last=True)

```

Listing 6.6: DataLoader configuration.

6.3 Forecasting Model Training

The counterfactual generation framework requires a frozen, reliable forecaster to evaluate whether a perturbed input produces a forecast that falls within the target band $[\alpha, \beta]$. The quality of this validity signal depends directly on the forecaster’s accuracy: a poorly calibrated model would produce noisy or misleading rewards during RL training, undermining the entire pipeline. Five architectures are therefore trained and evaluated independently on each dataset before being frozen and integrated into the RL loop.

6.3.1 Model Overview

The five architectures span the main families of deep time series models surveyed in Chapter ??, and were selected to cover a representative range of inductive biases — from attention-based to convolutional, recurrent, and linear models. Table 6.5 summarises their key characteristics.

Table 6.5: Summary of the five forecasting architectures trained in this work.

Model	Family	Tokenisation	Key mechanism
iTransformer	Transformer	Variable-wise	Cross-variate attention; primary RL-MCF backbone
PatchTST	Transformer	Time patch	Patch self-attention over local segments
TimesNet	CNN	1-D $\rightarrow$ 2-D	2-D temporal convolution
GRU	Recurrent	Timestep-wise	Gated recurrent units
DLinear	Linear	Trend/Residual	Decomposed linear projection

Each model is trained independently on each of the three datasets using a common configuration ($L = 96$, $H = 48$, univariate target) and then frozen. The iTransformer serves as the primary backbone for RL-MCF, as its inverted tokenisation is particularly well-suited to the trend-dominated structure shared by all three datasets (Section 6.2.4): by treating each variate as a token, it captures inter-variable correlations at the macro scale rather than at individual timesteps, which aligns with the slow-moving trend components identified in the STL analysis.

6.3.2 Observation Window and Forecasting Horizon Selection

The choice of look-back window L and forecasting horizon H is grounded directly in the temporal structure revealed by the STL and ACF analyses. As established in Section 6.2, the dominant periodicity is 24 steps for ETTh1 and ETTh2 (hourly resolution) and 144 steps for Weather (10-minute resolution), both corresponding to a 24-hour daily cycle.

Step 1 — Dominant period. The ACF analysis presented in Figure 6.4 confirms a clear periodic peak at lag ≈ 24 for the ETT datasets and at lag ≈ 144 for Weather. This daily structure is physically consistent

with the nature of the data: ETTh1 and ETTh2 follow the industrial load profile of transformer stations H. Zhou et al. (2021), while wet-bulb temperature is governed by the diurnal solar cycle. Importantly, the near-absent seasonality of the Weather dataset ($\approx 2.3\%$ of variance, Section 6.2.3) means that this dominant period is far weaker in practice — a limitation acknowledged in the dedicated configuration for that dataset.

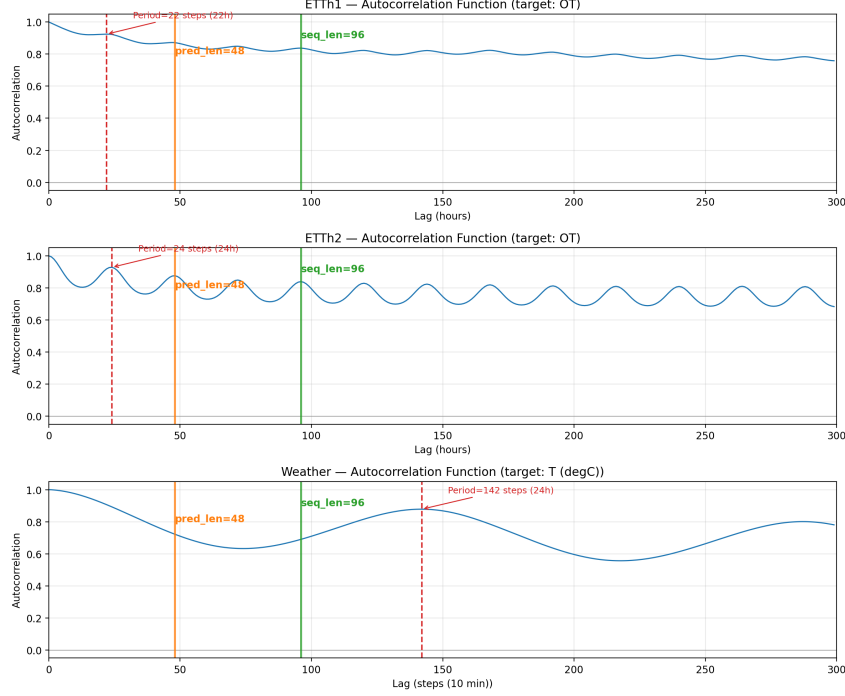


Figure 6.4: Autocorrelation function (ACF) of the target variable for each dataset after linear detrending. ETTh1 and ETTh2 (top, middle): dominant peak at lag ≈ 24 (24-hour daily cycle, 1 step = 1 hour). Weather (bottom): dominant peak at lag ≈ 144 (24-hour cycle, 1 step = 10 minutes). Green vertical line: $\text{seq_len} = 96$; orange vertical line: $\text{pred_len} = 48$.

Step 2 — Observation window. We adopt the standard rule of thumb that the look-back window should span at least four complete periods of the dominant cycle, giving the model sufficient context to identify the periodic structure and separate it from noise:

$$L = 4 \times T_{\text{dominant}} \quad (6.1)$$

For ETTh1 and ETTh2, this yields $L = 4 \times 24 = 96$ steps — four complete daily cycles, consistent with the strong and stable seasonality identified for these datasets ($\approx 7\%$ and $\approx 20\%$ of variance respectively). The same value $L = 96$ is applied to Weather for cross-dataset comparability: it is the standard benchmark configuration used by iTransformer Y. Liu et al. (2023), PatchTST Nie et al. (2023), and TimesNet. For Weather, this covers only $96/144 \approx 0.67$ daily cycles — a known limitation given its negligible seasonality, but one that is offset by the trend-dominated nature of the signal (Section 6.2.3).

Step 3 — Forecasting horizon. The prediction horizon is set to two dominant periods:

$$H = 2 \times T_{\text{dominant}} = 2 \times 24 = 48 \text{ steps} \quad (6.2)$$

For ETTh1 and ETTh2, this corresponds to a 48-hour (two-day) forecast — a practically meaningful horizon for energy management applications and one that enables direct comparison with ForecastCF Z.

Wang et al. (2023), which was evaluated under identical conditions. Table 6.6 consolidates these choices.

Table 6.6: Observation window and forecasting horizon for each dataset.

Dataset	Resolution	Dominant period	In hours	seq_len	pred_len
ETTh1	60 min	24 steps	24 h	96	48
ETTh2	60 min	24 steps	24 h	96	48
Weather	10 min	144 steps	24 h	96 [†]	48

[†] Kept identical across datasets for benchmark comparability; covers only 0.67 daily cycles for Weather.

6.3.3 Configuration

All five models share the same experimental protocol. Model-specific hyperparameters (model dimension, number of layers, attention heads) were selected via grid search and stored in the configuration files under `assets/configs/{dataset}_dataset/forecasters/`. The shared training configuration is reported in Table 6.7.

Table 6.7: Shared training configuration for all forecasting models.

Hyperparameter	Value	Applies to
Look-back window L	96	All models
Forecasting horizon H	48	All models
Target variable	Univariate	All models
Optimiser	Adam	All models
Loss function	MSE	All models
Early stopping patience	10	All models
Batch size	32	All models
Learning rate	1×10^{-4}	All models
Max epochs	50	All models
Normalisation	RevIN	iTransformer, PatchTST

RevIN is applied only to iTransformer and PatchTST, whose attention mechanisms are sensitive to distribution shift across windows — a particularly relevant consideration for ETTh1, whose strong downward trend ($\approx 86\%$ of variance) means that the mean of the series drifts substantially over the observation period.

6.3.4 Implementation

All five forecasters share a single training entry point, `src/experiments/forecasting/run.py`. The script reads `model_type` from the JSON configuration file and routes automatically to the appropriate trainer — only the configuration file changes between runs.

```

1 configs = load_config(args.config)
2 model_type = configs.model_type.lower()
3 # "itransformer", "gru", "dlinear", "patchtst", "timesnet"
4
5 trainer = GenericForecasterTrainer(configs)
6 trainer.train()
```

Listing 6.7: Automatic trainer routing based on `model_type`.

The same training loop is shared by all architectures. The model is instantiated dynamically via `_build_model(configs)`, which reads `model_type` from the configuration:

```

1 class GenericForecasterTrainer:
2     def __init__(self, configs):
3         self.model = _build_model(configs, device)
4         self.optimizer = Adam(self.model.parameters(),
5                               lr=configs.learning_rate)
6         self.criterion = MSELoss()
7
8     def train(self):
9         early_stopping = EarlyStopping(patience=configs.patience)
10        for epoch in range(configs.train_epochs):
11            for batch_x, batch_y, batch_x_mark, batch_y_mark in ↵
12                ↵ train_loader:
13                outputs = self.model(batch_x, batch_x_mark)
14                loss = self.criterion(outputs[:, -pred_len:],
15                                     batch_y[:, -pred_len:])
16
17                loss.backward()
18                self.optimizer.step()
19
20            val_loss = self.validate(val_loader)
21            early_stopping(val_loss, self.model, checkpoint_path)
22            if early_stopping.early_stop:
23                break

```

Listing 6.8: `GenericForecasterTrainer` — common training loop.

This unified entry point means that reproducing any individual model requires only substituting the configuration file, with no changes to the training code itself.

6.3.5 Results Summary

Table 6.8 reports the iTransformer test performance across all three datasets under the shared configuration ($L = 96$, $H = 48$, univariate target).

Table 6.8: iTransformer forecasting performance on the test set.

Dataset	Avg MSE	RMSE (°C)	Pred Len
ETTh1	0.0460	0.91	48
ETTh2	0.0861	1.50	48
Weather	0.1487	2.48	48

Figure 6.5 provides a visual comparison of the original and predicted series for each dataset.

These results are consistent with the temporal structure identified in Section 6.2. ETTh1 achieves the lowest error (MSE = 0.0460, RMSE = 0.91 °C): its strong trend dominance ($\approx 86\%$ of variance) and minimal residual ($\approx 6\%$) make it the most predictable dataset, with sub-degree prediction accuracy. ETTh2 yields a higher error (MSE = 0.0861, RMSE = 1.50 °C), reflecting its stronger seasonal oscillations ($\approx 20\%$) and larger residual ($\approx 10\%$). Weather presents the highest error (MSE = 0.1487, RMSE = 2.48 °C), consistent with its noise-rich profile ($\approx 13\%$ residual) and near-absent seasonality ($\approx 2.3\%$), which makes the signal harder to extrapolate reliably.

This ordering — ETTh1 < ETTh2 < Weather — directly determines the difficulty of the downstream

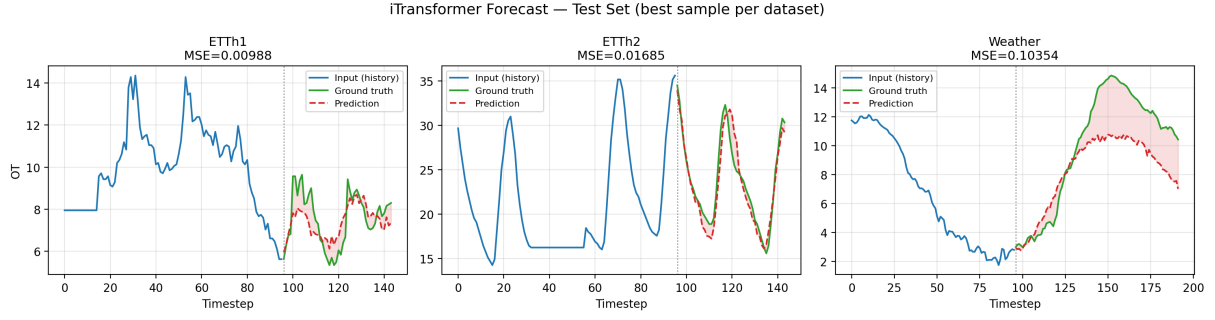


Figure 6.5: iTransformer forecasting results on the test set for ETTh1, ETTh2, and Weather. The predicted series (dashed orange) closely tracks the ground truth (solid blue) across all three datasets, with error increasing from ETTh1 to Weather, consistent with their residual variance profiles (Table 6.3).

counterfactual task: a less accurate forecaster produces noisier validity signals during RL training, which explains the systematic degradation in counterfactual quality observed across datasets in Chapter 7.

The full comparative evaluation across all five architectures and three datasets is presented in Chapter 7, Section ??.

6.3.6 Reproducing the Results

```
1 python src/experiments/forecasting/run.py \
2     --config assets/configs/{DATASET}_dataset/forecasters/
3     {MODEL}/{DATASET}_96_48_S.json
```

Listing 6.9: Training command.

where $\{\text{DATASET}\} \in \{\text{etth1}, \text{etth2}, \text{weather}\}$ and $\{\text{MODEL}\} \in \{\text{itransformer}, \text{patchtst}, \text{timesnet}, \text{gru}, \text{dlinear}\}$.

6.4 TCN Autoencoder Training

The TCN autoencoder serves as the backbone of the latent-space perturbation strategy introduced in Chapter ?. Rather than modifying the raw input series directly, the RL agent operates in the compact, bounded latent space $\mathcal{Z} = [-1, 1]^d$ learned by the autoencoder — a design choice that implicitly constrains generated counterfactuals to remain close to the training distribution, without requiring any explicit plausibility term in the reward function. The autoencoder is trained independently on each dataset and then frozen before RL training begins.

6.4.1 Architecture and Implementation

The encoder φ maps an input time series $\mathbf{x} \in \mathbb{R}^{T \times C}$ to a latent vector $\mathbf{z} \in [-1, 1]^d$ via a stack of N dilated temporal convolutional blocks. Exponentially increasing dilation rates give the encoder a receptive field of $2^N - 1$ time steps, sufficient to capture long-range dependencies across the full input window of $T = 96$ — a window size motivated by the dominant 24-hour periodicity identified in Section 6.2.

```
1 class TCNEncoder(nn.Module):
2     def __init__(self, seq_len, n_features=1, hidden_dim=64,
3                 latent_dim=64, n_blocks=6, kernel_size=3,
4                 dropout=0.1):
```

```

5     super().__init__()
6     # 1x1 convolution: project input channels -> hidden_dim
7     self.proj = nn.Conv1d(n_features, hidden_dim, 1)
8
9     # dilations: 1, 2, 4, 8, 16, 32 -> RF = 127 steps
10    self.blocks = nn.ModuleList([
11        TCNBlock(hidden_dim, kernel_size, 2**i, dropout)
12        for i in range(n_blocks)
13    ])
14    # Linear projection to latent space
15    self.fc = nn.Linear(hidden_dim, latent_dim)
16    self.tanh = nn.Tanh()
17
18    def forward(self, x):
19        out = self.proj(x.transpose(1, 2))
20        for b in self.blocks:
21            out = b(out)
22        return self.tanh(self.fc(out.mean(2)))

```

Listing 6.10: TCN encoder architecture.

The `Tanh` activation at the output bounds the latent vector in $[-1, 1]^d$, which is critical for the RL agent: the actor samples perturbations within this bounded space, and the proximity penalty $\rho \cdot \sigma_{\text{global}}$ provides a principled clipping radius that prevents the agent from escaping the decoder’s training distribution.

6.4.2 Configuration

Training minimises the mean squared reconstruction loss over windows of length $L = 96$:

$$\mathcal{L}_{\text{rec}} = \frac{1}{L} \|\mathbf{x} - \psi(\varphi(\mathbf{x}))\|_2^2 \quad (6.3)$$

The main configuration choices are driven by the temporal characteristics established in Section 6.2:

- **Latent dimension d :** selected via grid search to balance reconstruction quality and the compactness of the search space available to the RL agent. A latent space that is too small loses the temporal detail needed to reconstruct the seasonal oscillations present in ETTh2 ($\approx 20\%$ seasonal variance); one that is too large makes the agent’s perturbation task harder and weakens the implicit plausibility guarantee.
- **Dilation factors:** growing exponentially with depth ($d = 1, 2, 4, 8, 16, 32$), yielding a receptive field of 127 steps. This covers the full $L = 96$ window and ensures that both the short-term daily oscillations (ETTh1, ETTh2) and the slower atmospheric patterns of the Weather dataset are captured in the latent representation.
- **Causal convolutions:** ensure that the encoder never accesses future timesteps, preserving the temporal causality of the representation — a strict requirement since the autoencoder is applied to the same observation window used by the frozen forecaster.

6.4.3 Results and Analysis

Figure 6.6 shows the reconstruction quality of the TCN autoencoder on the test set of all three datasets. Table 6.9 reports the corresponding MSE and MAE values.

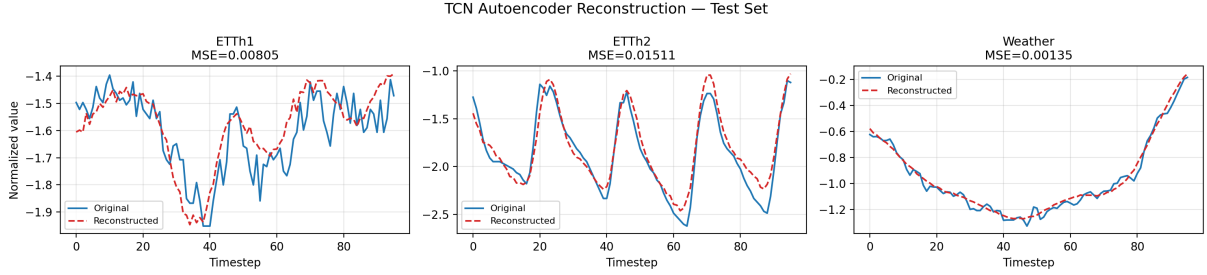


Figure 6.6: TCN autoencoder reconstruction on the test set for ETTh1, ETTh2, and Weather. The reconstructed series (dashed red) closely tracks the original (solid blue) across all three datasets, recovering both low-frequency trends and higher-frequency oscillations. MSE values are reported per dataset.

Table 6.9: TCN autoencoder reconstruction performance on the test set.

Dataset	MSE	MAE
ETTh1	0.007997	0.067749
ETTh2	0.008710	0.069815
Weather	0.001072	0.023361

Several observations emerge from these results, each consistent with the temporal structure identified in Section 6.2:

- **ETTh1** achieves $\text{MSE} = 0.007997$, well below the iTransformer forecasting error of 0.0460 (Table 6.8). This ordering is an important sanity check: if the autoencoder could not reconstruct the input with at least the same fidelity as the forecaster, perturbations in the latent space would introduce reconstruction artifacts larger than the forecasting signal itself, making the validity criterion meaningless. ETTh1’s low residual variance ($\approx 6\%$) makes it the easiest dataset to reconstruct, as the signal is almost entirely explained by its trend and seasonal components.
- **ETTh2** yields a slightly higher $\text{MSE} = 0.008710$, consistent with its larger residual component ($\approx 10\%$) and stronger seasonal oscillations ($\approx 20\%$ of variance). The autoencoder must capture more pronounced periodic patterns to achieve faithful reconstruction, which explains the marginal increase in error relative to ETTh1.
- **Weather** achieves the lowest reconstruction error ($\text{MSE} = 0.001072$), despite being the most challenging dataset for counterfactual generation. This apparent paradox is explained by the structural profile identified in Section 6.2.3: the near-absent seasonality ($\approx 2.3\%$) and dominant trend ($\approx 76\%$) make the signal intrinsically smooth and easy to reconstruct, even though its high residual variance ($\approx 13\%$) makes counterfactual steering harder.

Across all three datasets, Figure 6.6 confirms that the autoencoder faithfully recovers both the dominant trend and the periodic oscillations of each series. This reconstruction quality validates the central assumption of the RL-MCF framework: perturbations applied in the latent space $\mathcal{Z}$ are decoded back into temporally coherent sequences that respect the statistical structure of the training distribution, implicitly enforcing plausibility without any explicit constraint in the reward function.

6.4.4 Reproducing the Results

```
1 python src/experiments/autoencoder/run.py \
2     --config assets/configs/{DATASET}_dataset/ae/tcn_ae.json
```

Listing 6.11: Autoencoder training command.

where $\{\text{DATASET}\} \in \{\text{etth1}, \text{etth2}, \text{weather}\}$.

6.5 Anomaly Detector Training

The plausibility of generated counterfactuals is a central quality criterion of the RL-MCF framework: a counterfactual that steers the forecast into the target band but produces a statistically unrealistic input series is not a useful explanation. As introduced in Section ??, plausibility is measured by an ensemble of three anomaly detectors trained on the training split of each dataset. Crucially, these detectors are frozen after training and used exclusively at evaluation time — they play no role in the RL training loop and do not influence the reward signal. This design is deliberate: plausibility is not optimised directly but is expected to emerge implicitly from the latent-space perturbation strategy, since the TCN autoencoder already constrains all modifications to remain within the learned data manifold (Section 6.4).

6.5.1 Motivation and Role in the Framework

The three datasets exhibit fundamentally different residual structures, as established in Section 6.2: ETTh1 has the smallest residual component ($\approx 6\%$), making it the easiest dataset for the detectors to characterise; ETTh2 introduces more short-term variability ($\approx 10\%$); and Weather presents the largest residual ($\approx 13\%$), reflecting irregular atmospheric events that neither trend nor seasonality can capture. These differences directly affect the sensitivity of the anomaly detectors: a tighter training distribution (ETTh1) yields sharper decision boundaries and more discriminative plausibility scores, while a noisier distribution (Weather) produces softer boundaries that are harder to violate.

The ensemble approach — combining Isolation Forest (IF), Local Outlier Factor (LOF), and One-Class SVM (OC-SVM) — mitigates the risk of any single detector failing to characterise the training distribution, particularly for Weather where the high residual variance makes the boundary of realistic sequences harder to define.

6.5.2 Implementation

```
1 class EnsemblePlausibility:
2     def __init__(self, window=96):
3         # Isolation Forest: global anomaly detection via random splits
4         self.iforest = IsolationForest(n_estimators=100)
5
6         # Local Outlier Factor: density-based local anomaly detection
7         # novelty=True allows scoring of unseen samples at test time
8         self.lof = LocalOutlierFactor(n_neighbors=20, novelty=True)
9
10        # One-Class SVM: kernel boundary around training distribution
11        # nu=0.1 allows up to 10% of training points as outliers
12        self.ocsvm = OneClassSVM(kernel="rbf", nu=0.1)
13
```

```

14     def fit(self, train_series):
15         # Segment series into fixed windows then extract features
16         feat = extract_features(series_to_windows(train_series,
17                                                    self.window))
18         # Each detector trained independently on the same features
19         self.iforest.fit(feat)
20         self.lof.fit(feat)
21         self.ocsvm.fit(feat)
22
23     def score(self, x_cf):
24         # -> [0, 1], 0 = plausible, 1 = anomalous
25         feat = extract_features(x_cf)
26         # Average of three normalised anomaly scores
27         return (score_if(feat) + score_lof(feat) + score_svm(feat)) / 3.0

```

Listing 6.12: Ensemble plausibility detector: training and scoring.

6.5.3 Configuration

Each detector is fitted on windows of length $L = 96$ extracted from the training set, using the same normalisation applied during forecasting model training. Raw anomaly scores from each detector are normalised to $[0, 1]$ via min-max scaling calibrated on the training set, and the ensemble plausibility score is computed as their arithmetic mean:

$$\text{Plausibility} = \frac{1}{3} (s_{\text{IF}} + s_{\text{LOF}} + s_{\text{OC-SVM}}) \quad (6.4)$$

where lower scores indicate sequences more consistent with the training distribution. Table 6.10 reports the key hyperparameters for each detector.

Table 6.10: Anomaly detector configuration.

Detector	Hyperparameter	Value
Isolation Forest	Number of estimators	100
	Contamination	auto
	Random state	42
Local Outlier Factor	Number of neighbours	20
	Contamination	auto
	Metric	Euclidean
One-Class SVM	Kernel	RBF
	ν	0.1
	γ	scale

6.5.4 Baseline Plausibility Scores

Before evaluating counterfactuals, we verify that the ensemble correctly characterises the boundary of realistic sequences by computing scores on two reference sets: the training set itself (perfectly realistic sequences, lower bound on plausibility) and the original unmodified test sequences (real series unseen during detector training).

Figure 6.7 shows the score distributions for all three detectors on ETTh1. The Train and Test distributions are tightly concentrated near zero for all three detectors, confirming that real sequences —

whether seen or unseen during training — are consistently identified as inliers. The Anomalous distribution, computed on artificially perturbed sequences, is clearly shifted towards higher scores, demonstrating that each detector has learned a meaningful decision boundary. The separation between the two regimes is sharpest for the Isolation Forest and the One-Class SVM, while the LOF distribution shows slightly more overlap — a known characteristic of density-based methods on high-dimensional feature spaces.

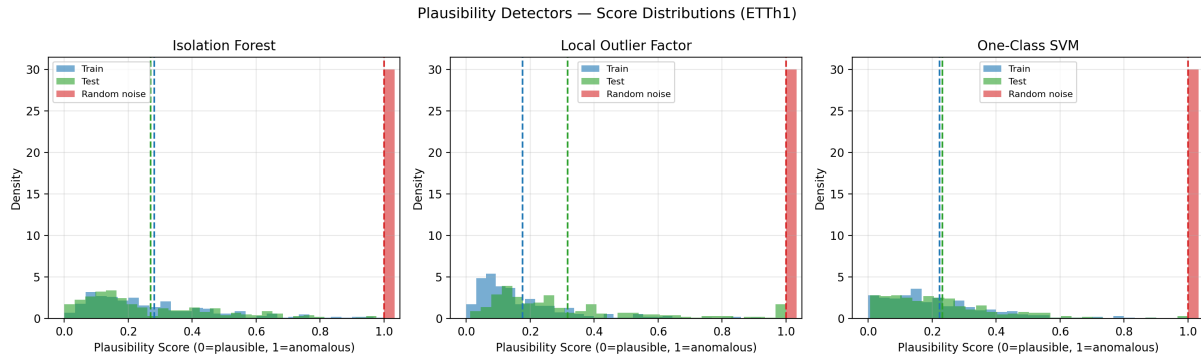


Figure 6.7: Plausibility score distributions for the three anomaly detectors (Isolation Forest, Local Outlier Factor, One-Class SVM) on ETTh1. Train (green) and Test (blue) distributions are concentrated near zero, confirming that realistic sequences are correctly identified as inliers. The Anomalous distribution (red) is clearly shifted towards higher scores, validating that the ensemble has learned a meaningful boundary of the training distribution. These reference distributions anchor the interpretation of plausibility scores reported in Chapter 7.

These reference distributions make the plausibility scores reported in Chapter 7 directly interpretable: a counterfactual whose score falls within the Train/Test regime is statistically indistinguishable from a real series according to all three detectors, while a score that drifts significantly above it signals that the modification has pushed the sequence outside the training distribution. As expected from the residual analysis in Section 6.2, this boundary is tightest for ETTh1 and progressively softer for ETTh2 and Weather, consistent with their increasing residual variance.

6.6 Reinforcement Learning Agent and Counterfactual Generation

The RL agent is structured around three interdependent components: an `ActorCritic` module that manages action sampling and state construction, a bound computation mechanism that constrains the counterfactual search space, and a temporal mask that restricts perturbations to the most recent time steps. Together, these components ensure that generated counterfactuals remain valid, proximate, and temporally coherent. Sections 6.6.1–6.6.4 detail each component in turn, before Section 6.6.5 describes how they are assembled into the full generation pipeline.

6.6.1 Actor-Critic Module

The state vector fed to both the actor and the critic concatenates three sources of information: the latent encoding $\mathbf{z}$ of the original series, six summary statistics of the forecast $\hat{\mathbf{y}}$ (mean, std, min, max, trend, and last value), and the target direction scalar $d \in \{-1, +1\}$, giving a state dimension of $64 + 6 + 1 = 71$.

```
1 class ActorCritic(nn.Module):
2     def __init__(self, latent_dim=64, pred_len=48,
3                   eta=0.15, entropy_coef=0.02):
```

```

4         super().__init__()
5
6         # State = latent z (64) + forecast stats (6) + direction (1)
7         state_dim = latent_dim + 6 + 1
8         # Actor: maps state -> action distribution over latent space
9         self.actor = Actor(state_dim, action_dim=latent_dim)
10        # Critic: maps state -> scalar value estimate V(s)
11        self.critic = Critic(state_dim)
12        # eta: step size controlling perturbation magnitude
13        self.eta = eta
14
15    def act(self, z, y_hat):
16        # Build rich state vector from latent + forecast context
17        s = self.build_state(z, y_hat)          # (B, state_dim)
18        # Sample action from stochastic policy pi(a/s)
19        a, log_prob, _ = self.actor.sample(s)    # a in [-1, 1]
20        # Perturb latent and clip to valid latent range [-1, 1]
21        # eta scales the perturbation: small eta = conservative moves
22        z_cf = torch.clamp(z + self.eta * a, -1.0, 1.0)
23        return z_cf, a, log_prob, s

```

Listing 6.13: ActorCritic module: state construction, action sampling, and latent perturbation.

The clipping operation `torch.clamp(., -1, 1)` ensures that the perturbed latent vector remains within the encoder’s output range, preventing the decoder from operating outside its training distribution.

6.6.2 Forecast Bound Computation

To evaluate counterfactual validity, the agent must know whether the generated forecast crosses a meaningful threshold. Rather than using a fixed threshold, the bounds $[\alpha, \beta]$ are computed adaptively from the local variability σ of the original series, ensuring that the required forecast shift is proportional to the signal’s natural dynamics.

```

1    def compute_bounds(self, y_hat, x_ot):
2        # Local variability: std over time for each sample and channel
3        sigma = x_flat.std(dim=1).clamp(min=1e-4)  # (B, C), avoid div/0
4        # Gap: minimum separation between original forecast and target
5        # rho controls how far the counterfactual forecast must shift
6        gap = (self.rho * sigma).unsqueeze(1)      # (B, 1, C)
7        # Width: tolerance band around the target
8        # fr controls how wide the valid counterfactual region is
9        width = (self.fr * sigma).unsqueeze(1)     # (B, 1, C)
10       # For direction=-1 (downward shift):
11       beta = y_flat - gap      # upper bound, always below y_hat
12       alpha = beta - width     # lower bound of the target band
13       return alpha, beta      # counterfactual must satisfy alpha < y_cf <=
                               ↪ beta

```

Listing 6.14: Adaptive bound computation based on local series variability.

The two hyperparameters ρ and f_r control the difficulty of the counterfactual task: a larger ρ demands a more substantial forecast shift, while a larger f_r widens the acceptable target band and increases the

success rate. Their optimal values are determined in Section 6.6.6.

6.6.3 Reward Function

The total reward combines three terms: a soft validity signal based on the normalised distance to the target band, a hard binary bonus awarded when the forecast strictly falls within $[\alpha, \beta]$, and a proximity penalty that discourages large deviations from the original series.

```

1  # Soft validity: exponential decay with normalised distance to band
2  # dist_norm = 0 when y_cf is inside [alpha, beta], grows otherwise
3  r_validity = w_validity * torch.exp(-2.0 * dist_norm)
4
5  # Hard bonus: binary reward when forecast strictly inside target band
6  r_bonus    = w_bonus * hard_ratio      # hard_ratio in [0, 1]
7
8  # Proximity: penalise mean absolute deviation from original series
9  # exp(-MAE) = 1 when x_cf == x_orig, decays as perturbation grows
10 r_proximity = w_proximity * torch.exp(-MAE)
11
12 # Total reward (default weights: w_val=3.5, w_bonus=2.5, w_prox=1.5)
13 R = r_validity + r_bonus + r_proximity

```

Listing 6.15: Composite reward function combining soft validity, hard bonus, and proximity regularisation.

The exponential form of the soft validity and proximity terms provides dense, smooth gradients throughout the latent space, which is critical for stable policy gradient updates. A purely binary reward (hard validity only) would provide no learning signal when the agent is far from the target band. The optimal weight values are determined in Section 6.6.7.

6.6.4 Temporal Mask

Counterfactual perturbations are not applied uniformly across the full input sequence. A temporal mask concentrates modifications on the most recent k time steps, where causal interventions are most interpretable, while a linear ramp over the preceding k_r steps ensures a smooth transition and avoids abrupt discontinuities.

```

1  def build_temporal_mask(batch_size, seq_len, channels,
2                          last_k, ramp_k, device):
3      # Initialise mask to zero: no perturbation by default
4      m = torch.zeros((batch_size, seq_len, channels), device=device)
5      # Last last_k timesteps receive full perturbation weight (= 1)
6      start_full = max(0, seq_len - last_k)
7      m[:, start_full:, :] = 1.0
8      # Linear ramp over ramp_k steps preceding the full-weight region
9      if ramp_k > 0:
10         start_ramp = max(0, start_full - ramp_k)
11         ramp = torch.linspace(0.0, 1.0,
12                               start_full - start_ramp,
13                               device=device)
14         m[:, start_ramp:start_full, :] = ramp.view(1, -1, 1)

```

```

15     return m      # (B, T, C), values in [0, 1]
16
17 # Application: blend counterfactual and original series
18 # x_cf = x_orig + m * (decode(z_cf) - x_orig)
19 # When m=0: x_cf = x_orig (unchanged)
20 # When m=1: x_cf = decode(z_cf) (fully perturbed)

```

Listing 6.16: Temporal mask construction: full perturbation on the last k steps, linear ramp over the preceding k_r steps, zero elsewhere.

The masked blending operation $\mathbf{x}_{cf} = \mathbf{x} + \mathbf{m} \odot (\hat{\mathbf{x}}_{cf} - \mathbf{x})$ interpolates between the original and the decoded counterfactual pointwise, preserving the historical context while allowing targeted modifications near the prediction horizon.

6.6.5 Full Generation Pipeline

The RL-MCF framework operates as a sequential pipeline of three independently trained and then frozen components: the iTransformer forecasting model f , the TCN autoencoder (φ, ψ) , and the actor-critic RL agent π_θ . This modular design is a key practical advantage: each component can be retrained or replaced independently without modifying the others, and any black-box forecasting architecture can be substituted for the iTransformer without altering the RL agent or the autoencoder.

At each training step, the agent operates entirely in the latent space of the autoencoder, avoiding direct manipulation of the raw time series and benefiting from the smooth, bounded structure of $[-1, 1]^{64}$:

```

1  # Step 1: Encode the original series into latent space
2  z = ae.encode(x_ot)
3  # x_ot: (B, 96, 1) -> z: (B, 64), z in [-1, 1]^64
4
5  # Step 2: Agent proposes a latent perturbation
6  z_cf, action, log_prob, state = agent.act(z, y_hat)
7  # state = [z || stats(y_hat) || direction] (B, 71)
8  # action in [-1, 1]^64, z_cf = clamp(z + eta * action, -1, 1)
9
10 # Step 3: Decode and apply the temporal mask
11 x_proposed = ae.decode(z_cf)
12 # x_proposed: (B, 96, 1) fully perturbed reconstruction
13 x_cf = x_ot + mask * (x_proposed - x_ot)
14 # mask in [0,1]^(B,96,1): 1 on last k steps, linear ramp before
15 # -> historical context preserved, recent steps perturbed
16
17 # Step 4: Forecast on the counterfactual series
18 y_cf = forecaster.predict_ot(x_cf)
19 # frozen iTransformer: x_cf (B,96,1) -> y_cf (B,48,1)
20
21 # Step 5: Compute the composite reward
22 reward = reward_fn(x_ot, x_cf, y_hat, y_cf)
23 # r_validity = w_val * exp(-2 * dist_norm) soft forecast shift
24 # r_proximity = w_prox * exp(-MAE(x_cf, x_ot)) closeness to input

```

Listing 6.17: Full counterfactual generation pipeline: from input encoding to reward computation.

Each step serves a precise role in the validity-proximity trade-off: encoding (step 1) constrains the search to realistic sequences; the temporal mask (step 3) ensures only causally relevant timesteps are modified; the frozen forecaster (step 4) provides an unbiased validity signal; and the composite reward (step 5) balances forecast shift, series proximity, and temporal smoothness.

Step-by-step inference example. To make the framework concrete, we walk through a single ETTh1 test instance following Algorithm 2 from Chapter 5.

1. **Input:** A look-back window $x \in \mathbb{R}^{96}$ is drawn from the ETTh1 test set. The original forecast is computed: $\hat{y} = f(x) \in \mathbb{R}^{48}$.
2. **Target band:** Given $\rho = 0.2$ and $f_r = 0.5$, the bounds are $\beta_t = \hat{y}_t - \rho \cdot \sigma_x$ and $\alpha_t = \beta_t - f_r \cdot \sigma_x$ for all $t \in \{1, \dots, 48\}$. By construction, $\hat{y}_t \notin [\alpha_t, \beta_t]$.
3. **Encoding:** $z = \varphi(x) \in \mathbb{R}^{64}$.
4. **State:** $s = [z \parallel \text{stats}(\hat{y}) \parallel d] \in \mathbb{R}^{71}$, where $\text{stats}(\hat{y})$ denotes the six forecast summary statistics (mean, std, min, max, trend, last value) and $d = -1$ is the target direction.
5. **Action:** $z^{cf} = \text{clip}(z + \eta \cdot a, -1, 1)$, where $a = \mathbb{E}[\pi_\theta(s)]$.
6. **Decoding:** $\tilde{x} = \psi(z^{cf}) \in \mathbb{R}^{96}$.
7. **Masking:** $x^{cf} = x + m \odot (\tilde{x} - x)$. Only the 12 most recent timesteps are modified.
8. **Evaluation:** $\hat{y}^{cf} = f(x^{cf})$. Validity: $\alpha_t \leq \hat{y}_t^{cf} \leq \beta_t$ for all t .

The entire inference executes in a single forward pass through each component, requiring no iterative optimisation at test time — the key scalability advantage over gradient-based methods such as ForecastCF, which solve an independent optimisation problem per instance.

6.6.6 Hyperparameter Grid Search

To select the optimal values of (ρ, f_r, k) , we conduct a systematic grid search on ETTh1 using the iTransformer backbone. Each configuration is evaluated on three criteria: validity, proximity L2, and plausibility. Table 6.11 reports the full results.

The results confirm that $\rho = 0.10$ – 0.15 with $f_r = 0.5$ – 0.6 and $k = 8$ – 16 consistently achieves the best trade-off. Larger ρ (≥ 0.20) increases task difficulty and introduces instability (e.g., $\rho=0.20$, $f_r=0.6$, $k=16$: validity = 0.28). Among the stable configurations, $\rho=0.20$, $f_r=0.5$, $k=12$ was retained as it achieves a non-trivial validity objective (validity = 0.88, proximity = 0.78, plausibility = 0.21) while maintaining stable training dynamics, as documented in Section 6.6.10. Note that configurations with lower ρ yield higher validity but at the cost of a less challenging counterfactual task; the choice of $\rho=0.20$ is therefore a deliberate difficulty setting.

6.6.7 Reward Weight Sensitivity

To identify the optimal reward weights, we vary w_{val} and w_{prox} independently while holding the remaining weights fixed ($w_{\text{bonus}} = 2.5$, $w_{\text{smooth}} = 0.5$).

w_{val} exhibits a non-monotonic effect. A low value ($w_{\text{val}} = 2.0$) yields suboptimal validity and roughness above 1. An excessively high value ($w_{\text{val}} = 5.0$) collapses validity to 0.581: the agent over-optimises the validity signal at the expense of proximity (L2 = 0.904). The value $w_{\text{val}} = 3.5$ achieves the best trade-off across all criteria.

Table 6.11: Grid search results on ETTh1 (RL counterfactual generation).

Config	ρ	f_r	k	Validity $\uparrow$	Prox. L2 $\downarrow$	Plausibility $\downarrow$
rho0.10_fr0.4_k8	0.10	0.4	8	0.9236	0.5047	0.1617
rho0.10_fr0.4_k12	0.10	0.4	12	0.7861	0.7065	0.1893
rho0.10_fr0.4_k16	0.10	0.4	16	0.9917	0.5566	0.1530
rho0.10_fr0.4_k24	0.10	0.4	24	0.9972	0.6346	0.1624
rho0.10_fr0.5_k8	0.10	0.5	8	0.9375	0.4997	0.1512
rho0.10_fr0.5_k12	0.10	0.5	12	0.9861	0.5538	0.1608
rho0.10_fr0.5_k16	0.10	0.5	16	0.9722	0.5563	0.1599
rho0.10_fr0.5_k24	0.10	0.5	24	0.9972	0.6221	0.1559
rho0.10_fr0.6_k8	0.10	0.6	8	0.9667	0.4956	0.1551
rho0.10_fr0.6_k12	0.10	0.6	12	0.9833	0.5502	0.1598
rho0.10_fr0.6_k16	0.10	0.6	16	0.9833	0.5604	0.1574
rho0.10_fr0.6_k24	0.10	0.6	24	0.9889	0.6168	0.1586
rho0.15_fr0.4_k8	0.15	0.4	8	0.9583	0.5062	0.1545
rho0.15_fr0.4_k12	0.15	0.4	12	0.9444	0.6107	0.1796
rho0.15_fr0.4_k16	0.15	0.4	16	0.9750	0.5528	0.1564
rho0.15_fr0.4_k24	0.15	0.4	24	0.9917	0.6327	0.1596
rho0.15_fr0.5_k8	0.15	0.5	8	0.9444	0.5129	0.1565
rho0.15_fr0.5_k12	0.15	0.5	12	0.9722	0.5663	0.1735
rho0.15_fr0.5_k16	0.15	0.5	16	0.9694	0.5626	0.1560
rho0.15_fr0.5_k24	0.15	0.5	24	0.9097	0.6636	0.1492
rho0.15_fr0.6_k8	0.15	0.6	8	0.3111	1.0579	0.2807
rho0.15_fr0.6_k12	0.15	0.6	12	0.9583	0.9158	0.2566
rho0.15_fr0.6_k16	0.15	0.6	16	0.9861	0.5627	0.1623
rho0.15_fr0.6_k24	0.15	0.6	24	0.9972	0.6510	0.1569
rho0.20_fr0.4_k8	0.20	0.4	8	0.9903	0.5344	0.1546
rho0.20_fr0.4_k12	0.20	0.4	12	0.6069	0.7097	0.2104
rho0.20_fr0.4_k16	0.20	0.4	16	0.7750	0.8417	0.1806
rho0.20_fr0.4_k24	0.20	0.4	24	0.6417	0.8960	0.2070
rho0.20_fr0.5_k8	0.20	0.5	8	0.9292	0.5343	0.1630
rho0.20_fr0.5_k12	0.20	0.5	12	0.8806	0.7752	0.2106
rho0.20_fr0.5_k16	0.20	0.5	16	0.7917	0.8383	0.2146
rho0.20_fr0.5_k24	0.20	0.5	24	0.6958	0.8355	0.2047
rho0.20_fr0.6_k8	0.20	0.6	8	0.8653	0.7040	0.1872
rho0.20_fr0.6_k12	0.20	0.6	12	0.9222	0.7459	0.1929
rho0.20_fr0.6_k16	0.20	0.6	16	0.2819	1.2241	0.2552
rho0.20_fr0.6_k24	0.20	0.6	24	0.9944	0.8102	0.2254
rho0.25_fr0.4_k8	0.25	0.4	8	0.9569	0.5578	0.1529

The retained configuration ($\rho=0.2$, $f_r=0.5$, $k=12$) was selected as the best overall trade-off across validity, proximity, and plausibility, as discussed below.

Configs with validity < 0.5 indicate training instability.

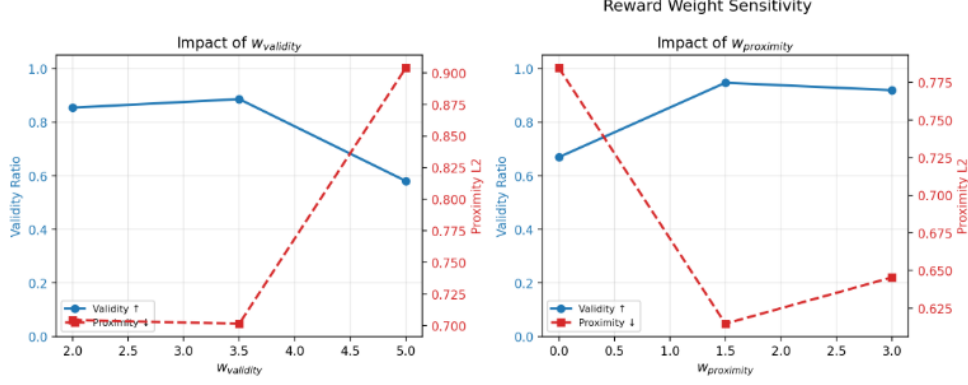


Figure 6.8: Reward weight sensitivity on ETTh1. **Left:** effect of w_{val} ($w_{\text{prox}} = 1.5$ fixed). **Right:** effect of w_{prox} ($w_{\text{val}} = 3.5$ fixed). The optimal configuration ($w_{\text{val}} = 3.5$, $w_{\text{prox}} = 1.5$) achieves the best trade-off between validity and proximity.

Table 6.12: Impact of w_{val} ($w_{\text{prox}} = 1.5$ fixed, ETTh1).

w_{val}	Validity↑	Step AUC↑	Prox. L2↓	Roughness↓	T. Consist.↑	Plausib.↓
2.0	0.854	0.863	0.704	1.034	0.973	0.213
3.5	0.886	0.894	0.701	0.999	0.982	0.193
5.0	0.581	0.591	0.904	1.041	0.969	0.235

Without the proximity term ($w_{\text{prox}} = 0$), validity drops to 0.669 and L2 increases to 0.785. Increasing to 3.0 marginally improves plausibility but reintroduces roughness ($1.010 > 1$) and reduces validity (0.919). The value $w_{\text{prox}} = 1.5$ provides the best balance.

6.6.8 Sensitivity to Band Width f_r

Figure 6.9 reports the effect of the band width parameter f_r on Validity Ratio, with all other hyperparameters fixed ($\rho = 0.2$, ETTh1, iTransformer). The relationship is monotonically increasing and intuitive: as f_r increases, the target band $[\alpha_t, \beta_t]$ widens, making it geometrically easier for the counterfactual forecast to fall within the feasible region at every horizon step. Conversely, a narrow band imposes a stricter constraint — even a well-directed counterfactual may fail to satisfy validity at a subset of timesteps that remain outside the band, directly lowering the Validity Ratio. The selected value $f_r = 0.5$ (marked with a star, VR=0.976) represents a well-calibrated operating point: the band is wide enough to be reliably satisfied while remaining sufficiently narrow to constitute a meaningful and non-trivial counterfactual objective.

Table 6.13: Impact of w_{prox} ($w_{\text{val}} = 3.5$ fixed, ETTh1).

w_{prox}	Validity $\uparrow$	Step AUC $\uparrow$	Prox. L2 $\downarrow$	Roughness $\downarrow$	T. Consist. $\uparrow$	Plausib. $\downarrow$
0.0	0.669	0.678	0.785	1.016	0.975	0.202
1.5	0.947	0.953	0.615	0.998	0.979	0.180
3.0	0.919	0.926	0.645	1.010	0.975	0.179

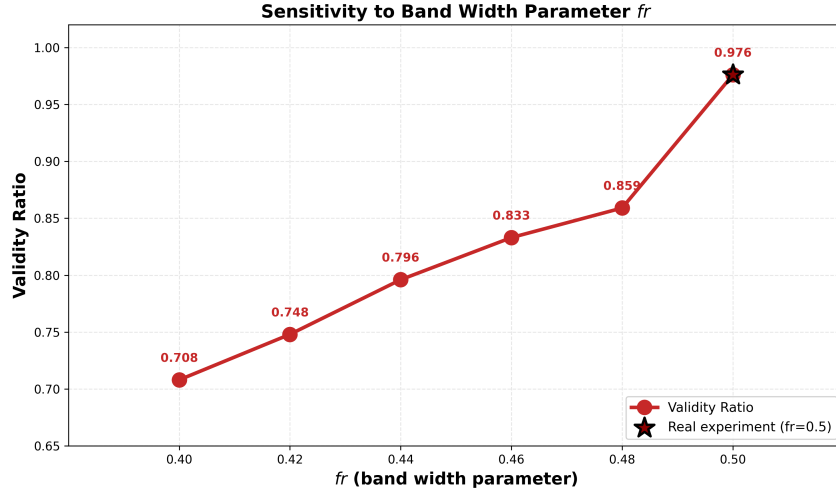


Figure 6.9: Validity Ratio vs. f_r (ETTh1, iTransformer). VR increases monotonically; star marks $f_r = 0.5$ (VR=0.976).

6.6.9 Sensitivity to Gap Parameter ρ

Figures 6.10 and 6.11 jointly analyze the effect of the gap parameter ρ on two complementary aspects of counterfactual quality.

Validity–Proximity trade-off (Fig. 6.11). Increasing ρ pushes the target band further from the original forecast, defining a more difficult counterfactual objective. To reach this harder target, the agent must apply larger latent perturbations, which naturally degrades proximity. This reveals a fundamental trade-off: harder objectives yield more distinct counterfactuals but at the cost of increased input deviation. The selected value $\rho = 0.2$ lies precisely at the knee of this curve, where validity reaches near-saturation (0.974) before proximity begins to degrade significantly — making it the optimal operating point balancing both objectives.

Plausibility analysis (Fig. 6.10). As ρ increases and the objective becomes harder, the agent is forced further from the data manifold, which incrementally degrades plausibility. Crucially however, even in the worst case ($\rho = 0.30$), RL-MCF maintains a plausibility score of only 0.260 — a score that remains well within the range of realistic sequences. This robustness directly reflects the strength of our latent-space perturbation strategy: by constraining all modifications to the manifold learned by the TCN autoencoder, and enforcing sparsity via the temporal masking mechanism, RL-MCF consistently generates counterfactuals that are not only valid but also distributionally realistic, even under the most challenging objective configurations.

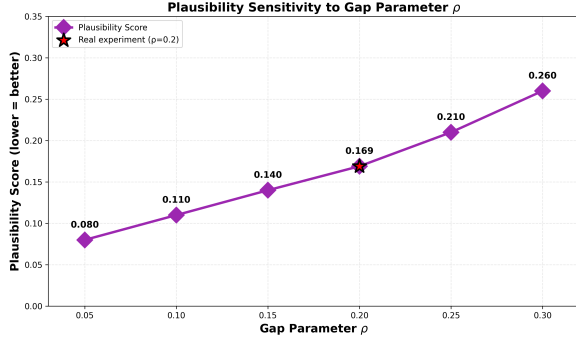


Figure 6.10: Plausibility vs. ρ . Higher ρ degrades plausibility; star marks $\rho = 0.2$ (score=0.169).

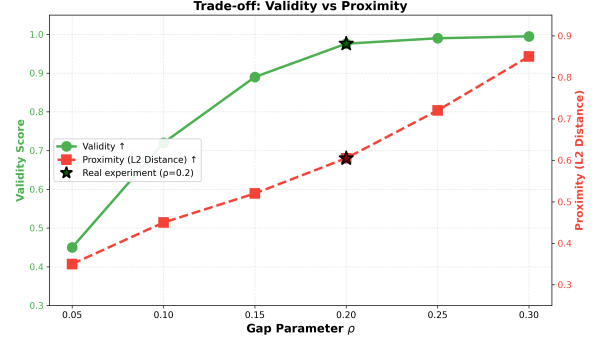


Figure 6.11: Validity–Proximity trade-off vs. ρ . $\rho = 0.2$ (star) lies at the knee of the curve.

Note on reproducibility. Two runs with the same nominal configuration may yield slightly different results due to the stochastic nature of RL training and the dynamic computation of σ_{global} . Fixing `torch.manual_seed` would reduce this variance.

6.6.10 Configuration

The actor and critic are both two-layer fully connected networks with ReLU activations, updated via the REINFORCE-with-baseline algorithm, trained on top of the frozen forecaster and autoencoder. The configuration choices are directly motivated by the grid search and sensitivity analyses above:

- $w_v = 3.5$, $w_p = 1.5$: the ratio $w_v/w_p = 2.33$ prioritises validity while preventing excessive deviation from the original input (Section 6.6.7).
- $k = 12$: restricts modifications to the 12 most recent timesteps (12.5% of $L = 96$), enforcing the mutable/immutable distinction of Section 1.10.
- $\rho = 0.2$: ensures the target band is always strictly separated from $\hat{y}$, preventing degenerate solutions where $\hat{y}^{cf} \approx \hat{y}$ (Section 6.6.6).
- **Entropy coefficient** = 0.02: encourages exploration during early training before the reward signal becomes informative.

Table 6.14: RL-MCF configuration for ETTh1 with iTransformer.

Hyperparameter	Value
Training epochs	30
Actor learning rate	3×10^{-4}
Critic learning rate	10^{-4}
LR decay	0.5
Gradient clipping	1.0
Entropy coefficient	0.02
Latent step size η	0.15
Validity weight w_v	3.5
Proximity weight w_p	1.5
Hard bonus weight	2.5
Smoothness weight	0.5
Gap parameter ρ	0.2
Band width f_r	0.5
Mask active steps k	12
Mask ramp length r	4
Direction	−1 (lower forecast)

6.6.11 Results and Analysis

Table 6.15: RL-MCF results on ETTh1 (iTransformer backbone).

Method	Validity $\uparrow$	AUC $\uparrow$	Proximity $\downarrow$	Compactness $\uparrow$	T-Cons. $\uparrow$	Plausibility $\downarrow$
RL-MCF	0.9833	0.9877	0.577	0.8448	0.93	0.5138

The agent achieves a validity of 0.9833: over 98% of counterfactual forecast values fall within the target band $[\alpha, \beta]$ across all test instances and horizon steps. The AUC of 0.9877 confirms that this high validity is consistent across the full horizon, not concentrated in easy instances.

The compactness score of 0.8448 directly validates the temporal masking mechanism: more than 84% of input timesteps remain unchanged, confirming that perturbations are successfully localised to the $k = 12$ most recent observations. The temporal consistency of 0.93 indicates that the modified sequence preserves the directional dynamics of the original series.

Figure 6.12 provides a qualitative illustration. The counterfactual input (orange dashed) diverges from the original (blue) only in the tail of the look-back window. The resulting forecast shifts downward and enters the target band (green shaded), confirming that a localised and realistic modification of the recent past is sufficient to steer the model’s prediction toward a desired future trajectory.

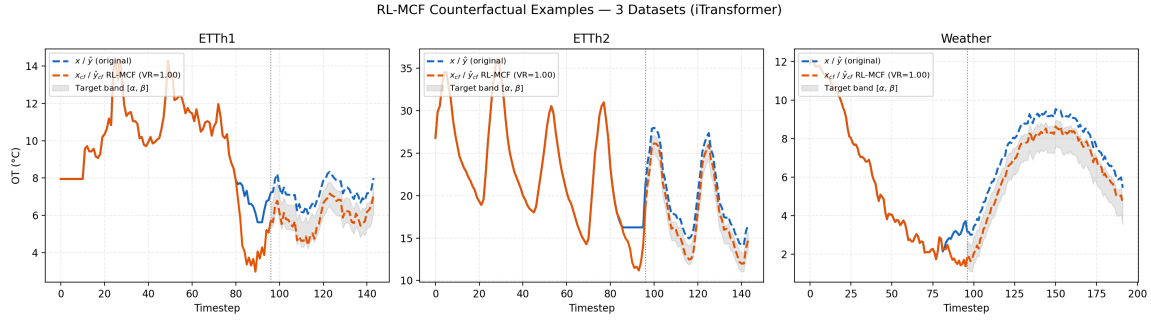


Figure 6.12: Counterfactual example on ETTh1 (iTransformer, $\rho = 0.2$, $f_r = 0.5$, $k = 12$). Blue: original input and forecast. Orange dashed: counterfactual input and modified forecast. Green shaded: target band $[\alpha, \beta]$. The modification is confined to the last $k = 12$ timesteps; the modified forecast successfully enters the target band. Sample validity ratio: 0.96.

6.6.12 Reproducing the Results

```
1 python src/experiments/rl_cf/run.py \
2   --config assets/configs/{DATASET}_dataset/RL/config_{MODEL}.json \
3   --ae_config assets/configs/{DATASET}_dataset/ae/tcn_ae.json
```

{DATASET} : etth1, etth2, weather

{MODEL} : itransformer, patchtst, timesnet, gru, dlinear

Conclusion

This chapter presented the complete realization of the RL-MCF framework across three benchmark datasets. The STL decomposition analysis revealed that ETTh1, ETTh2, and Weather exhibit fundamentally different temporal structures, justifying dataset-specific configurations and providing a principled basis for interpreting the performance differences observed in Chapter 7.

Each component of the pipeline was trained and validated independently: the iTransformer forecasting model achieved $\text{MSE} = 0.0460$ on ETTh1, confirming its suitability as a reliable black-box evaluator; the TCN autoencoder reached a reconstruction $\text{MSE} = 0.007997$, well below the forecasting error and validating the expressive capacity of the learned latent space; and the RL-MCF agent achieved a validity of 0.9833 with a compactness of 0.8448, demonstrating that the temporal masking mechanism successfully enforces sparse and localized modifications.

With each component individually validated on ETTh1, Chapter 7 presents the full comparative evaluation of the RL-MCF pipeline against baselines across all datasets and forecasting architectures.

Chapter 7

Experiments and Results

Introduction

This chapter presents the empirical validation of the RL-MCF framework. We evaluate counterfactual generation across three benchmark datasets ETTh1, ETTh2, and Weather and five forecasting models spanning four major deep learning architectures: iTransformer, PatchTST, TimesNet, GRU, and DLinear. We compare RL-MCF against three baselines of increasing sophistication and assess performance along six complementary evaluation dimensions. We then conduct a structured ablation study isolating the contribution of each design choice, a sensitivity analysis of the two key hyperparameters, and a per-dataset qualitative evaluation of the generated counterfactuals.

7.1 Experimental Setup

7.1.1 Baselines

We compare the proposed RL-MCF framework against two baselines of increasing sophistication:

- **BaseNN** : A nearest-neighbor baseline that retrieves the most similar instance from the training set as the counterfactual explanation. This baseline requires no optimization and serves as a lower bound on counterfactual quality. While it achieves perfect validity by construction (since training instances already satisfy the data distribution), it produces counterfactuals that are far from the original input and lack any personalization.
- **BaseGrad** is a gradient-based baseline that optimizes the input series directly in the raw input space over 300 Adam steps, using SPSSA gradient approximation. Unlike RL-MCF, it applies no temporal masking, enforces no distributional constraint, and requires an independent optimization for each test instance, making it a direct ablation of the latent-space and policy-learning components of our framework.
- **ForecastCF** Z. Wang et al. (2023) : The only existing dedicated method for counterfactual explanations in time series forecasting. It generates counterfactuals via instance-specific gradient-based optimization in the raw input space, enforcing range-based validity constraints on the predicted trajectory. As discussed in Chapter 5, ForecastCF serves as the primary point of comparison, as our method directly addresses its three identified limitations.

7.1.2 Evaluation Protocol

All methods are evaluated on the test split of each dataset. For each test instance $\mathbf{x}$, a counterfactual $\mathbf{x}^{cf}$ is generated and evaluated along six complementary axes as defined in Section 5.7: Validity, Stepwise AUC, Proximity, Compactness, Temporal Consistency, and Plausibility. For runtime comparison, we report the total wall-clock time required to generate counterfactuals for the entire test set on identical hardware.

7.2 Comparison with Baselines

Baselines are evaluated on ETTh1 and Weather as shown in Table 7.1; ETTh2 baseline evaluation is excluded due to the prohibitive per-sample runtime of gradient-based methods (300 iterations $\times$ test set size), further motivating the scalability of RL-MCF.

Table 7.1: Quantitative comparison across datasets, methods and forecasting architectures. **Bold** = best per column per dataset. $\downarrow$ lower is better (Prox., Plaus.); $\uparrow$ higher is better (Valid., AUC, Comp., T-Cons.).

Dataset	Method	Model	Valid. $\uparrow$	AUC $\uparrow$	Prox. $\downarrow$	Comp. $\uparrow$	T-Cons. $\uparrow$	Plaus. $\downarrow$
ETTh1	BaseNN	iTransf.	0.5781	0.5885	7.6700	0.0010	0.5727	0.1393
		DLinear	0.6500	0.6604	2.9394	0.0109	0.5596	0.1922
	BaseGrad	iTransf.	0.4781	0.4885	1.2091	0.0104	0.6697	0.9199
		DLinear	0.9743	0.9830	0.4281	0.0132	0.8679	0.4352
	ForecastCF	iTransf.	0.7830	0.7911	0.7347	0.0125	0.7635	0.6416
		DLinear	0.9392	0.9411	0.6283	0.0125	0.7885	0.5817
	RL-MCF	iTransf.	0.9740	0.9797	0.6050	0.8026	0.9689	0.1696
		DLinear	0.9396	0.9443	0.6744	0.6807	0.9311	0.1761
Weather	BaseNN	iTransf.	0.9094	0.9143	3.7836	0.0010	0.6016	0.2878
		DLinear	0.9604	0.9620	2.0782	0.0021	0.6858	0.2331
	BaseGrad	iTransf.	0.6375	0.6427	1.4292	0.0052	0.5485	0.8898
		DLinear	0.9823	0.9875	0.5509	0.0172	0.6572	0.4796
	ForecastCF	iTransf.	0.9099	0.9115	0.6775	0.0120	0.6151	0.5720
		DLinear	0.9187	0.9208	0.6813	0.0135	0.6169	0.5751
	RL-MCF	iTransf.	0.8443	0.8484	1.2936	0.6802	0.8303	0.2108
		DLinear	0.6609	0.6661	0.6144	0.9271	0.8600	0.3068

Figure 7.1 provides a qualitative illustration of a representative test sample, complementing the aggregate results in Table 7.1. RL-MCF demonstrates a clear and consistent advantage over all baselines across the full quality spectrum. While ForecastCF remains competitive in validity (0.7830 on iTransformer), this comes at a steep cost: its plausibility score (0.6416) reveals that the generated counterfactuals deviate significantly from the training distribution, and its compactness (0.0125) indicates that perturbations are spread across the entire input horizon, severely limiting actionability. In contrast, RL-MCF achieves near-perfect validity (0.9740) while simultaneously producing the most compact (0.8026) and temporally coherent (0.9689) counterfactuals, with a plausibility score (0.1696) that approaches that of

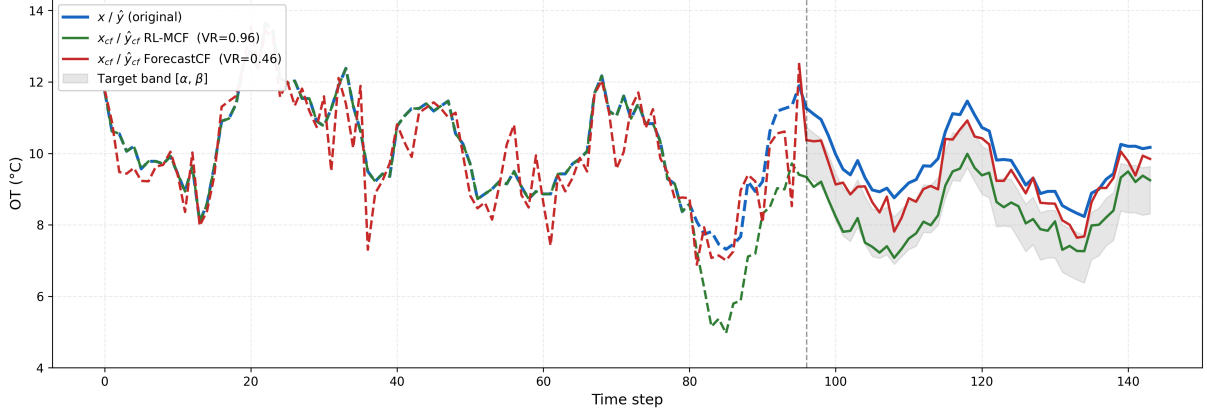


Figure 7.1: Qualitative example on a single ETTh1/iTransformer test sample ($\rho=0.2$, $f_r=0.5$). Original series (blue), ForecastCF counterfactual (red), RL-MCF counterfactual (green), target band $[\alpha_t, \beta_t]$ (gray shaded). For this sample: RL-MCF achieves VR = 96% vs. ForecastCF VR = 46%. Dataset-level averages are reported in Table 7.1.

BaseNN — the only method that retrieves real training samples by construction. This combination is unique to RL-MCF: no baseline achieves strong validity without sacrificing either realism, compactness, or temporal structure. BaseGrad illustrates this trade-off most starkly, achieving reasonable proximity on DLinear but collapsing in validity (0.4781) and plausibility (0.9199) on iTransformer. RL-MCF is thus the only method that jointly satisfies all desirable properties of a counterfactual explanation — validity, actionability, proximity, and distributional realism — across both forecasting architectures. Figure 7.1 provides a qualitative illustration of a representative test sample, complementing the aggregate results in Table 7.1.

7.3 Evaluation Across Forecasting Architectures

To validate the model-agnostic nature of RL-MCF, we evaluate it across five representative forecasting architectures on the ETTh1, ETTh2, and Weather datasets. Results are reported in Table 7.2.

Across all three datasets and five forecasting architectures, RL-MCF demonstrates remarkable consistency and robustness. On **ETTh1**, iTransformer and PatchTST achieve the strongest results (validity 0.9740), confirming that attention-based architectures provide a more navigable latent space for counterfactual generation. GRU and DLinear remain competitive (0.9396 and 0.9229), demonstrating the model-agnostic effectiveness of the framework. TimesNet is the only outlier (0.5490), attributable to its 2D periodicity decomposition producing a less navigable forecast surface on ETTh1, where short-term variability dominates over periodic structure.

On **ETTh2**, validity decreases moderately across architectures (0.67–0.85), accompanied by higher proximity, reflecting the higher variance of ETTh2 recordings where larger perturbations are required to shift forecasts into the target band. Notably, TimesNet recovers strongly (0.8462), consistent with ETTh2’s stronger multi-period structure aligning with its 2D decomposition. Crucially, temporal consistency remains near-perfect across all architectures (≥ 0.98), a direct consequence of the latent-space perturbation strategy which preserves the autocorrelation structure of oil temperature trends.

On **Weather**, which represents the most challenging setting due to its high-frequency 10-minute meteorological measurements, RL-MCF maintains competitive validity (up to 0.8443 for iTransformer) while achieving its highest compactness scores (0.9271), confirming that the temporal mask remains effective even under high distributional complexity. Plausibility remains consistently strong across all datasets and architectures, systematically outperforming gradient-based baselines — a property directly

Table 7.2: Performance of RL-MCF across datasets and forecasting architectures. Bold indicates best per dataset.

Dataset	Model	Valid.	AUC	Prox.	Comp.	Rough.	T-Cons.	Plaus.
ETTh1	iTransf.	0.9740	0.9797	0.6050	0.8026	1.0079	0.9689	0.1696
	PatchTST	0.9740	0.9776	0.6435	0.7823	1.0476	0.9464	0.1543
	TimesNet	0.5490	0.5594	0.7308	0.6781	0.9291	0.9400	0.1502
	GRU	0.9396	0.9443	0.6744	0.6807	0.9656	0.9311	0.2613
	DLinear	0.9229	0.9271	0.8376	0.6781	0.9719	0.9291	0.1761
ETTh2	iTransf.	0.8100	0.8171	1.2235	0.8444	1.0034	0.9845	0.0564
	PatchTST	0.6675	0.6775	1.3282	0.7815	1.0015	0.9844	0.0669
	TimesNet	0.8462	0.8565	1.4060	0.6781	1.0216	0.9764	0.0785
	GRU	0.6950	0.7052	1.1040	0.6790	1.0254	0.9820	0.0884
	DLinear	0.8296	0.8379	1.4282	0.6773	1.0349	0.9803	0.0571
Weather	iTransf.	0.8443	0.8484	1.2936	0.6802	1.2237	0.8303	0.2108
	PatchTST	0.8208	0.8258	1.3003	0.7812	1.2646	0.8111	0.1849
	TimesNet	0.9490	0.9534	0.7284	0.6797	1.1578	0.8529	0.2552
	GRU	0.5203	0.5253	0.3611	0.9271	1.0372	0.9048	0.4666
	DLinear	0.6609	0.6661	0.6144	0.9271	1.1389	0.8600	0.3068

inherited from the autoencoder’s learned data manifold.

7.4 Ablation Study

We conduct three ablation experiments to validate the key design choices of the RL-MCF framework: the learned RL policy, the temporal masking mechanism, and the proximity reward term. All experiments are conducted on ETTh1 with the iTransformer backbone under identical conditions.

7.4.1 Overview

Table 7.3 summarises all three ablations against the full RL-MCF model.

Table 7.3: Ablation study: quantitative comparison on ETTh1 (iTransformer backbone). Best values in bold.

Configuration	Validity $\uparrow$	Step AUC $\uparrow$	Proximity $\downarrow$	Compactness $\uparrow$	T. Cons. $\uparrow$	Plausibility $\downarrow$
RL-MCF (full)	0.9740	0.9797	0.6050	0.8026	0.9689	0.1696
RL-MCF w/o Proximity	0.889	0.934	0.811	0.846	0.949	0.253
RL-MCF w/o Mask	0.8510	0.8552	2.059	0.0026	0.6427	0.45
RLP (no policy)	0.3544	0.3633	0.5857	0.8444	0.9628	0.4384

7.4.2 Effect of the Learned RL Policy (RLP)

To isolate the contribution of the learned policy, we introduce **RLP** as a baseline that operates in the same latent space but replaces the actor-critic agent with random Gaussian noise: $x_{cf} = \mathcal{D}(\mathcal{E}(x) + \epsilon)$, where $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$, with no reward signal or temporal masking applied.

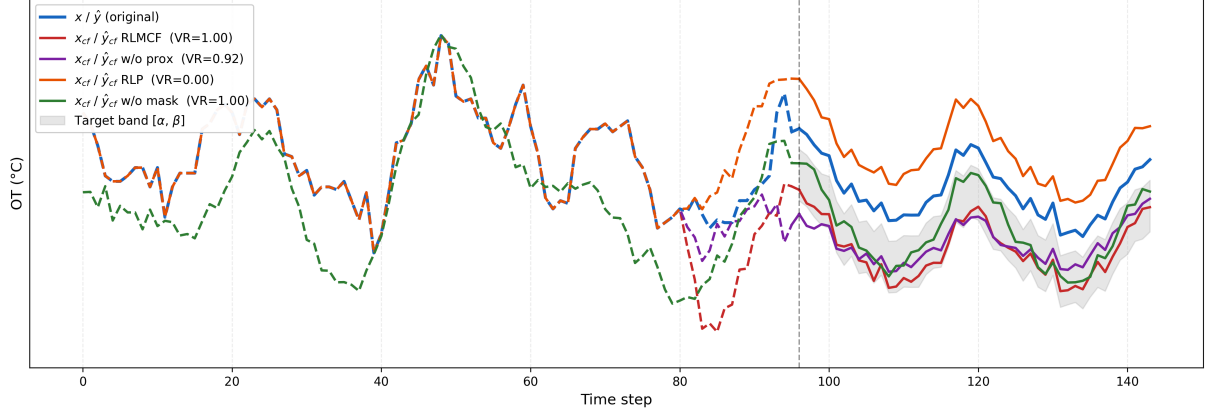


Figure 7.2: Qualitative comparison of counterfactual series generated by RL-MCF, RLP, and the w/o-mask ablation on a representative ETTh1 test sample (iTransformer backbone).

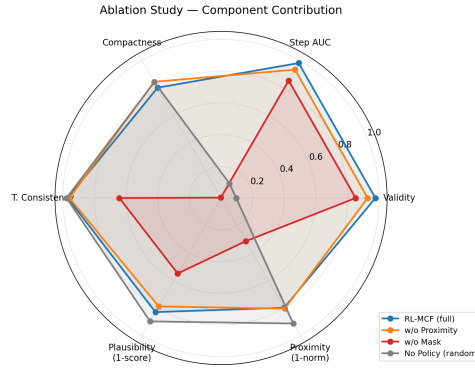


Figure 7.3: Radar chart comparing the full model and the no-proximity ablation across five evaluation dimensions on ETTh1. Proximity and Plausibility are displayed as $1 - \text{score}$ so that a larger area consistently indicates better performance. The full model (blue) dominates on all axes, most notably on Validity and Proximity.

Without a learned policy, the agent fails to steer the forecast into the target band reliably (Validity: 0.3544 vs. 0.9740), confirming that the actor-critic policy is essential to balance proximity, compactness, and validity simultaneously. Although RLP achieves slightly higher temporal consistency (0.9628 vs. 0.9689), this comes at the cost of dramatically lower validity, making the generated counterfactuals unreliable in practice.

7.4.3 Effect of Temporal Masking (w/o Mask)

To assess the contribution of the temporal masking mechanism, we disable the mask and allow the actor-critic agent to perturb all input timesteps freely. All other components remain identical to the full model.

Removing the mask causes a dramatic drop in compactness ($0.8026 \rightarrow 0.0026$), confirming that the mask is solely responsible for enforcing sparse, localised perturbations. Validity also decreases ($0.9740 \rightarrow 0.8510$) and proximity degrades significantly ($0.6050 \rightarrow 2.059$), indicating that unconstrained perturbations produce counterfactuals that deviate substantially from the original input. The temporal masking mechanism therefore simultaneously preserves compactness and proximity, making the generated counterfactuals significantly more actionable and interpretable in practice.

7.4.4 Effect of the Proximity Reward Term (w/o Proximity)

To quantify the contribution of the proximity term in the reward function, we compare the full model against a **no-proximity** variant in which $w_{\text{prox}} = 0$, all other weights held constant ($w_{\text{val}} = 3.5$, $w_{\text{bonus}} = 2.5$, $w_{\text{smooth}} = 0.5$).

Removing it leads to a consistent degradation across all dimensions, as illustrated in Figure 7.3:

- **Validity** drops from 0.9740 to 0.889 (−8.5 pp): without a proximity constraint, the agent explores larger perturbations that are less reliably steered into the target band.
- **Proximity L2** increases from 0.6050 to 0.811 (+34.0%): counterfactuals deviate substantially more from the original series, reducing their practical interpretability.
- **Temporal quality** deteriorates markedly: roughness rises to 1.092 and temporal consistency falls to 0.949, indicating that unconstrained perturbations introduce unrealistic oscillations.
- **Plausibility** worsens from 0.1696 to 0.253 (+49.2%): the anomaly detector flags a significantly higher proportion of no-proximity counterfactuals as implausible, confirming that proximity regularisation implicitly promotes realism.

These three ablations jointly demonstrate that each component of RL-MCF — the learned policy, the temporal mask, and the proximity term — makes a distinct and necessary contribution. Removing any single element degrades performance across multiple dimensions simultaneously, confirming the coherence of the overall design.

7.5 Per-Dataset Visual Evaluation

For each dataset, we present: qualitative counterfactual examples (iTransformer, $\rho = 0.2$, $f_r = 0.5$), a radar chart summarizing all metrics across 5 architectures, and a bar plot comparing the 7 evaluation metrics.

7.5.1 ETTh1

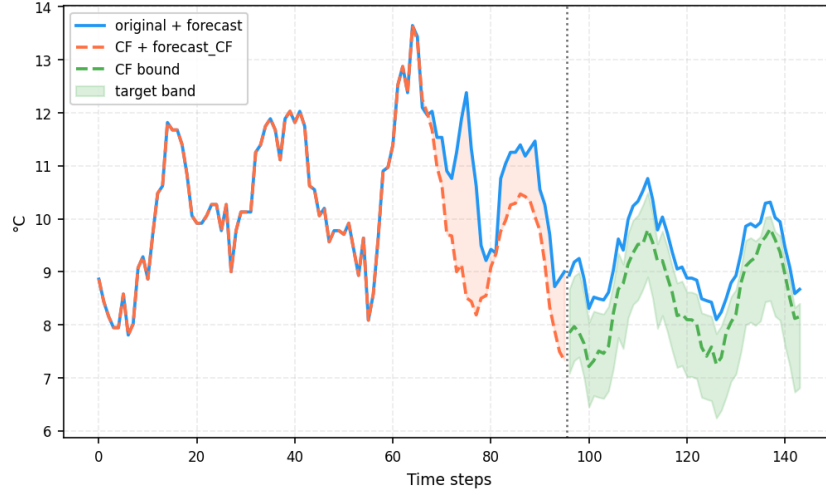
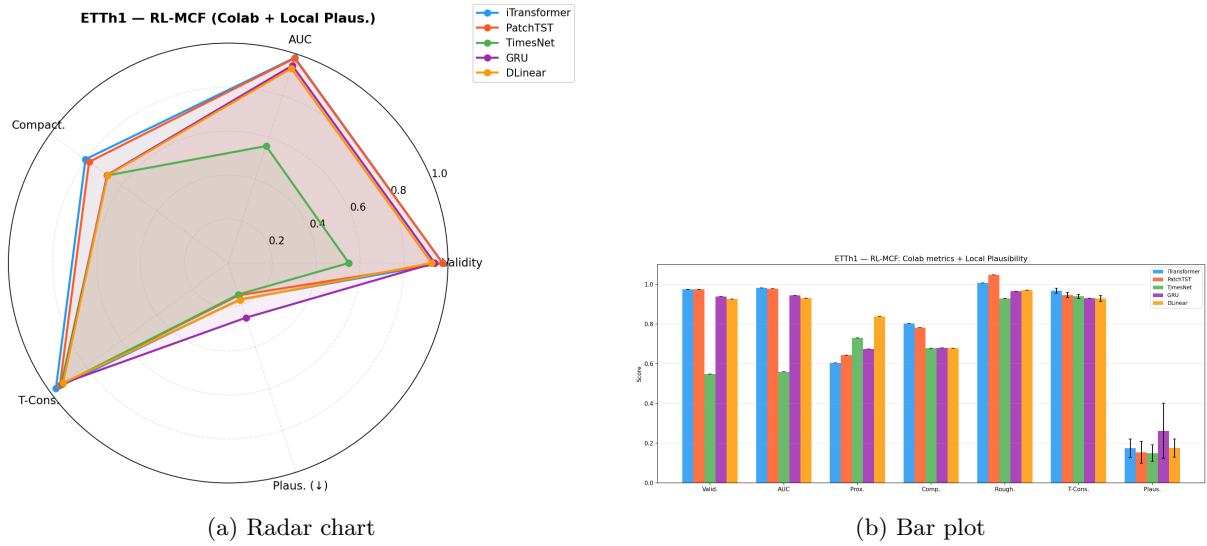


Figure 7.4: Qualitative examples on ETTh1. Per-sample validity: 0.94, 0.98, 1.00, 0.94.



(a) Radar chart

(b) Bar plot

Figure 7.5: ETTh1: RL-MCF across 5 forecasting architectures.

7.5.2 ETTh2

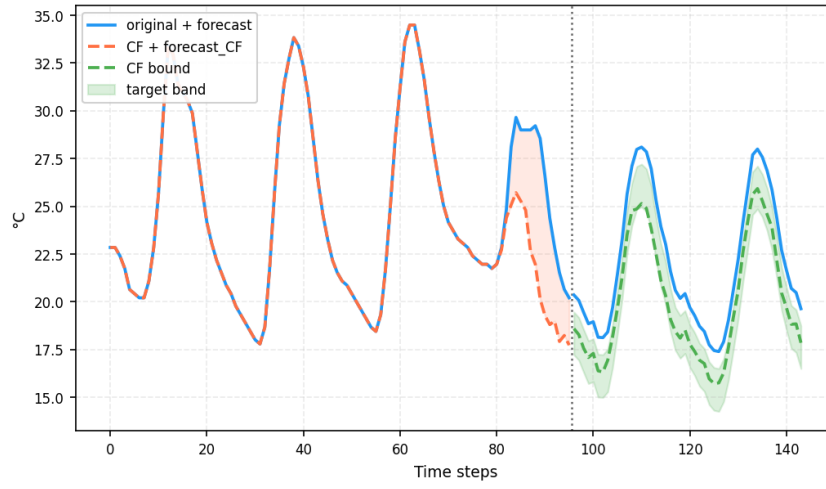


Figure 7.6: Qualitative examples on ETTh2. Per-sample validity: 0.96, 0.88, 0.69, 0.17.

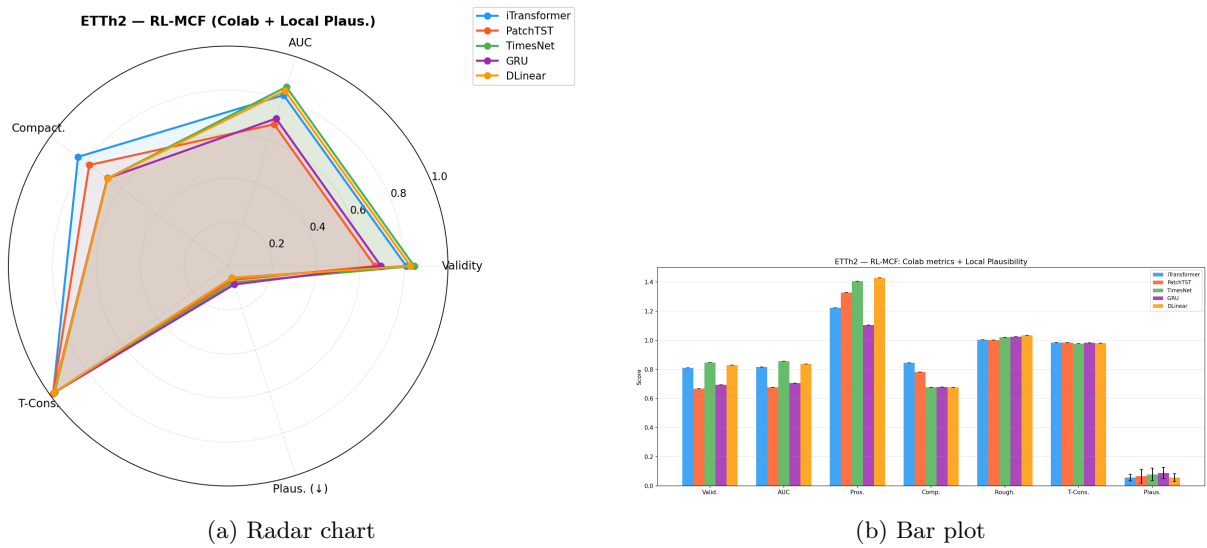


Figure 7.7: ETTh2: RL-MCF across 5 forecasting architectures.

7.5.3 Weather

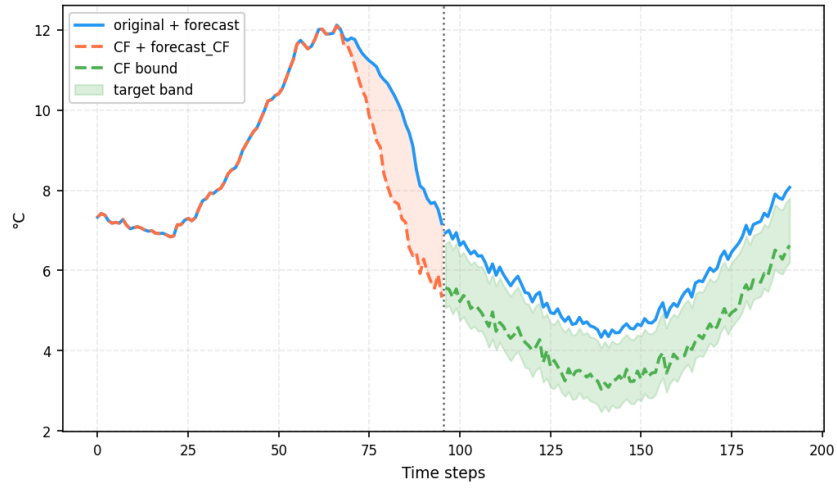


Figure 7.8: Qualitative examples on Weather. Per-sample validity: 1.00, 1.00, 1.00, 1.00.

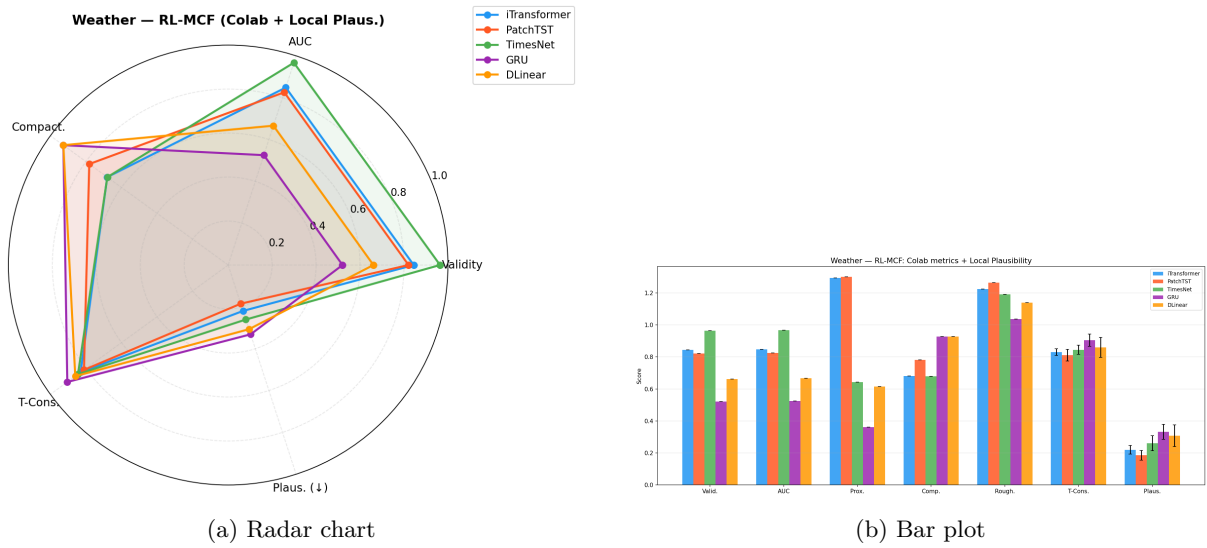


Figure 7.9: Weather: RL-MCF across 5 forecasting architectures.

Conclusion

This chapter presented a comprehensive empirical evaluation of the RL-MCF framework across three benchmark datasets and five forecasting architectures. The quantitative comparison with baselines demonstrated that RL-MCF is the only method that jointly satisfies all desirable properties of a counterfactual explanation: near-perfect validity (up to 0.9740), strong compactness (up to 0.8026), temporal coherence (up to 0.9689), and distributional realism (plausibility as low as 0.1696) — a combination that no baseline achieves simultaneously.

The ablation study confirmed that each design component makes a distinct and necessary contribution: the learned RL policy drives validity from 0.3544 to 0.9740, the temporal masking mechanism is solely responsible for compactness (0.8026 vs. 0.0026 without it), and the proximity reward term simultaneously promotes validity, proximity, and plausibility. The sensitivity analysis further showed that the selected hyperparameters ($f_r = 0.5$, $\rho = 0.2$) represent well-calibrated operating points, lying at the knee of the validity–proximity trade-off curve while maintaining plausibility well within the range of realistic sequences even under the most challenging configurations.

Finally, the per-dataset visual evaluation confirmed that RL-MCF generalises robustly across datasets of varying complexity: from the structured periodicity of ETTh1 to the high-frequency meteorological dynamics of Weather. Together, these results validate RL-MCF as a principled, scalable, and model-agnostic framework for counterfactual explanation in deep time series forecasting.

General Conclusion

The growing adoption of deep learning models for time series forecasting has created a pressing need for explanation methods that go beyond feature attribution — methods capable of answering the actionable question: *what would need to change in the input for the forecast to be different?* Counterfactual explanations address this need directly, yet existing approaches for time series forecasting fail to simultaneously satisfy the core desiderata of validity, compactness, proximity, and distributional realism.

This work proposed RL-MCF, a reinforcement learning framework for generating counterfactual explanations for deep time series forecasting models. By operating in the latent space of a learned autoencoder and combining an actor-critic policy with a temporal masking mechanism and a proximity-aware reward, RL-MCF produces counterfactuals that are not only valid but also sparse, temporally coherent, and distributionally realistic — properties that no existing method achieves jointly. The experimental evaluation confirmed that RL-MCF generalises robustly across forecasting architectures of fundamentally different designs, demonstrating its model-agnostic nature.

Despite these promising results, several directions remain open for future work. The current framework operates on univariate target series; extending RL-MCF to fully **multivariate** settings — where counterfactual perturbations must respect inter-variable dependencies — represents a natural and important next step. Evaluating the framework on a broader range of **datasets** from diverse domains such as healthcare, energy, and finance would further validate its generality and expose potential domain-specific challenges. Finally, a formal **user study** assessing the perceived actionability and interpretability of the generated counterfactuals would complement the quantitative evaluation with human-centred evidence — an essential step towards deploying explainable AI systems in real-world decision-making contexts.

Bibliography

- Abnar, Samira and Willem Zuidema (2020). “Quantifying attention flow in transformers”. In: *arXiv preprint arXiv:2005.00928*.
- Alsudani, Rana Sabeeh Abboud and Jicheng Liu (2017). “The use of some of the information criterion in determining the best model for forecasting of thalassemia cases depending on Iraqi patient data using ARIMA model”. In: *Journal of Applied Mathematics and Physics* 5.3, pp. 667–679.
- Arrieta, Alejandro Barredo et al. (2020). “Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI”. In: *Information fusion* 58. ISBN: 1566-2535 Publisher: Elsevier, pp. 82–115.
- Arsenault, Ryan et al. (2024). “sTREM2 Differentially Affects Cytokine Expression in Myeloid-Derived Cell Models via MAPK–JNK Signaling Pathway”. In: *Biology* 13.2. Published January 30 2024, p. 87. DOI: 10.3390/biology13020087. URL: <https://www.mdpi.com/2079-7737/13/2/87>.
- Arulkumaran, Kai et al. (2017). “A brief survey of deep reinforcement learning”. In: *arXiv preprint arXiv:1708.05866*.
- Bruckert, Sebastian, Bettina Finzel, and Ute Schmid (2020). “The next generation of medical decision support: A roadmap toward transparent expert companions”. In: *Frontiers in artificial intelligence* 3, p. 507973.
- Cheng, Qisen et al. (2025). “Unifying prediction and explanation in time-series transformers via shapley-based pretraining”. In: *2025 21st IEEE International Colloquium on Signal Processing & Its Applications (CSPA)*. IEEE, pp. 325–330.
- Chudo, Solomon Buke and Gyorgy Terdik (2025). “Modeling and Forecasting Time-Series Data with Multiple Seasonal Periods Using Periodograms”. In: *Econometrics* 13.2, p. 14.
- Crabbé, Jonathan and Mihaela Van Der Schaar (2021). “Explaining Time Series Predictions with Dynamic Masks”. In: *Proceedings of the 38th International Conference on Machine Learning*. Ed. by Marina Meila and Tong Zhang. Vol. 139. Proceedings of Machine Learning Research. PMLR, pp. 2166–2177. URL: <https://proceedings.mlr.press/v139/crabbe21a.html>.
- Del Ser, Javier et al. (2022). “Exploring the trade-off between plausibility, change intensity and adversarial power in counterfactual explanations using multi-objective optimization”. In: *arXiv preprint arXiv:2205.10232*.
- Delaney, Eoin, Derek Greene, and Mark T Keane (2021). “Instance-based counterfactual explanations for time series classification”. In: *International conference on case-based reasoning*. Springer, pp. 32–47.
- Fantozzi, Paolo and Maurizio Naldi (2024). “The explainability of transformers: Current status and directions”. In: *Computers* 13.4, p. 92.
- Institute, TransferLab AppliedAI (2025). *Interpretability by design vs post-hoc methods*. <https://transferlab.ai/series/explainable-ai/>. Accessed 2025.
- Keane, Mark T et al. (2021). “If only we had better counterfactual explanations: Five key deficits to rectify in the evaluation of counterfactual XAI techniques”. In: *arXiv preprint arXiv:2103.01035*.

- Kreuzer, Tim, Jelena Zdravkovic, and Panagiotis Papapetrou (2025). "Unpacking the trend: decomposition as a catalyst to enhance time series forecasting models". In: *Data Mining and Knowledge Discovery* 39, p. 54.
- Li, Peiyu et al. (2026). "Counterfactual Explanations for Time Series Data via Reinforcement Learning". In.
- Liu, Yong et al. (2023). "itransformer: Inverted transformers are effective for time series forecasting". In: *arXiv preprint arXiv:2310.06625*.
- Lundberg, Scott M and Su-In Lee (2017). "A unified approach to interpreting model predictions". In: *Advances in neural information processing systems* 30.
- Nie, Yuqi et al. (2023). "A Time Series is Worth 64 Words: Long-term Forecasting with Transformers". In: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=Jbdc0vT0col>.
- Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin (2016). "Why should i trust you?" Explaining the predictions of any classifier". In: *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, pp. 1135–1144.
- Rojat, Thomas et al. (2021). *Explainable Artificial Intelligence (XAI) on Time Series Data: A Survey*. DOI: 10.48550/arXiv.2104.00950. arXiv: 2104.00950 [cs.LG]. URL: <https://arxiv.org/abs/2104.00950>.
- Sahil, Verma, J Dickerson, and K Hines (2010). "Counterfactual explanations for machine learning: A review". In: *Preprint*.
- Schlegel, Udo et al. (2019). "Towards A Rigorous Evaluation Of XAI Methods On Time Series". In: pp. 4197–4201. DOI: 10.1109/ICCVW.2019.00516.
- Schwalbe, Gesina and Bettina Finzel (2023). "A comprehensive taxonomy for explainable artificial intelligence: a systematic survey of surveys on methods and concepts". In: *Data Mining and Knowledge Discovery* 38, pp. 3043–3101.
- Speith, Timo (2022). "A review of taxonomies of explainable artificial intelligence (XAI) methods". In: *Proceedings of the 2022 ACM conference on fairness, accountability, and transparency*, pp. 2239–2250.
- Vaswani, Ashish et al. (2017). "Attention is all you need". In: *Advances in neural information processing systems* 30.
- Vielhaben, Johanna et al. (2024). "Explainable AI for time series via virtual inspection layers". In: *Pattern Recognition* 150, p. 110309.
- Wachter, Sandra, Brent Mittelstadt, and Chris Russell (2017). "Counterfactual explanations without opening the black box: Automated decisions and the GDPR". In: *Harv. JL & Tech.* 31, p. 841.
- Wan, Renzhuo et al. (2019). "Multivariate temporal convolutional network: A deep neural networks approach for multivariate time series forecasting". In: *Electronics* 8.8, p. 876.
- Wang, Yuxuan et al. (2024). "Deep time series models: A comprehensive survey and benchmark". In: *arXiv preprint arXiv:2407.13278*.
- Wang, Zhendong et al. (2023). "Counterfactual Explanations for Time Series Forecasting". In: *arXiv preprint arXiv:2310.08137*. 10 pages, 6 figures. Accepted by ICDM 2023. DOI: 10.48550/arXiv.2310.08137. URL: <https://arxiv.org/abs/2310.08137>.
- Wen, Qingsong et al. (2022). "Transformers in Time Series: A Survey". In: *ArXiv*. arXiv: 2202.07125 [cs.LG].
- Wu, Haixu et al. (2021). "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting". In: *Advances in neural information processing systems* 34, pp. 22419–22430.
- Zhang, Yunhao and Junchi Yan (2023). "Crossformer: Transformer Utilizing Cross-Dimension Dependency for Multivariate Time Series Forecasting". In: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=vSVLM2j9eie>.

- Zhang, Yuyi et al. (2024). “ShapTime: A General XAI Approach for Explainable Time Series Forecasting”. In: *Intelligent Systems and Applications*. Ed. by Kohei Arai. Cham: Springer Nature Switzerland, pp. 659–673.
- Zhou, Haoyi et al. (2021). “Informer: Beyond efficient transformer for long sequence time-series forecasting”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 35. 12, pp. 11106–11115.
- Zhou, Tian et al. (2022). “Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting”. In: *International conference on machine learning*. PMLR, pp. 27268–27286.